

Computer Networking: A Top-Down Approach Featuring the Internet, 5th Edition

Solutions to Review Questions and Problems

Version Date: 1 September 2009

This document contains the solutions to review questions and problems for the 5th edition of *Computer Networking: A Top-Down Approach Featuring the Internet* by Jim Kurose and Keith Ross. These solutions are being made available to instructors ONLY. Please do NOT copy or distribute this document to others (even other instructors). Please do not post any solutions on a publicly-available Web site. We'll be happy to provide a copy (up-to-date) of this solution manual ourselves to anyone who asks.

Acknowledgments: Over the years, several students and colleagues have helped us prepare this solutions manual. Special thanks goes to Honggang Zhang, Rakesh Kumar, and Prithula Dughel.

All material © copyright 1996-2009 by J.F. Kurose and K.W. Ross. All rights reserved

Chapter 1 Review Questions

1. There is no difference. Throughout this text, the words “host” and “end system” are used interchangeably. End systems include PCs, workstations, Web servers, mail servers, Internet-connected PDAs, WebTVs, etc.
2. Suppose Alice, an ambassador of country A wants to invite Bob, an ambassador of country B, over for dinner. Alice doesn’t simply just call Bob on the phone and say, “come to our dinner table now”. Instead, she calls Bob and suggests a date and time. Bob may respond by saying he’s not available that particular date, but he is available another date. Alice and Bob continue to send “messages” back and forth until they agree on a date and time. Bob then shows up at the embassy on the agreed date, hopefully not more than 15 minutes before or after the agreed time. Diplomatic protocols also allow for either Alice or Bob to politely cancel the engagement if they have reasonable excuses.
3. A networking program usually has two programs, each running on a different host, communicating with each other. The program that initiates the communication is the client. Typically, the client program requests and receives services from the server program.
4. 1. Dial-up modem over telephone line: residential; 2. DSL over telephone line: residential or small office; 3. Cable to HFC: residential; 4. 100 Mbps switched Ethernet: company; 5. Wireless LAN: mobile; 6. Cellular mobile access (for example, WAP): mobile
5. HFC bandwidth is shared among the users. On the downstream channel, all packets emanate from a single source, namely, the head end. Thus, there are no collisions in the downstream channel.
6. Current possibilities include: dial-up; DSL; cable modem; fiber-to-the-home.
7. Ethernet LANs have transmission rates of 10 Mbps, 100 Mbps, 1 Gbps and 10 Gbps. For an X Mbps Ethernet (where X = 10, 100, 1,000 or 10,000), a user can continuously transmit at the rate X Mbps if that user is the only person sending data. If there are more than one active user, then each user cannot continuously transmit at X Mbps.
8. Ethernet most commonly runs over twisted-pair copper wire and “thin” coaxial cable. It also can run over fibers optic links and thick coaxial cable.
9. Dial up modems: up to 56 Kbps, bandwidth is dedicated; ISDN: up to 128 kbps, bandwidth is dedicated; ADSL: downstream channel is .5-8 Mbps, upstream channel is up to 1 Mbps, bandwidth is dedicated; HFC, downstream channel is 10-30 Mbps and upstream channel is usually less than a few Mbps, bandwidth is shared. FTTH: 2-10Mbps upload; 10-20 Mbps download; bandwidth is not shared.

10. There are two most popular wireless Internet access technologies today:
- Wireless LAN**
In a wireless LAN, wireless users transmit/receive packets to/from a base station (wireless access point) within a radius of few tens of meters. The base station is typically connected to the wired Internet and thus serves to connect wireless users to the wired network.
 - Wide-area wireless access network**
In these systems, packets are transmitted over the same wireless infrastructure used for cellular telephony, with the base station thus being managed by a telecommunications provider. This provides wireless access to users within a radius of tens of kilometers of the base station.
11. A circuit-switched network can guarantee a certain amount of end-to-end bandwidth for the duration of a call. Most packet-switched networks today (including the Internet) cannot make any end-to-end guarantees for bandwidth.
12. In a packet switched network, the packets from different sources flowing on a link do not follow any fixed, pre-defined pattern. In TDM circuit switching, each host gets the same slot in a revolving TDM frame.
13. At time t_0 the sending host begins to transmit. At time $t_1 = L/R_1$, the sending host completes transmission and the entire packet is received at the router (no propagation delay). Because the router has the entire packet at time t_1 , it can begin to transmit the packet to the receiving host at time t_1 . At time $t_2 = t_1 + L/R_2$, the router completes transmission and the entire packet is received at the receiving host (again, no propagation delay). Thus, the end-to-end delay is $L/R_1 + L/R_2$.
14. A tier-1 ISP connects to all other tier-1 ISPs; a tier-2 ISP connects to only a few of the tier-1 ISPs. Also, a tier-2 ISP is a customer of one or more tier-1.
15. a) 2 users can be supported because each user requires half of the link bandwidth.
- Since each user requires 1Mbps when transmitting, if two or fewer users transmit simultaneously, a maximum of 2Mbps will be required. Since the available bandwidth of the shared link is 2Mbps, there will be no queuing delay before the link. Whereas, if three users transmit simultaneously, the bandwidth required will be 3Mbps which is more than the available bandwidth of the shared link. In this case, there will be queuing delay before the link.
 - Probability that a given user is transmitting = 0.2
 - Probability that all three users are transmitting simultaneously = $\binom{3}{3} p^3 (1-p)^{3-3}$

$= (0.2)^3 = 0.008$. Since the queue grows when all the users are transmitting, the fraction of time during which the queue grows (which is equal to the probability that all three users are transmitting simultaneously) is 0.008.

16. The delay components are processing delays, transmission delays, propagation delays, and queuing delays. All of these delays are fixed, except for the queuing delays, which are variable.

17. Java Applet

18. 10msec; d/s; no; no

19. a) 500 kbps
b) 64 seconds
c) 100kbps; 320 seconds

20. End system A breaks the large file into chunks. To each chunk, it adds header generating multiple packets from the file. The header in each packet includes the address of the destination: end system B. The packet switch uses the destination address to determine the outgoing link. Asking which road to take is analogous to a packet asking which outgoing link it should be forwarded on, given the packet's address.

21. Java Applet

22. Five generic tasks are error control, flow control, segmentation and reassembly, multiplexing, and connection setup. Yes, these tasks can be duplicated at different layers. For example, error control is often provided at more than one layer.

23. The five layers in the Internet protocol stack are – from top to bottom – the application layer, the transport layer, the network layer, the link layer, and the physical layer. The principal responsibilities are outlined in Section 1.5.1.

24. Application-layer message: data which an application wants to send and passed onto the transport layer; transport-layer segment: generated by the transport layer and encapsulates application-layer message with transport layer header; network-layer datagram: encapsulates transport-layer segment with a network-layer header; link-layer frame: encapsulates network-layer datagram with a link-layer header.

25. Routers process layers 1 through 3. (This is a little bit of a white lie, as modern routers sometimes act as firewalls or caching components, and process layer four as well.) Link layer switches process layers 1 through 2. Hosts process all five layers.

26. a) Virus
Requires some form of human interaction to spread. Classic example: E-mail viruses.

b)Worms

No user replication needed. Worm in infected host scans IP addresses and port numbers, looking for vulnerable processes to infect.

c) Trojan horse

Hidden, devious part of some otherwise useful software.

27. Creation of a botnet requires an attacker to find vulnerability in some application or system (e.g. exploiting the buffer overflow vulnerability that might exist in an application). After finding the vulnerability, the attacker needs to scan for hosts that are vulnerable. The target is basically to compromise a series of systems by exploiting that particular vulnerability. Any system that is part of the botnet can automatically scan its environment and propagate by exploiting the vulnerability. An important property of such botnets is that the originator of the botnet can remotely control and issue commands to all the nodes in the botnet. Hence, it becomes possible for the attacker to issue a command to all the nodes, that target a single node (for example, all nodes in the botnet might be commanded by the attacker to send a TCP SYN message to the target, which might result in a TCP SYN flood attack at the target).
28. Trudy can pretend to be Bob to Alice (and vice-versa) and partially or completely modify the message(s) being sent from Bob to Alice. For example, she can easily change the phrase "Alice, I owe you \$1000" to "Alice, I owe you \$10,000". Furthermore, Trudy can even drop the packets that are being sent by Bob to Alice (and vice-versa), even if the packets from Bob to Alice are encrypted.

Chapter 1 Problems

Problem 1

There is no single right answer to this question. Many protocols would do the trick. Here's a simple answer below:

Messages from ATM machine to Server

Msg name	purpose
-----	-----
HELO <userid>	Let server know that there is a card in the ATM machine
	ATM card transmits user ID to Server
PASSWD <passwd>	User enters PIN, which is sent to server
BALANCE	User requests balance
WITHDRAWL <amount>	User asks to withdraw money
BYE	user all done

Messages from Server to ATM machine (display)

Msg name	purpose
-----	-----
PASSWD	Ask user for PIN (password)
OK	last requested operation (PASSWD, WITHDRAWL)
ERR	OK
	last requested operation (PASSWD, WITHDRAWL)
AMOUNT <amt>	in ERROR
BYE	sent in response to BALANCE request
	user done, display welcome screen at ATM

Correct operation:

client		server
HELO (userid)	----->	(check if valid userid)
	<-----	PASSWD
PASSWD <passwd>	----->	(check password)
	<-----	OK (password is OK)
BALANCE	----->	
	<-----	AMOUNT <amt>
WITHDRAWL <amt>	----->	check if enough \$ to cover
	<-----	withdrawl
		OK
ATM dispenses \$		
BYE	----->	
	<-----	BYE

In situation when there's not enough money:

HELO (userid)	----->	(check if valid userid)
	<-----	PASSWD
PASSWD <passwd>	----->	(check password)
	<-----	OK (password is OK)
BALANCE	----->	
	<-----	AMOUNT <amt>
WITHDRAWL <amt>	----->	check if enough \$ to cover
withdrawl	<-----	ERR (not enough funds)
error msg displayed		
no \$ given out		
BYE	----->	
	<-----	BYE

Problem 2

- a) A circuit-switched network would be well suited to the application described, because the application involves long sessions with predictable smooth bandwidth requirements. Since the transmission rate is known and not bursty, bandwidth can be

reserved for each application session circuit with no significant waste. In addition, we need not worry greatly about the overhead costs of setting up and tearing down a circuit connection, which are amortized over the lengthy duration of a typical application session.

- b) Given such generous link capacities, the network needs no congestion control mechanism. In the worst (most potentially congested) case, all the applications simultaneously transmit over one or more particular network links. However, since each link offers sufficient bandwidth to handle the sum of all of the applications' data rates, no congestion (very little queuing) will occur.

Problem 3

- a) We can have n connections between each of the four pairs of adjacent switches. This gives a maximum of $4n$ connections.
- b) We can n connections passing through the switch in the upper-right-hand corner and another n connections passing through the switch in the lower-left-hand corner, giving a total of $2n$ connections.

Problem 4

Tollbooths are 75 km apart, and the cars propagate at 100km/hr. A tollbooth services a car at a rate of one car every 12 seconds.

- a) There are ten cars. It takes 120 seconds, or 2 minutes, for the first tollbooth to service the 10 cars. Each of these cars has a propagation delay of 45 minutes (travel 75 km) before arriving at the second tollbooth. Thus, all the cars are lined up before the second tollbooth after 47 minutes. The whole process repeats itself for traveling between the second and third tollbooths. It also takes 2 minutes for the third tollbooth to service the 10 cars. Thus the total delay is 96 minutes.
- b) Delay between tollbooths is 8×12 seconds plus 45 minutes, i.e., 46 minutes and 36 seconds. The total delay is twice this amount plus 8×12 seconds, i.e., 94 minutes and 48 seconds.

Problem 5

- a) $d_{prop} = m / s$ seconds.
- b) $d_{trans} = L / R$ seconds.
- c) $d_{end-to-end} = (m / s + L / R)$ seconds.
- d) The bit is just leaving Host A.
- e) The first bit is in the link and has not reached Host B.
- f) The first bit has reached Host B.
- g) Want

$$m = \frac{L}{R} s = \frac{120}{56 \times 10^3} (2.5 \times 10^8) = 536 \text{ km.}$$

Problem 6

Consider the first bit in a packet. Before this bit can be transmitted, all of the bits in the packet must be generated. This requires

$$\frac{56 \cdot 8}{64 \times 10^3} \text{ sec} = 7 \text{ msec.}$$

The time required to transmit the packet is

$$\frac{56 \cdot 8}{2 \times 10^6} \text{ sec} = 224 \mu \text{ sec.}$$

Propagation delay = 10 msec.

The delay until decoding is

$$7 \text{ msec} + 224 \mu \text{ sec} + 10 \text{ msec} = 17.224 \text{ msec}$$

A similar analysis shows that all bits experience a delay of 17.224 msec.

Problem 7

a) 20 users can be supported.

b) $p = 0.1$.

$$\text{c) } \binom{120}{n} p^n (1-p)^{120-n}.$$

$$\text{d) } 1 - \sum_{n=0}^{20} \binom{120}{n} p^n (1-p)^{120-n}.$$

We use the central limit theorem to approximate this probability. Let X_j be independent random variables such that $P(X_j = 1) = p$.

$$P(\text{“21 or more users”}) = 1 - P\left(\sum_{j=1}^{120} X_j \leq 21\right)$$

$$P\left(\sum_{j=1}^{120} X_j \leq 21\right) = P\left(\frac{\sum_{j=1}^{120} X_j - 12}{\sqrt{120 \cdot 0.1 \cdot 0.9}} \leq \frac{9}{\sqrt{120 \cdot 0.1 \cdot 0.9}}\right)$$

$$\approx P\left(Z \leq \frac{9}{3.286}\right) = P(Z \leq 2.74) \\ = 0.997$$

when Z is a standard normal r.v. Thus $P(\text{"21 or more users"}) \approx 0.003$.

Problem 8

a) 10,000

b)
$$\sum_{n=N+1}^M \binom{M}{n} p^n (1-p)^{M-n}$$

Problem 9

The first end system requires L/R_1 to transmit the packet onto the first link; the packet propagates over the first link in d_1/s_1 ; the packet switch adds a processing delay of d_{proc} ; after receiving the entire packet, the packet switch connecting the first and the second link requires L/R_2 to transmit the packet onto the second link; the packet propagates over the second link in d_2/s_2 . Similarly, we can find the delay caused by the second switch and the third link: L/R_3 , d_{proc} , and d_3/s_3 .

Adding these five delays gives

$$d_{end-end} = L/R_1 + L/R_2 + L/R_3 + d_1/s_1 + d_2/s_2 + d_3/s_3 + d_{proc} + d_{proc}$$

To answer the second question, we simply plug the values into the equation to get $6 + 6 + 6 + 20 + 16 + 4 + 3 + 3 = 64$ msec.

Problem 10

Because bits are immediately transmitted, the packet switch does not introduce any delay; in particular, it does not introduce a transmission delay. Thus,

$$d_{end-end} = L/R + d_1/s_1 + d_2/s_2 + d_3/s_3$$

For the values in Problem 9, we get $6 + 20 + 16 + 4 = 46$ msec.

Problem 11

The arriving packet must first wait for the link to transmit 6,750 bytes or 54,000 bits. Since these bits are transmitted at 2 Mbps, the queuing delay is 27 msec. Generally, the queuing delay is $(nL + (L - x))/R$.

Problem 12

The queuing delay is 0 for the first transmitted packet, L/R for the second transmitted packet, and generally, $(n-1)L/R$ for the n^{th} transmitted packet. Thus, the average delay for the N packets is

$$\begin{aligned} & (L/R + 2L/R + \dots + (N-1)L/R)/N \\ &= L/(RN) * (1 + 2 + \dots + (N-1)) \\ &= L/(RN) * N(N-1)/2 \\ &= LN(N-1)/(2RN) \\ &= (N-1)L/(2R) \end{aligned}$$

Note that here we used the well-known fact that

$$1 + 2 + \dots + N = N(N+1)/2$$

Problem 13

It takes LN/R seconds to transmit the N packets. Thus, the buffer is empty when a batch of N packets arrive.

The first of the N packets has no queuing delay. The 2nd packet has a queuing delay of L/R seconds. The n^{th} packet has a delay of $(n-1)L/R$ seconds.

The average delay is

$$\frac{1}{N} \sum_{n=1}^N (n-1)L/R = \frac{L}{R} \frac{1}{N} \sum_{n=0}^{N-1} n = \frac{L}{R} \frac{1}{N} \frac{(N-1)N}{2} = \frac{L}{R} \frac{(N-1)}{2}.$$

Problem 14

a) The transmission delay is L/R . The total delay is

$$\frac{IL}{R(1-I)} + \frac{L}{R} = \frac{L/R}{1-I}$$

b) Let $x = L/R$.

$$\text{Total delay} = \frac{x}{1-ax}$$

Problem 15

$$\text{Total delay} = \frac{L/R}{1-I} = \frac{L/R}{1-aL/R} = \frac{1/\mu}{1-a/\mu} = \frac{1}{\mu-a}.$$

Problem 16

The total number of packets in the system includes those in the buffer and the pack is being transmitted. So, $N=10+1$.

Because $N = a \cdot d$, so $(10+1)=a \cdot (\text{queuing delay} + \text{transmission delay})$. That is, $11=a \cdot (0.01+1/100)=a \cdot (0.01+0.01)$, thus, $a=550$ packets/sec.

Problem 17

a) There are Q nodes (the source host and the $Q-1$ routers). Let d_{proc}^q denote the processing delay at the q th node. Let R^q be the transmission rate of the q th link and let $d_{trans}^q = L / R^q$. Let d_{prop}^q be the propagation delay across the q th link. Then

$$d_{end-to-end} = \sum_{q=1}^Q [d_{proc}^q + d_{trans}^q + d_{prop}^q].$$

b) Let d_{queue}^q denote the average queuing delay at node q . Then

$$d_{end-to-end} = \sum_{q=1}^Q [d_{proc}^q + d_{trans}^q + d_{prop}^q + d_{queue}^q].$$

Problem 18

The command:

```
tracert -q 20 www.eurecom.fr
```

will get 20 delay measurements from the issuing host to the host, www.eurecom.fr. The average and standard deviation of these 20 measurements can then be collected. Do you see any differences in your answers as a function of time of day?

Problem 19

Throughput = $\min\{R_s, R_c, R/M\}$

Problem 20

If only use one path, the max throughput is given by

$$\max \{ \min \{ R_1^1, R_2^1, \dots, R_N^1 \}, \min \{ R_1^2, R_2^2, \dots, R_N^2 \}, \dots, \min \{ R_1^M, R_2^M, \dots, R_N^M \} \}.$$

If use all paths, the max throughput is given by $\sum_{k=1}^M \min \{R_1^k, R_2^k, \dots, R_N^k\}$.

Problem 21

Probability of successfully receiving a packet is:

$$p_s = (1-p)^N.$$

The number of transmissions needs to be performed until the packet is successfully received by the client is a geometric random variable with success probability p_s .

Thus, the average number of transmissions needed is given by: $1/p_s$

Then, the average number of re-transmissions needed is given by: $1/p_s - 1$.

Problem 22

Lets call the first packet A and call the second packet B.

a). If the bottleneck link is the first link, then, packet B is queued at the first link waiting for the transmission of packet A. So, the packet inter-arrival time at the destination is simply L/R_s .

b). If the second link is the bottleneck link and since both packets are sent back to back, it must be true that the second packet arrives at the input queue of the second link before the second link finishes the transmission of the first packet. That is,

$$L/R_s + L/R_s + d_{prop} < L/R_s + d_{prop} + L/R_c \quad (1)$$

The left hand side of the above inequality represents the time needed by the second packet to *arrive at* the input queue of the second link (the second link has not started transmitting the second packet yet).

The right hand side represents the time needed by the first packet to finish its transmission onto the second link.

We know that (1) is possible as $R_c < R_s$. And it is clear that (1) shows that the second packet must have queuing delay at the input queue of the second link.

If we send the second packet T seconds later, then we can ensure there is no queuing delay for the second packet at the second link if we have,

$$L/R_s + L/R_s + d_{prop} + T \geq L/R_s + d_{prop} + L/R_c$$

Thus, T must be $L/R_c - L/R_s$.

Problem 23

40 terabytes = $40 * 10^{12} * 8$ bits.

So, if using the dedicated link, it will take $40 * 10^{12} * 8 / (100 * 10^6) = 3200000$ seconds = 37 days.

But with FedEx overnight delivery, you can guarantee the data arrives in one day, and it only costs you no more than \$100.

Problem 24

- a) 160,000 bits
- b) 160,000 bits
- c) The bandwidth-delay product of a link is the maximum number of bits that can be in the link.
- d) the width of a bit = length of link / bandwidth-delay product, so 1 bit is 125 meters long, which is longer than a football field
- e) s/R

Problem 25

$s/R=20000\text{km}$, then $R=s/20000\text{km}=2.5*10^8/(2*10^7)=12.5\text{ bps}$

Problem 26

- a) 80,000,000 bits
- b) 800,000 bits, this is because that the maximum number of bits that will be in the link at any given time = $\min(\text{bandwidth delay product, packet size}) = 800,000\text{ bits}$.
- c) .25 meters

Problem 27

- a) $t_{trans} + t_{prop} = 400\text{ msec} + 80\text{ msec} = 480\text{ msec}$
- b) $20 * (t_{trans} + 2 t_{prop}) = 20*(20\text{ msec} + 80\text{ msec}) = 2\text{ sec}$

Problem 28

Recall geostationary satellite is 36,000 kilometers away from earth surface.

- a) 150 msec
- b) 1,500,000 bits
- c) 600,000,000 bits

Problem 29

Let's suppose the passenger and his/her bags correspond to the data unit arriving to the top of the protocol stack. When the passenger checks in, his/her bags are checked, and a

tag is attached to the bags and ticket. This is additional information added in the Baggage layer of Figure 1.20 that allows the Baggage layer to implement the service of separating the passengers and baggage on the sending side, and then reuniting them (hopefully!) on the destination side. When a passenger then passes through security, and additional stamp is often added to his/her ticket, indicating that the passenger has passed through a security check. This information is used to ensure (e.g., by later checks for the security information) secure transfer of people.

Problem 30

- a) Time to send message from source host to first packet switch = $\frac{8 \times 10^6}{2 \times 10^6} \text{ sec} = 4 \text{ sec}$. With store-and-forward switching, the total time to move message from source host to destination host = $4 \text{ sec} \times 3 \text{ hops} = 12 \text{ sec}$
- b) Time to send 1st packet from source host to first packet switch = $\frac{2 \times 10^3}{2 \times 10^6} \text{ sec} = 1 \text{ msec}$. Time at which 2nd packet is received at the first switch = time at which 1st packet is received at the second switch = $2 \times 1 \text{ msec} = 2 \text{ msec}$
- c) Time at which 1st packet is received at the destination host = $1 \text{ msec} \times 3 \text{ hops} = 3 \text{ msec}$. After this, every 1 msec one packet will be received; thus time at which last (4000th) packet is received = $3 \text{ msec} + 3999 \times 1 \text{ msec} = 4.002 \text{ sec}$. It can be seen that delay in using message segmentation is significantly less (almost 1/3rd).
- d) Drawbacks:
 - i. Packets have to be put in sequence at the destination.
 - ii. Message segmentation results in many smaller packets. Since header size is usually the same for all packets regardless of their size, with message segmentation the total amount of header bytes is more.

Problem 31

Java Applet

Problem 32

Time at which the 1st packet is received at the destination = $\frac{S+80}{R} \times 3$ sec. After this, one

packet is received at destination every $\frac{S+80}{R}$ sec. Thus delay in sending the whole file =

$$delay = \frac{S+80}{R} \times 3 + \left(\frac{F}{S} - 1\right) \times \left(\frac{S+80}{R}\right) = \frac{S+80}{R} \times \left(\frac{F}{S} + 2\right)$$

To calculate the value of S which leads to the minimum delay,

$$\frac{d}{dS} delay = 0 \Rightarrow S = \sqrt{40F}$$

Chapter 2 Review Questions

1. The Web: HTTP; file transfer: FTP; remote login: Telnet; Network News: NNTP; e-mail: SMTP.
2. Network architecture refers to the organization of the communication process into layers (e.g., the five-layer Internet architecture). Application architecture, on the other hand, is designed by an application developer and dictates the broad structure of the application (e.g., client-server or P2P)
3. The process which initiates the communication is the client; the process that waits to be contacted is the server.
4. No. As stated in the text, all communication sessions have a client side and a server side. In a P2P file-sharing application, the peer that is receiving a file is typically the client and the peer that is sending the file is typically the server.
5. The IP address of the destination host and the port number of the destination socket.
6. You would use UDP. With UDP, the transaction can be completed in one roundtrip time (RTT) - the client sends the transaction request into a UDP socket, and the server sends the reply back to the client's UDP socket. With TCP, a minimum of two RTTs are needed - one to set-up the TCP connection, and another for the client to send the request, and for the server to send back the reply.
7. There are no good examples of an application that requires no data loss and timing. If you know of one, send an e-mail to the authors.
8. a) Reliable data transfer
TCP provides a reliable byte-stream between client and server but UDP does not.

b) A guarantee that a certain value for throughput will be maintained
Neither

c) A guarantee that data will be delivered within a specified amount of time
Neither

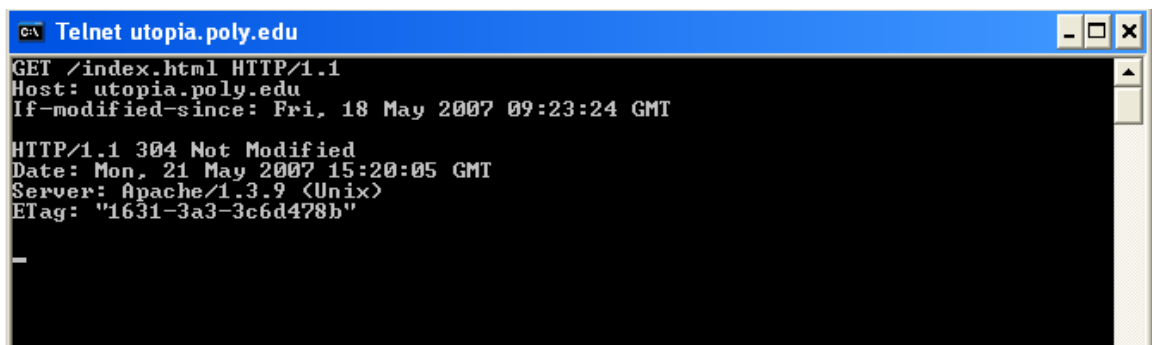
d) Security
Neither
9. SSL operates at the application layer. The SSL socket takes unencrypted data from the application layer, encrypts it and then passes it to the TCP socket. If the

application developer wants TCP to be enhanced with SSL, she has to include the SSL code in the application.

10. A protocol uses handshaking if the two communicating entities first exchange control packets before sending data to each other. SMTP uses handshaking at the application layer whereas HTTP does not.
11. The applications associated with those protocols require that all application data be received in the correct order and without gaps. TCP provides this service whereas UDP does not.
12. When the user first visits the site, the site returns a cookie number. This cookie number is stored on the user's host and is managed by the browser. During each subsequent visit (and purchase), the browser sends the cookie number back to the site. Thus the site knows when this user (more precisely, this browser) is visiting the site.
13. Web caching can bring the desired content "closer" to the user, perhaps to the same LAN to which the user's host is connected. Web caching can reduce the delay for all objects, even objects that are not cached, since caching reduces the traffic on links.
14. Issued the following command (in Windows command prompt) followed by the HTTP GET message to the "utopia.poly.edu" web server:

```
> telnet utopia.poly.edu 80
```

Since the index.html page in this web server was not modified since Fri, 18 May 2007 09:23:34 GMT, the following output was displayed when the above commands were issued on Sat, 19 May 2007. Note that the first 4 lines are the GET message and header lines input by the user and the next 4 lines (starting from HTTP/1.1 304 Not Modified) is the response from the web server.



```
C:\> Telnet utopia.poly.edu
GET /index.html HTTP/1.1
Host: utopia.poly.edu
If-modified-since: Fri, 18 May 2007 09:23:24 GMT

HTTP/1.1 304 Not Modified
Date: Mon, 21 May 2007 15:20:05 GMT
Server: Apache/1.3.9 (Unix)
ETag: "1631-3a3-3c6d478b"
```

15. FTP uses two parallel TCP connections, one connection for sending control information (such as a request to transfer a file) and another connection for actually transferring the file. Because the control information is not sent over the

same connection that the file is sent over, FTP sends control information out of band.

16. Message is sent from Alice's host to her mail server over HTTP. Alice's mail server then sends the message to Bob's mail server over SMTP. Bob then transfers the message from his mail server to his host over POP3.

17.

Received: from 65.54.246.203 (EHLO bay0-omc3-s3.bay0.hotmail.com)
(65.54.246.203) by mta419.mail.mud.yahoo.com with SMTP; Sat, 19
May 2007 16:53:51 -0700
Received: from hotmail.com ([65.55.135.106]) by bay0-omc3-s3.bay0.hotmail.com
with Microsoft SMTPSVC(6.0.3790.2668); Sat, 19 May 2007 16:52:42 -
0700
Received: from mail pickup service by hotmail.com with Microsoft SMTPSVC; Sat,
19 May 2007 16:52:41 -0700
Message-ID: <BAY130-F26D9E35BF59E0D18A819AFB9310@phx.gbl>
Received: from 65.55.135.123 by by130fd.bay130.hotmail.msn.com with HTTP;
Sat, 19 May 2007 23:52:36 GMT
From: "prithula dhungel" <prithuladhungel@hotmail.com>
To: prithula@yahoo.com
Bcc:
Subject: Test mail
Date: Sat, 19 May 2007 23:52:36 +0000
Mime-Version: 1.0
Content-Type: Text/html; format=flowed
Return-Path: prithuladhungel@hotmail.com

Figure: A sample mail message header

Received: This header field indicates the sequence in which the SMTP servers send and receive the mail message including the respective timestamps.

In this example there are 4 "Received:" header lines. This means the mail message passed through 5 different SMTP servers before being delivered to the receiver's mail box. The last (forth) "Received:" header indicates the mail message flow from the SMTP server of the sender to the second SMTP server in the chain of servers. The sender's SMTP server is at address 65.55.135.123 and the second SMTP server in the chain is by130fd.bay130.hotmail.msn.com.

The third "Received:" header indicates the mail message flow from the second SMTP server in the chain to the third server, and so on.

Finally, the first "Received:" header indicates the flow of the mail message from the forth SMTP server to the last SMTP server (i.e. the receiver's mail server) in the chain.

Message-id: The message has been given this number BAY130-F26D9E35BF59E0D18A819AFB9310@phx.gbl (by bay0-omc3-s3.bay0.hotmail.com. Message-id is a unique string assigned by the mail system when the message is first created.

From: This indicates the email address of the sender of the mail. In the given example, the sender is “prithuladhungel@hotmail.com”

To: This field indicates the email address of the receiver of the mail. In the example, the receiver is “prithula@yahoo.com”

Subject: This gives the subject of the mail (if any specified by the sender). In the example, the subject specified by the sender is “Test mail”

Date: The date and time when the mail was sent by the sender. In the example, the sender sent the mail on 19th May 2007, at time 23:52:36 GMT.

Mime-version: MIME version used for the mail. In the example, it is 1.0.

Content-type: The type of content in the body of the mail message. In the example, it is “text/html”.

Return-Path: This specifies the email address to which the mail will be sent if the receiver of this mail wants to reply to the sender. This is also used by the sender’s mail server for bouncing back undeliverable mail messages of mailer-daemon error messages. In the example, the return path is “prithuladhungel@hotmail.com”.

18. With download and delete, after a user retrieves its messages from a POP server, the messages are deleted. This poses a problem for the nomadic user, who may want to access the messages from many different machines (office PC, home PC, etc.). In the download and keep configuration, messages are not deleted after the user retrieves the messages. This can also be inconvenient, as each time the user retrieves the stored messages from a new machine, all of non-deleted messages will be transferred to the new machine (including very old messages).
19. Yes an organization’s mail server and Web server can have the same alias for a host name. The MX record is used to map the mail server’s host name to its IP address.
20. It is not necessary that Bob will also provide chunks to Alice. Alice has to be in the top 4 neighbors of Bob for Bob to send out chunks to her; this might not occur even if Alice is provides chunks to Bob throughout a 30-second interval.
21. Alice will get her first chunk as a result of she being selected by one of her neighbors as a result of an “optimistic unchoke,” for sending out chunks to her.

22. The overlay network in a P2P file sharing system consists of the nodes participating in the file sharing system and the logical links between the nodes. There is a logical link (an “edge” in graph theory terms) from node A to node B if there is a semi-permanent TCP connection between A and B. An overlay network does not include routers. With Gnutella, when a node wants to join the Gnutella network, it first discovers (“out of band”) the IP address of one or more nodes already in the network. It then sends join messages to these nodes. When the node receives confirmations, it becomes a member of the of Gnutella network. Nodes maintain their logical links with periodic refresh messages.
23. It is a hybrid of client server and P2P architectures:
- a) There is a centralized component (the index) like in the case of a client server system.
 - b) Other functions (except the indexing) do not use any kind of central server. This is similar to what exists in a P2P system.
24. Mesh DHT: The advantage is to a route a message to the peer closest to the key, only one hop is required; the disadvantage is that each peer must track all other peers in the in the DHT. Circular DHT: the advantage is that each peer needs to track only a few other peers; the disadvantage is that $O(N)$ hops are needed to route a message to a peer responsible for the key.
25. a) User location
b) NAT traversal
26. a) File Distribution
b) Instant Messaging
c) Video Streaming
d) Distributed Computing
27. With the UDP server, there is no welcoming socket, and all data from different clients enters the server through this one socket. With the TCP server, there is a welcoming socket, and each time a client initiates a connection to the server, a new socket is created. Thus, to support n simultaneous connections, the server would need $n+1$ sockets.
28. For the TCP application, as soon as the client is executed, it attempts to initiate a TCP connection with the server. If the TCP server is not running, then the client will fail to make a connection. For the UDP application, the client does not initiate connections (or attempt to communicate with the UDP server) immediately upon execution

Chapter 2 Problems

Problem 1

- a) F
- b) T
- c) F
- d) F
- e) F

Problem 2

Access control commands:

USER, PASS, ACT, CWD, CDUP, SMNT, REIN, QUIT.

Transfer parameter commands:

PORT, PASV, TYPE STRU, MODE.

Service commands:

RETR, STOR, STOU, APPE, ALLO, REST, RNFR, RNT0, ABOR, DELE, RMD, MRD, PWD, LIST, NLST, SITE, SYST, STAT, HELP, NOOP.

Problem 3

Application layer protocols: DNS and HTTP

Transport layer protocols: UDP for DNS; TCP for HTTP

Problem 4

- a) The document request was `http://gaia.cs.umass.edu/cs453/index.html`. The Host : field indicates the server's name and `/cs453/index.html` indicates the file name.
- b) The browser is running HTTP version 1.1, as indicated just before the first `<cr><lf>` pair.
- c) The browser is requesting a persistent connection, as indicated by the Connection: keep-alive.
- d) This is a trick question. This information is not contained in an HTTP message anywhere. So there is no way to tell this from looking at the exchange of HTTP messages alone. One would need information from the IP datagrams (that carried the TCP segment that carried the HTTP GET request) to answer this question.

e) Mozilla/5.0. The browser type information is needed by the server to send different versions of the same object to different types of browsers.

Problem 5

- a) The status code of 200 and the phrase OK indicate that the server was able to locate the document successfully. The reply was provided on Tuesday, 07 Mar 2008 12:39:45 Greenwich Mean Time.
- b) The document index.html was last modified on Saturday 10 Dec 2005 18:27:46 GMT.
- c) There are 3874 bytes in the document being returned.
- d) The first five bytes of the returned document are : <!doc. The server agreed to a persistent connection, as indicated by the Connection: Keep-Alive field

Problem 6

- a) Persistent connections are discussed in section 8 of RFC 2616 (the real goal of this question was to get you to retrieve and read an RFC). Sections 8.1.2 and 8.1.2.1 of the RFC indicate that either the client or the server can indicate to the other that it is going to close the persistent connection. It does so by including the including the connection-token "close" in the Connection-header field of the http request/reply.
- b) HTTP does not provide any encryption services.
- c) (From RFC 2616) "Clients that use persistent connections should limit the number of simultaneous connections that they maintain to a given server. A single-user client SHOULD NOT maintain more than 2 connections with any server or proxy."
- d) Yes. (From RFC 2616) "A client might have started to send a new request at the same time that the server has decided to close the "idle" connection. From the server's point of view, the connection is being closed while it was idle, but from the client's point of view, a request is in progress."

Problem 7

The total amount of time to get the IP address is

$$RTT_1 + RTT_2 + \dots + RTT_n .$$

Once the IP address is known, RTT_o elapses to set up the TCP connection and another RTT_o elapses to request and receive the small object. The total response time is

$$2RTT_o + RTT_1 + RTT_2 + \dots + RTT_n$$

Problem 8

a)

$$RTT_1 + \dots + RTT_n + 2RTT_o + 8 \cdot 2RTT_o \\ = 18RTT_o + RTT_1 + \dots + RTT_n.$$

b)

$$RTT_1 + \dots + RTT_n + 2RTT_o + 2 \cdot 2RTT_o \\ = 6RTT_o + RTT_1 + \dots + RTT_n$$

c)

$$RTT_1 + \dots + RTT_n + 2RTT_o + RTT_o \\ = 3RTT_o + RTT_1 + \dots + RTT_n.$$

Problem 9

- a) The time to transmit an object of size L over a link of rate R is L/R . The average time is the average size of the object divided by R :

$$\Delta = (850,000 \text{ bits}) / (15,000,000 \text{ bits/sec}) = .0567 \text{ sec}$$

The traffic intensity on the link is given by $\beta\Delta = (16 \text{ requests/sec})(.0567 \text{ sec/request}) = 0.907$. Thus, the average access delay is $(.0567 \text{ sec}) / (1 - .907) \approx .6 \text{ seconds}$. The total average response time is therefore $.6 \text{ sec} + 3 \text{ sec} = 3.6 \text{ sec}$.

- b) The traffic intensity on the access link is reduced by 60% since the 60% of the requests are satisfied within the institutional network. Thus the average access delay is $(.0567 \text{ sec}) / [1 - (.4)(.907)] = .089 \text{ seconds}$. The response time is approximately zero if the request is satisfied by the cache (which happens with probability .6); the average response time is $.089 \text{ sec} + 3 \text{ sec} = 3.089 \text{ sec}$ for cache misses (which happens 40% of the time). So the average response time is $(.6)(0 \text{ sec}) + (.4)(3.089 \text{ sec}) = 1.24 \text{ seconds}$. Thus the average response time is reduced from 3.6 sec to 1.24 sec.

Problem 10

Note that each downloaded object can be completely put into one data packet. Let T_p denote the one-way propagation delay between the client and the server.

First consider parallel downloads via non-persistent connections. Parallel download would allow 10 connections share the 150 bits/sec bandwidth, thus each gets just 15 bits/sec. Thus, the total time needed to receive all objects is given by:

$$(200/150 + T_p + 200/150 + T_p + 200/150 + T_p + 100,000/150 + T_p) \\ + (200/(150/10) + T_p + 200/(150/10) + T_p + 200/(150/10) + T_p + 100,000/(150/10) + T_p) \\ = 7377 + 8 \cdot T_p \text{ (seconds)}$$

Then consider persistent HTTP connection. The total time needed is give by:

$$(200/150+T_p + 200/150 +T_p + 200/150+T_p + 100,000/150+ T_p) \\ + 10*(200/150+T_p + 100,000/150+ T_p) \\ =7351 + 24*T_p \text{ (seconds)}$$

Assume the speed of light is $300*10^6$ m/sec, then $T_p=10/(300*10^6)=0.03$ microsec. T_p is negligible compared with transmission delay.

Thus, we see that the persistent HTTP does not have significant gain (less than 1 percent) over the non-persistent case with parallel download.

Problem 11

- a). Yes, because Bob has more connections, so he can proportionally get more aggregate bandwidth share out of the total link bandwidth.
- b) Yes, Bob still needs to perform parallel download, otherwise he will get less bandwidth share than other four users. In fact, all users might tend to open more connections in order to gain more bandwidth share.

Problem 12

TCPServer.java

```
import java.io.*;
import java.net.*;

class TCPServer {
    public static void main(String argv[]) throws Exception
    {
        String clientSentence;
        ServerSocket welcomeSocket = new ServerSocket(6789);
        while(true) {
            Socket connectionSocket = welcomeSocket.accept();

            BufferedReader inFromClient = new BufferedReader(new
                InputStreamReader(connectionSocket.getInputStream( ) ));

            clientSentence = inFromClient.readLine();

            System.out.println("RECEIVED FROM CLIENT : " +
                clientSentence + "\n");
        }
    }
}
```


Problem 13

The MAIL FROM: in SMTP is a message from the SMTP client that identifies the sender of the mail message to the SMTP server. The From: on the mail message itself is NOT an SMTP message, but rather is just a line in the body of the mail message.

Problem 14

SMTP uses a line containing only a period to mark the end of a message body. HTTP uses "Content-Length header field" to indicate the length of a message body. No, HTTP cannot use the method used by SMTP, because HTTP message could be binary data, whereas in SMTP, the message body must be in 7-bit ASCII format.

Problem 15

MTA stands for Mail Transfer Agents. A mail is forwarded by a source to a MTA and then it follows a sequence of MTAs to reach the receiver's mail reader. We see that this spam email follows a chain of MTAs. An honest MTA should report where it receives the message. Notice that in this email, "asusus-4b96 ([58.88.21.177])" does not report where it receives the email. As we assume that the only the originator is dishonest, so "asusus-4b96 ([58.88.21.177])" must be the originator.

Problem 16

UIDL abbreviates "unique-ID listing". When a POP3 client issues the UIDL command, the server responds with the unique message ID for all of the messages present in the users mailbox. This command is useful for "download and keep". By keeping a file that lists the messages retrieved in earlier sessions, the client can use the UIDL command to determine which messages on the server have already been seen.

Problem 17

```
a) C: dele 1
    C: retr 2
    S: (blah blah ...
    S: .....blah)
    S: .
    C: dele 2
    C: quit
    S: +OK POP3 server signing off
```

- b) C: retr 2
 S: blah blah ...
 S:blah
 S: .
 C: quit
 S: +OK POP3 server signing off
- c) C: list
 S: 1 498
 S: 2 912
 S: .
 C: retr 1
 S: blah
 S:blah
 S: .
 C: retr 2
 S: blah blah ...
 S:blah
 S: .
 C: quit
 S: +OK POP3 server signing off

Problem 18

- a) For a given input of domain name (such as ccn.com), IP address or network administrator name, *whois* database can be used to locate the corresponding registrar, whois server, DNS server, and so on.
- b) NS4.YAHOO.COM from www.register.com; NS1.MSFT.NET from ww.register.com
- c) *Local Domain: www.mindspring.com*
 Web servers : www.mindspring.com
 207.69.189.21, 207.69.189.22,
 207.69.189.23, 207.69.189.24,
 207.69.189.25, 207.69.189.26, 207.69.189.27,
 207.69.189.28
 Mail Servers : mx1.mindspring.com (207.69.189.217)
 mx2.mindspring.com (207.69.189.218)
 mx3.mindspring.com (207.69.189.219)
 mx4.mindspring.com (207.69.189.220)
 Name Servers: itchy.earthlink.net (207.69.188.196)
 scratchy.earthlink.net (207.69.188.197)
- www.yahoo.com*
 Web Servers: www.yahoo.com (216.109.112.135, 66.94.234.13)

Mail Servers: a.mx.mail.yahoo.com (209.191.118.103)
b.mx.mail.yahoo.com (66.196.97.250)
c.mx.mail.yahoo.com (68.142.237.182, 216.39.53.3)
d.mx.mail.yahoo.com (216.39.53.2)
e.mx.mail.yahoo.com (216.39.53.1)
f.mx.mail.yahoo.com (209.191.88.247, 68.142.202.247)
g.mx.mail.yahoo.com (209.191.88.239, 206.190.53.191)

Name Servers: ns1.yahoo.com (66.218.71.63)
ns2.yahoo.com (68.142.255.16)
ns3.yahoo.com (217.12.4.104)
ns4.yahoo.com (68.142.196.63)
ns5.yahoo.com (216.109.116.17)
ns8.yahoo.com (202.165.104.22)
ns9.yahoo.com (202.160.176.146)

www.hotmail.com

Web Servers: www.hotmail.com (64.4.33.7, 64.4.32.7)

Mail Servers: mx1.hotmail.com (65.54.245.8, 65.54.244.8, 65.54.244.136)
mx2.hotmail.com (65.54.244.40, 65.54.244.168, 65.54.245.40)
mx3.hotmail.com (65.54.244.72, 65.54.244.200, 65.54.245.72)
mx4.hotmail.com (65.54.244.232, 65.54.245.104, 65.54.244.104)

Name Servers: ns1.msft.net (207.68.160.190)
ns2.msft.net (65.54.240.126)
ns3.msft.net (213.199.161.77)
ns4.msft.net (207.46.66.126)
ns5.msft.net (65.55.238.126)

- d) The yahoo web server has multiple IP addresses
www.yahoo.com (216.109.112.135, 66.94.234.13)
- e) The address range for Polytechnic University: 128.238.0.0 – 128.238.255.255
- f) An attacker can use the *whois* database and nslookup tool to determine the IP address ranges, DNS server addresses, etc., for the target institution.
- g) By analyzing the source address of attack packets, the victim can use whois to obtain information about domain from which the attack is coming and possibly inform the administrators of the origin domain.

Problem 19

- a)
The following delegation chain is used for gaia.cs.umass.edu

a.root-servers.net
E.GTLD-SERVERS.NET
ns1.umass.edu(authoritative)

First command:

dig +norecurse @a.root-servers.net any gaia.cs.umass.edu

;; AUTHORITY SECTION:

edu.	172800	IN	NS	E.GTLD-SERVERS.NET.
edu.	172800	IN	NS	A.GTLD-SERVERS.NET.
edu.	172800	IN	NS	G3.NSTLD.COM.
edu.	172800	IN	NS	D.GTLD-SERVERS.NET.
edu.	172800	IN	NS	H3.NSTLD.COM.
edu.	172800	IN	NS	L3.NSTLD.COM.
edu.	172800	IN	NS	M3.NSTLD.COM.
edu.	172800	IN	NS	C.GTLD-SERVERS.NET.

Among all returned edu DNS servers, we send a query to the first one.

dig +norecurse @E.GTLD-SERVERS.NET any gaia.cs.umass.edu

umass.edu.	172800	IN	NS	ns1.umass.edu.
umass.edu.	172800	IN	NS	ns2.umass.edu.
umass.edu.	172800	IN	NS	ns3.umass.edu.

Among all three returned authoritative DNS servers, we send a query to the first one.

dig +norecurse @ns1.umass.edu any gaia.cs.umass.edu

gaia.cs.umass.edu. 21600 IN A 128.119.245.12

b) The answer for google.com could be:

a.root-servers.net
E.GTLD-SERVERS.NET
ns1.google.com(authoritative)

Problem 20

We can periodically take a snapshot of the DNS caches in those local DNS servers. The Web server that appears most frequently in the DNS caches is the most popular server. This is because if more users are interested in a Web server, then DNS requests for that server are more frequently sent by users. Thus, that Web server will appear in the DNS caches more frequently.

For a complete measurement study, see:

Craig E. Wills, Mikhail Mikhailov, Hao Shang

“Inferring Relative Popularity of Internet Applications by Actively Querying DNS Caches”, in IMC'03, October 27-29, 2003, Miami Beach, Florida, USA

Problem 21

Yes, we can use dig to query that Web site in the local DNS server.

For example, “dig cnn.com” will return the query time for finding cnn.com. If cnn.com is just accessed a couple of seconds ago, an entry for cnn.com is cached in the local DNS cache, so the query time is 0 msec. Otherwise, the query time is large.

Problem 22

For calculating the minimum distribution time for client-server distribution, we use the following formula:

$$D_{cs} = \max \{NF/u_s, F/d_{\min}\}$$

Similarly, for calculating the minimum distribution time for P2P distribution, we use the following formula:

$$D_{p2p} = \max \{F/u_s, F/d_{\min}, NF/(u_s + \sum_{i=1}^N u_i)\}$$

Where, $F = 15 \text{ Gbits} = 15 * 1024 \text{ Mbits}$

$u_s = 30 \text{ Mbps}$

$d_{\min} = d_i = 2 \text{ Mbps}$

Note, 300Kbps = 300/1024 Mbps.

Client Server

		N		
		10	100	1000
u	300 Kbps	7680	51200	512000
	700 Kbps	7680	51200	512000
	2 Mbps	7680	51200	512000

Peer to Peer

		N		
		10	100	1000
u	300 Kbps	7680	25904	47559
	700 Kbps	7680	15616	21525
	2 Mbps	7680	7680	7680

Problem 23

a) Consider a distribution scheme in which the server sends the file to each client, in parallel, at a rate of a rate of u_s/N . Note that this rate is less than each of the client's download rate, since by assumption $u_s/N \leq d_{\min}$. Thus each client can also receive at rate

u_s/N . Since each client receives at rate u_s/N , the time for each client to receive the entire file is $F/(u_s/N) = NF/u_s$. Since all the clients receive the file in NF/u_s , the overall distribution time is also NF/u_s .

b) Consider a distribution scheme in which the server sends the file to each client, in parallel, at a rate of $d_{\min}$. Note that the aggregate rate, $N d_{\min}$, is less than the server's link rate u_s , since by assumption $u_s/N \geq d_{\min}$. Since each client receives at rate $d_{\min}$, the time for each client to receive the entire file is $F/d_{\min}$. Since all the clients receive the file in this time, the overall distribution time is also $F/d_{\min}$.

c) From Section 2.6 we know that

$$D_{CS} \geq \max \{NF/u_s, F/d_{\min}\} \quad (\text{Equation 1})$$

Suppose that $u_s/N \leq d_{\min}$. Then from Equation 1 we have $D_{CS} \geq NF/u_s$. But from (a) we have $D_{CS} \leq NF/u_s$. Combining these two gives:

$$D_{CS} = NF/u_s \text{ when } u_s/N \leq d_{\min}. \quad (\text{Equation 2})$$

We can similarly show that:

$$D_{CS} = F/d_{\min} \text{ when } u_s/N \geq d_{\min} \quad (\text{Equation 3}).$$

Combining Equation 2 and Equation 3 gives the desired result.

Problem 24

a) Define $u = u_1 + u_2 + \dots + u_N$. By assumption

$$u_s \leq (u_s + u)/N \quad \text{Equation 1}$$

Divide the file into N parts, with the i^{th} part having size $(u_i/u)F$. The server transmits the i^{th} part to peer i at rate $r_i = (u_i/u)u_s$. Note that $r_1 + r_2 + \dots + r_N = u_s$, so that the aggregate server rate does not exceed the link rate of the server. Also have each peer i forward the bits it receives to each of the $N-1$ peers at rate r_i . The aggregate forwarding rate by peer i is $(N-1)r_i$. We have

$$(N-1)r_i = (N-1)(u_s u_i)/u \leq u_i,$$

where the last inequality follows from Equation 1. Thus the aggregate forwarding rate of peer i is less than its link rate u_i .

In this distribution scheme, peer i receives bits at an aggregate rate of

$$r_i + \sum_{j \neq i} r_j = u_s$$

Thus each peer receives the file in F/u_s .

b) Again define $u = u_1 + u_2 + \dots + u_N$. By assumption

$$u_s \geq (u_s + u)/N \quad \text{Equation 2}$$

Let $r_i = u_i/(N-1)$ and

$$r_{N+1} = (u_s - u/(N-1))/N$$

In this distribution scheme, the file is broken into $N+1$ parts. The server sends bits from the i^{th} part to the i^{th} peer ($i = 1, \dots, N$) at rate r_i . Each peer i forwards the bits arriving at rate r_i to each of the other $N-1$ peers. Additionally, the server sends bits from the $(N+1)^{\text{st}}$ part at rate r_{N+1} to each of the N peers. The peers do not forward the bits from the $(N+1)^{\text{st}}$ part.

The aggregate send rate of the server is

$$r_1 + \dots + r_N + N r_{N+1} = u/(N-1) + u_s - u/(N-1) = u_s$$

Thus, the server's send rate does not exceed its link rate. The aggregate send rate of peer i is

$$(N-1)r_i = u_i$$

Thus, each peer's send rate does not exceed its link rate.

In this distribution scheme, peer i receives bits at an aggregate rate of

$$r_i + r_{N+1} + \sum_{j \neq i} r_j = u/(N-1) + (u_s - u/(N-1))/N = (u_s + u)/N$$

Thus each peer receives the file in $NF/(u_s+u)$.

(For simplicity, we neglected to specify the size of the file part for $i = 1, \dots, N+1$. We now provide that here. Let $\Delta = (u_s+u)/N$ be the distribution time. For $i = 1, \dots, N$, the i^{th} file part is $F_i = r_i \Delta$ bits. The $(N+1)^{\text{st}}$ file part is $F_{N+1} = r_{N+1} \Delta$ bits. It is straightforward to show that $F_1 + \dots + F_{N+1} = F$.)

c) The solution to this part is similar to that of 17 (c). We know from section 2.6 that

$$D_{p2p} \geq \max\{F/u_s, NF/(u_s + u)\}$$

Combining this with (a) and (b) gives the desired result.

Problem 25

There are N nodes in the overlay network. There are $N(N-1)/2$ edges.

Problem 26

Yes. His first claim is possible, as long as there are enough peers staying in the swarm for a long enough time. Bob can always receive data through optimistic unchoking by other peers.

His second claim is also true. He can run a client on each machine, and let each client do “free-riding”, and combine those collected chunks from different machines into a single file. He can even write a small scheduling program to let different machines only asking for different chunks of the file. This is actually a kind of Sybil attack in P2P networks.

Problem 27

a).

Note that we assume $n_b \geq n_a$.

$\frac{C(N - n_a, n_b - n_a)}{C(N, n_b)}$, where $C(N, n)$ is the notation for combination, which means the number of ways of choosing n out of N .

$$b). p(n_a) = \sum_{n_b=n_a}^{N-1} \frac{1}{N} \frac{C(N - n_a, n_b - n_a)}{C(N, n_b)}.$$

$$c). \text{prob} = 1 - \left(\sum_{n_a=0}^{N-1} \frac{1}{N} p(n_a) \right)^5.$$

For a complete analysis, see:

Donyu Qiu and R. Srikant.

Modeling and Performance Analysis of BitTorrent-Like Peer-to-Peer Networks.

ACM Sigcomm 2004, Portland, Oregon, USA

Problem 28

Peer 3 learns that peer 5 has just left the system, so Peer 3 asks its first successor (peer 4) for the identifier of its immediate successor (peer 8). Then peer 3 will make peer 8 as its second successor.

Note that peer 3 knows that peer 5 was originally the first successor of peer 4, so peer 3 would wait until peer 4 finishes updating its first successor.

Problem 29

Peer 6 would first send peer 15 a message, saying “what will be peer 6’s predecessor and successor?” This message gets forwarded through the DHT until it reaches peer 5, who realizes that it will be 6’s predecessor and that its current successor, peer 8, will become 6’s successor. Next, peer 5 sends this predecessor and successor information back to 6. Peer 6 can now join the DHT by making peer 8 its successor and by notifying peer 5 that it should change its immediate successor to 6.

Problem 30

a).

Our assumption about keys and queries:

1. All keys are uniformly at random distributed in the key range, and all 8 peers are responsible for the same number of queries on average.
2. The queries generated by a peer are for keys uniformly at random distributed in the key range. That is, the query for any key is generated with the same probability.

Because of the homogeneity of peers and queries, we know that all peers will choose a shortcut peer with the same number of *overlay hops* away.

We also assume that *by default*, a peer node *only* knows about its immediate successor peer and its immediate predecessor node. (Unlike the case where a peer must know its second immediate successor in order to deal with peer churn).

We further assume that a peer can forward query to its predecessor.

And if there are multiple routing paths exist for a query, a peer always chooses the shortest path.

Note the number of messages sent for a query for a key is equal to the number of *routing hops* needed from the query generating peer to the peer that is holding the key.

Note that in our description, a routing hop is different from an overlay hop. An overlay hop simply means a logical hop between two *adjacent* peers along the DHT overlay ring. But a routing hop can span multiple overlay hops (or multiple consecutive adjacent peers) if shortcut is allowed.

Thus, minimizing the number of messages sent for any query (starting from any peer) is equivalent to minimizing the average or total number of routing hops traversed from one peer to all other peers.

Without loss of generality, let's look at peer 0.

To solve this problem, we look at all possibilities: a node shortcuts to a peer two overlay hops away in DHT id ring; or three overlay hops away; or four overlay hops away, so on. We can find that *the best configuration is for a peer to choose a shortcut peer with 4 overlay hops away*. "Best" in the sense that the average number of messages per query (or the total number of routing hops for all queries for all keys in the key space) is minimized. The computation is shown in the following table. Each column (except the last column) shows the number of messages needed for routing a query within a range. We see that the total number of messages needed is 11 when every peer shortcuts to another peer with 4 overlay hops (i.e., span 4 consecutive adjacent peers) away.

Queries for keys in range		(0,8]	(8,16]	(16,24]	(24,32]	(32,40]	(40,48]	(48,56]	(56,0]	Total messages
Number of overlay hops away for shortcutting	2	1	1	2	2	3	3	1	0	13
	3	1	2	1	2	3	2	1	0	12
	4	1	2	2	1	2	2	1	0	11
	5	Same as 3 hops away								12
	6	Same as 2 hops away								13
Note, if there are multiple routing paths exist for a query, we choose the shortest path's length as the number of needed messages. So for example, for the case where a peer shortcuts to another peer 4 overlay hops away, and if peer 0's query is for a key in range (48,56], then the shortest path is 0→56, only one routing hop (i.e., one message).										

b).

Follow the same reasoning as in part a).

We can find that *the best configuration is for a peer to choose two shortcut peers with 3 and 6 overlay hops away, or two shortcut peers with 5 and 6 overlay hops away.*

Queries for keys in range		(0,8]	(8,16]	(16,24]	(24,32]	(32,40]	(40,48]	(48,56]	(56,0]	Total messages
Num. of overlay hops away										
Nbr 1	Nbr 2									
2	3	1	1	1	2	2	2	1	0	10
2	4	1	1	2	2	2	1	1	0	10
2	5	1	1	1	2	2	2	1	0	10
2	6	1	1	1	2	2	2	1	0	10
3	4	1	2	1	1	1	2	1	0	9
3	5	1	2	1	1	1	2	1	0	9
3	6	1	1	1	2	1	1	1	0	8
4	5	Same as nbr 1, nbr 2 with 3 and 4 hops away respectively								9
4	6	1	1	2	1	2	1	1	0	9
5	6	Same as nbr 1, nbr 2 with 3 and 6 hops away respectively								8

Problem 31

For each key, we first calculate the distances (according to $d(k,p)$) between itself and all peers, and then store this key into the peer that is closed to this key (with smallest distance value).

Then, as in circular DHT described in textbook, we arrange those peers in a ring. In this ring, there are many clusters of keys, and each cluster is centered at a particular peer. Each key in a peer's cluster is closer to this peer than to all other peers. Some of the keys in this cluster can be larger than this peer's ID. Note that a peer is responsible for the keys in its cluster, instead of being responsible for only keys that are preceding it (i.e, keys have smaller value than the ID of this peer) in the key range.

Each peer keeps a routing table of n lists of entries. Each entry contains the information of one other peer. These lists are arranged sequentially, and the k -th ($1 \leq k \leq n$) list contains peers with IDs that differ from that of this peer in k -th significant bit but match with all $k-1$ bits more significant than k , including the most significant bit, the second most significant bit, so on, until the $(k-1)$ -th most significant bit. Note here we use longest-prefix matching. Also note that with this arrangement, it is possible that half of the IDs in the ID range can be put into the first list.

If $i > j$, then a peer in i -th list is closer to this node than a peer in j -th list.

The query routing can be done as follows. A peer first tries to match the bits of the key with its own ID's bits, and finds the "right" list in its routing table, and then forwards the query to any one entry in the list. A "right" list is a list that has the longest prefix matching with the target key. Once a peer receives the target key, it also checks its routing table, and forwards the search query to a peer in the "right" list, so on, so forth, until a peer that is responsible for the key is located, or returns "no such key" if no further routing is possible.

Problem 32

This is a generalized Kademlia DHT, and also similar to Pastry's prefix-matching DHT. The DHT based on binary numbers generates more messages, $\log_2 N$ in the worst case. This new DHT generates $\log_b N$ messages in the worst case.

Problem 33

Yes, that assignment scheme of keys to peers does not consider underlying network at all, so it very likely causes mismatch.

The mismatch may potentially degrade the search performance. For example, consider a logical path p_1 (consisting of only two logical links): $A \rightarrow B \rightarrow C$, where A and B are neighboring peers, and B and C are neighboring peers. Suppose that there is another logical path p_2 from A to B (consisting of 3 logical links): $A \rightarrow D \rightarrow E \rightarrow C$.

It might be the case that A and B could be very far away physically, and B and C could be very far away physically. But A , D , E , and C are very close physically. In other words, a shorter logical path corresponds to a longer physical path than does a longer logical path.

Problem 34

a) If you run TCPClient first, then the client will attempt to make a TCP connection with a non-existent server process. A TCP connection will not be made.

b) UDPClient doesn't establish a TCP connection with the server. Thus, everything should work fine if you first run UDPClient, then run UDPServer, and then type some input into the keyboard.

c) If you use different port numbers, then the client will attempt to establish a TCP connection with the wrong process or a non-existent process. Errors will occur.

Problem 35

With the original line, UDPClient does not specify a port number when it creates the socket. In this case, the code lets the underlying operating system choose a port number. With the replacement line, when UDPClient is executed, a UDP socket is created with port number 5432 .

UDPServer needs to know the client port number so that it can send packets back to the correct client socket. Glancing at UDPServer, we see that the client port number is not “hard-wired” into the server code; instead, UDPServer determines the client port number by unraveling the datagram it receives from the client (using the `.getPort()` method). Thus UDP server will work with any client port number, including 5432. UDPServer therefore does not need to be modified.

Before:

Client socket = x (chosen by OS)
Server socket = 9876

After:

Client socket = 5432

Chapter 3 Review Questions

1.
 - a) Call this protocol Simple Transport Protocol (STP). At the sender side, STP accepts from the sending process a chunk of data not exceeding 1196 bytes, a destination host address, and a destination port number. STP adds a four-byte header to each chunk and puts the port number of the destination process in this header. STP then gives the destination host address and the resulting segment to the network layer. The network layer delivers the segment to STP at the destination host. STP then examines the port number in the segment, extracts the data from the segment, and passes the data to the process identified by the port number.
 - b) The segment now has two header fields: a source port field and destination port field. At the sender side, STP accepts a chunk of data not exceeding 1192 bytes, a destination host address, a source port number, and a destination port number. STP creates a segment which contains the application data, source port number, and destination port number. It then gives the segment and the destination host address to the network layer. After receiving the segment, STP at the receiving host gives the application process the application data and the source port number.
 - c) No, the transport layer does not have to do anything in the core; the transport layer “lives” in the end systems.
2.
 - a) For sending a letter, the family member is required to give the delegate the letter itself, the address of the destination house, and the name of the recipient. The delegate clearly writes the recipient’s name on the top of the letter. The delegate then puts the letter in an envelope and writes the address of the destination house on the envelope. The delegate then gives the letter to the planet’s mail service. At the receiving side, the delegate receives the letter from the mail service, takes the letter out of the envelope, and takes note of the recipient name written at the top of the letter. The delegate then gives the letter to the family member with this name.
 - b) No, the mail service does not have to open the envelope; it only examines the address on the envelope.
3. Source port number y and destination port number x .
4. An application developer may not want its application to use TCP’s congestion control, which can throttle the application’s sending rate at times of congestion. Often, designers of IP telephony and IP videoconference applications choose to run their applications over UDP because they want to avoid TCP’s congestion control. Also, some applications do not need the reliable data transfer provided by TCP.

5. Since most firewalls are configured to block UDP traffic, using TCP for video and voice traffic lets the traffic through the firewalls.
6. Yes. The application developer can put reliable data transfer into the application layer protocol. This would require a significant amount of work and debugging, however.
7. Yes, both segments will be directed to the same socket. For each received segment, at the socket interface, the operating system will provide the process with the IP addresses to determine the origins of the individual segments.
8. For each persistent connection, the Web server creates a separate “connection socket”. Each connection socket is identified with a four-tuple: (source IP address, source port number, destination IP address, destination port number). When host C receives an IP datagram, it examines these four fields in the datagram/segment to determine to which socket it should pass the payload of the TCP segment. Thus, the requests from A and B pass through different sockets. The identifier for both of these sockets has 80 for the destination port; however, the identifiers for these sockets have different values for source IP addresses. Unlike UDP, when the transport layer passes a TCP segment’s payload to the application process, it does not specify the source IP address, as this is implicitly specified by the socket identifier.
9. Sequence numbers are required for a receiver to find out whether an arriving packet contains new data or is a retransmission.
10. To handle losses in the channel. If the ACK for a transmitted packet is not received within the duration of the timer for the packet, the packet (or its ACK or NACK) is assumed to have been lost. Hence, the packet is retransmitted.
11. A timer would still be necessary in the protocol rdt 3.0. If the round trip time is known then the only advantage will be that, the sender knows for sure that either the packet or the ACK (or NACK) for the packet has been lost, as compared to the real scenario, where the ACK (or NACK) might still be on the way to the sender, after the timer expires. However, to detect the loss, for each packet, a timer of constant duration will still be necessary at the sender.
12. Java Applet
13. Java Applet
14. a) false b) false c) true d) false e) true f) false g) false
15. a) 20 bytes b) ack number = 90

16. 3 segments. First segment: seq = 43, ack = 80; Second segment: seq = 80, ack = 44; Third segment; seq = 44, ack = 81

17. $R/2$

18. False, it is set to half of the current value of the congestion window.

Chapter 3 Problems

Problem 1

	source port numbers	destination port numbers
a) $A \rightarrow S$	467	23
b) $B \rightarrow S$	513	23
c) $S \rightarrow A$	23	467
d) $S \rightarrow B$	23	513

e) Yes.

f) No.

Problem 2

Suppose the IP addresses of the hosts A, B, and C are a, b, c, respectively. (Note that a,b,c are distinct.)

To host A: Source port =80, source IP address = b, dest port = 26145, dest IP address = a

To host C, left process: Source port =80, source IP address = b, dest port = 7532, dest IP address = c

To host C, right process: Source port =80, source IP address = b, dest port = 26145, dest IP address = c

Problem 3

Note, wrap around if overflow.

$$\begin{array}{r} 0\ 1\ 0\ 1\ 0\ 0\ 1\ 1 \\ +\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 0 \\ \hline 1\ 0\ 1\ 0\ 0\ 1\ 1\ 1 \end{array}$$

$$\begin{array}{r} 1\ 0\ 1\ 0\ 0\ 1\ 1\ 1 \\ +\ 0\ 1\ 1\ 1\ 0\ 1\ 0\ 0 \\ \hline 0\ 0\ 0\ 1\ 1\ 1\ 0\ 0 \end{array}$$

One's complement = 1 1 1 0 0 0 1 1.

To detect errors, the receiver adds the four words (the three original words and the checksum). If the sum contains a zero, the receiver knows there has been an error. All one-bit errors will be detected, but two-bit errors can be undetected (e.g., if the last digit of the first word is converted to a 0 and the last digit of the second word is converted to a 1).

Problem 4

(a) Adding the two bytes gives 10110010. Taking the one's complement gives 01001101.

(b) Adding the two bytes gives 00010001; the one's complement gives 11101110.

(c) First byte = 01011110 ; second byte = 01010100.

Problem 5

No, the receiver cannot be absolutely certain that no bit errors have occurred. This is because of the manner in which the checksum for the packet is calculated. If the corresponding bits (that would be added together) of two 16-bit words in the packet were 0 and 1 then even if these get flipped to 1 and 0 respectively, the sum still remains the same. Hence, the 1s complement the receiver calculates will also be the same. This means the checksum will verify even if there was transmission error.

Problem 6

Suppose the sender is in state "Wait for call 1 from above" and the receiver (the receiver shown in the homework problem) is in state "Wait for 1 from below." The sender sends a packet with sequence number 1, and transitions to "Wait for ACK or NAK 1," waiting for an ACK or NAK. Suppose now the receiver receives the packet with sequence number 1 correctly, sends an ACK, and transitions to state "Wait for 0 from below," waiting for a data packet with sequence number 0. However, the ACK is corrupted. When the rdt2.1 sender gets the corrupted ACK, it resends the packet with sequence number 1. However, the receiver is waiting for a packet with sequence number 0 and (as shown in the home work problem) always sends a NAK when it doesn't get a packet with sequence number 0. Hence the sender will always be sending a packet with sequence number 1, and the receiver will always be NAKing that packet. Neither will progress forward from that state.

Problem 7

To best answer this question, consider why we needed sequence numbers in the first place. We saw that the sender needs sequence numbers so that the receiver can tell if a data packet is a duplicate of an already received data packet. In the case of ACKs, the sender does not need this info (i.e., a sequence number on an ACK) to tell detect a duplicate ACK. A duplicate ACK is obvious to the rdt3.0 receiver, since when it has

received the original ACK it transitioned to the next state. The duplicate ACK is not the ACK that the sender needs and hence is ignored by the rdt3.0 sender.

Problem 8

The sender side of protocol rdt3.0 differs from the sender side of protocol 2.2 in that timeouts have been added. We have seen that the introduction of timeouts adds the possibility of duplicate packets into the sender-to-receiver data stream. However, the receiver in protocol rdt.2.2 can already handle duplicate packets. (Receiver-side duplicates in rdt 2.2 would arise if the receiver sent an ACK that was lost, and the sender then retransmitted the old data). Hence the receiver in protocol rdt2.2 will also work as the receiver in protocol rdt 3.0.

Problem 9

Suppose the protocol has been in operation for some time. The sender is in state “Wait for call from above” (top left hand corner) and the receiver is in state “Wait for 0 from below”. The scenarios for corrupted data and corrupted ACK are shown in Figure 1.

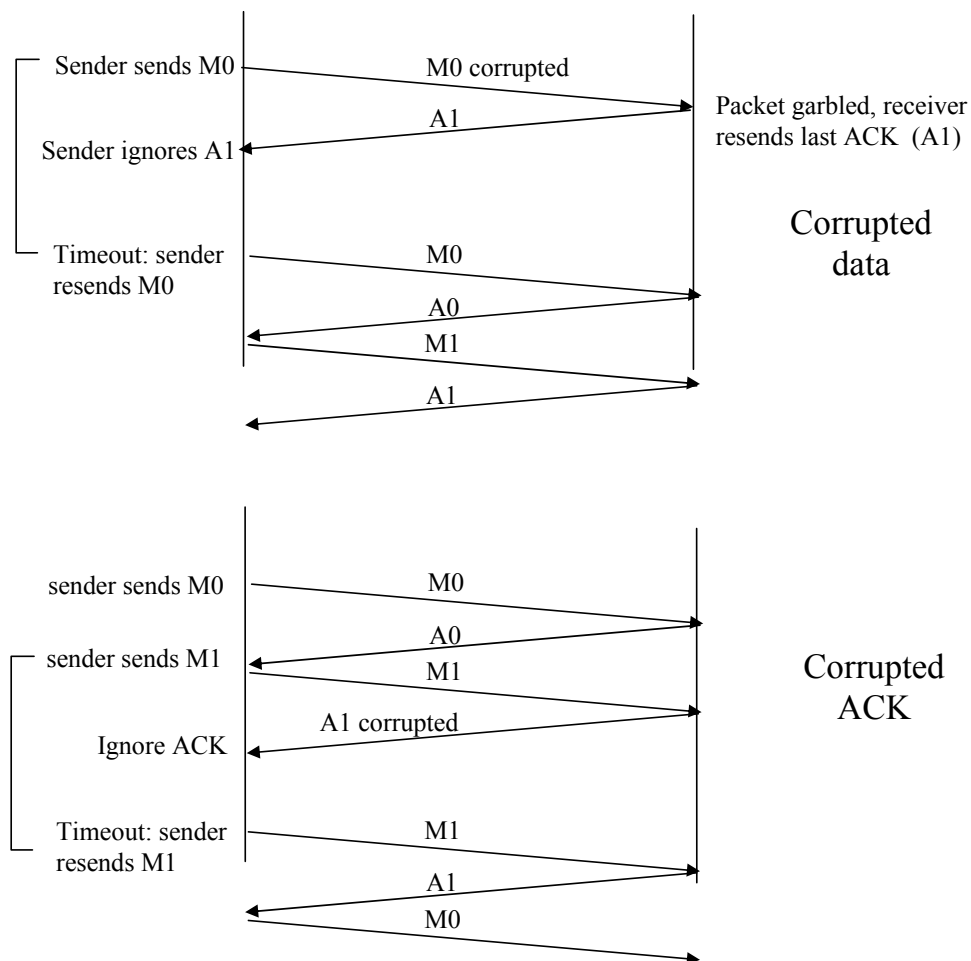


Figure 1: rdt 3.0 scenarios: corrupted data, corrupted ACK

Problem 10

Here, we add a timer, whose value is greater than the known round-trip propagation delay. We add a timeout event to the “Wait for ACK or NAK0” and “Wait for ACK or NAK1” states. If the timeout event occurs, the most recently transmitted packet is retransmitted. Let us see why this protocol will still work with the rdt2.1 receiver.

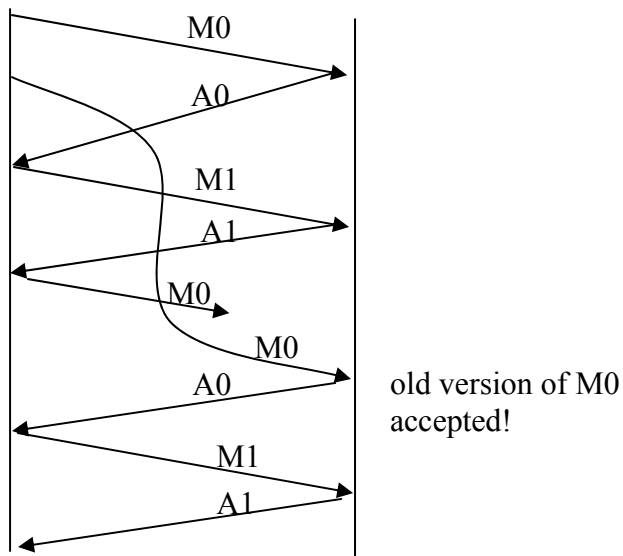
- Suppose the timeout is caused by a lost data packet, i.e., a packet on the sender-to-receiver channel. In this case, the receiver never received the previous transmission and, from the receiver's viewpoint, if the timeout retransmission is received, it look *exactly* the same as if the original transmission is being received.
- Suppose now that an ACK is lost. The receiver will eventually retransmit the packet on a timeout. But a retransmission is exactly the same action that is take if 1an ACK is garbled. Thus the sender's reaction is the same with a loss, as with a garbled ACK. The rdt 2.1 receiver can already handle the case of a garbled ACK.

Problem 11

The protocol would still work, since a retransmission would be what would happen if the packet received with errors has actually been lost (and from the receiver standpoint, it never knows which of these events, if either, will occur).

To get at the more subtle issue behind this question, one has to allow for premature timeouts to occur. In this case, if each extra copy of the packet is ACKed and each received extra ACK causes another extra copy of the current packet to be sent, the number of times packet n is sent will increase without bound as n approaches infinity.

Problem 12



Problem 13

In a NAK only protocol, the loss of packet x is only detected by the receiver when packet $x+1$ is received. That is, the receiver receives $x-1$ and then $x+1$, only when $x+1$ is received does the receiver realize that x was missed. If there is a long delay between the transmission of x and the transmission of $x+1$, then it will be a long time until x can be recovered, under a NAK only protocol.

On the other hand, if data is being sent often, then recovery under a NAK-only scheme could happen quickly. Moreover, if errors are infrequent, then NAKs are only occasionally sent (when needed), and ACK are never sent – a significant reduction in feedback in the NAK-only case over the ACK-only case.

Problem 14

It takes 12 microseconds (or 0.012 milliseconds) to send a packet, as $1500 \times 8 / 10^9 = 12$ microseconds. In order for the sender to be busy 95 percent of the time, we must have

$$util = 0.95 = (0.012n) / 30.012$$

or n approximately 2376 packets.

Problem 15

Yes. This actually causes the sender to send a number of pipelined data into the channel. Yes. Here is one potential problem. If data segments are lost in the channel, then the sender of rdt 3.0 won't re-send those segments, unless there are some additional mechanism in the application to recover from loss.

Problem 16

In our solution, the sender will wait until it receives an ACK for a pair of messages (seqnum and seqnum+1) before moving on to the next pair of messages. Data packets have a data field and carry a two-bit sequence number. That is, the valid sequence numbers are 0, 1, 2, and 3. (Note: you should think about why a 1-bit sequence number space of 0, 1 only would not work in the solution below.) ACK messages carry the sequence number of the data packet they are acknowledging.

The FSM for the sender and receiver are shown in Figure 2. Note that the sender state records whether (i) no ACKs have been received for the current pair, (ii) an ACK for seqnum (only) has been received, or an ACK for seqnum+1 (only) has been received. In this figure, we assume that the seqnum is initially 0, and that the sender has sent the first

two data messages (to get things going). A timeline trace for the sender and receiver recovering from a lost packet is shown below:

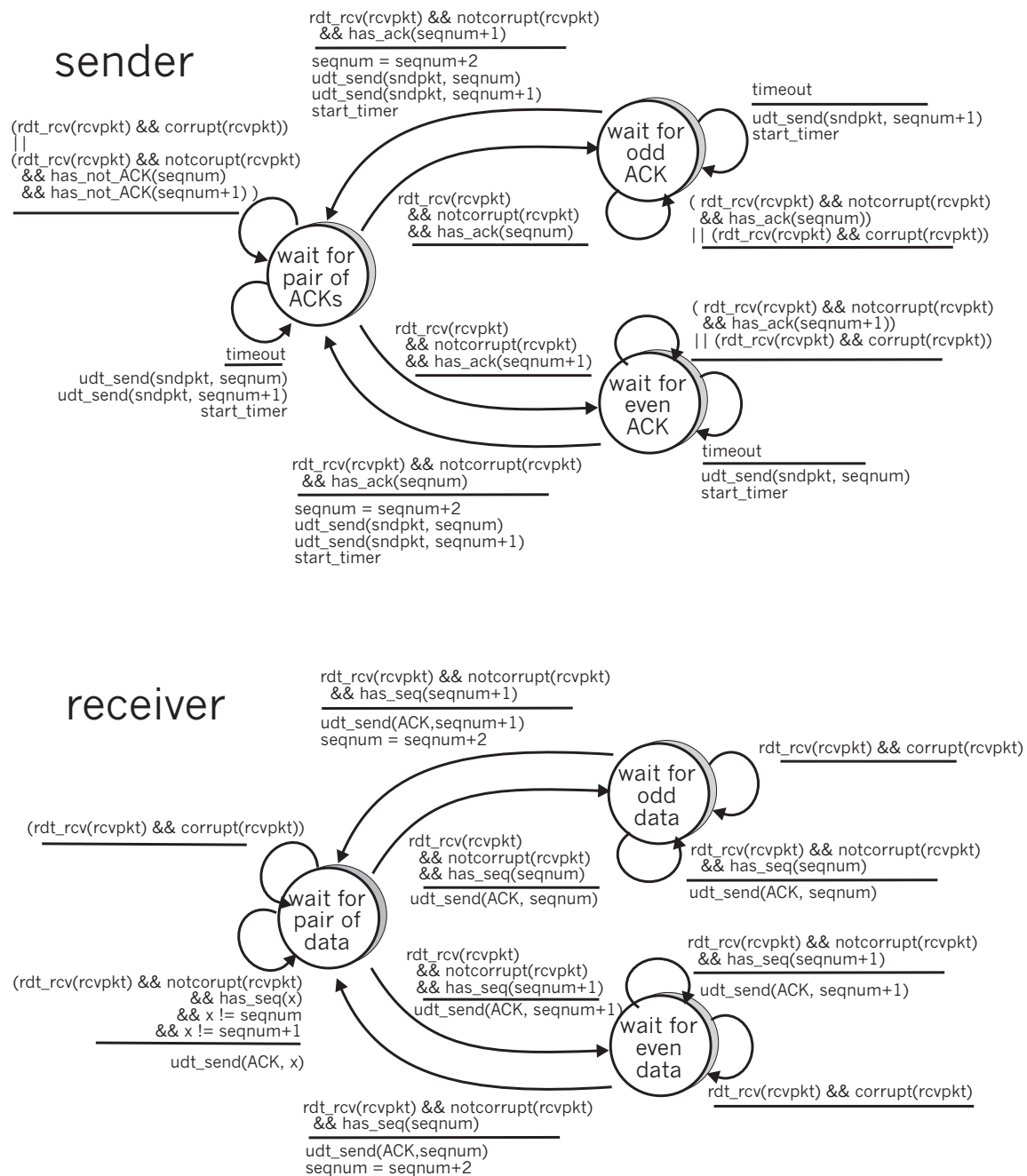


Figure 2: Sender and receiver for Problem 3.15

Sender

Receiver

make pair (0,1)
send packet 0

	Packet 0 drops	
send packet 1		receive packet 1
		buffer packet 1
		send ACK 1
receive ACK 1 (timeout)		
resend packet 0		receive packet 0
		deliver pair (0,1)
		send ACK 0
receive ACK 0		

Problem 17

This problem is a variation on the simple stop and wait protocol (rdt3.0). Because the channel may lose messages and because the sender may resend a message that one of the receivers has already received (either because of a premature timeout or because the other receiver has yet to receive the data correctly), sequence numbers are needed. As in rdt3.0, a 0-bit sequence number will suffice here.

The sender and receiver FSM are shown in Figure 3. In this problem, the sender state indicates whether the sender has received an ACK from B (only), from C (only) or from neither C nor B. The receiver state indicates which sequence number the receiver is waiting for.

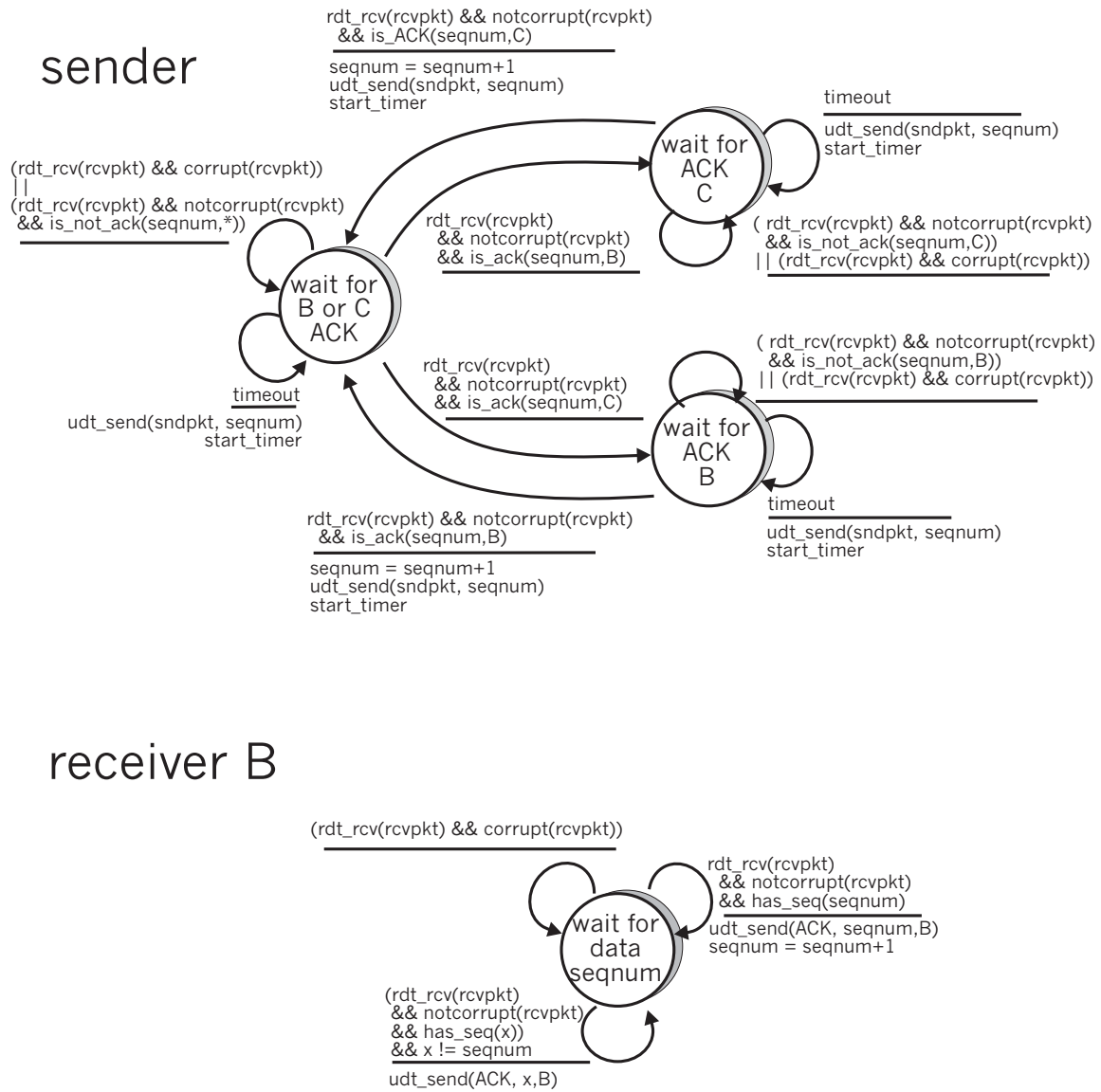


Figure 3. Sender and receiver for Problem 3.16

Problem 18

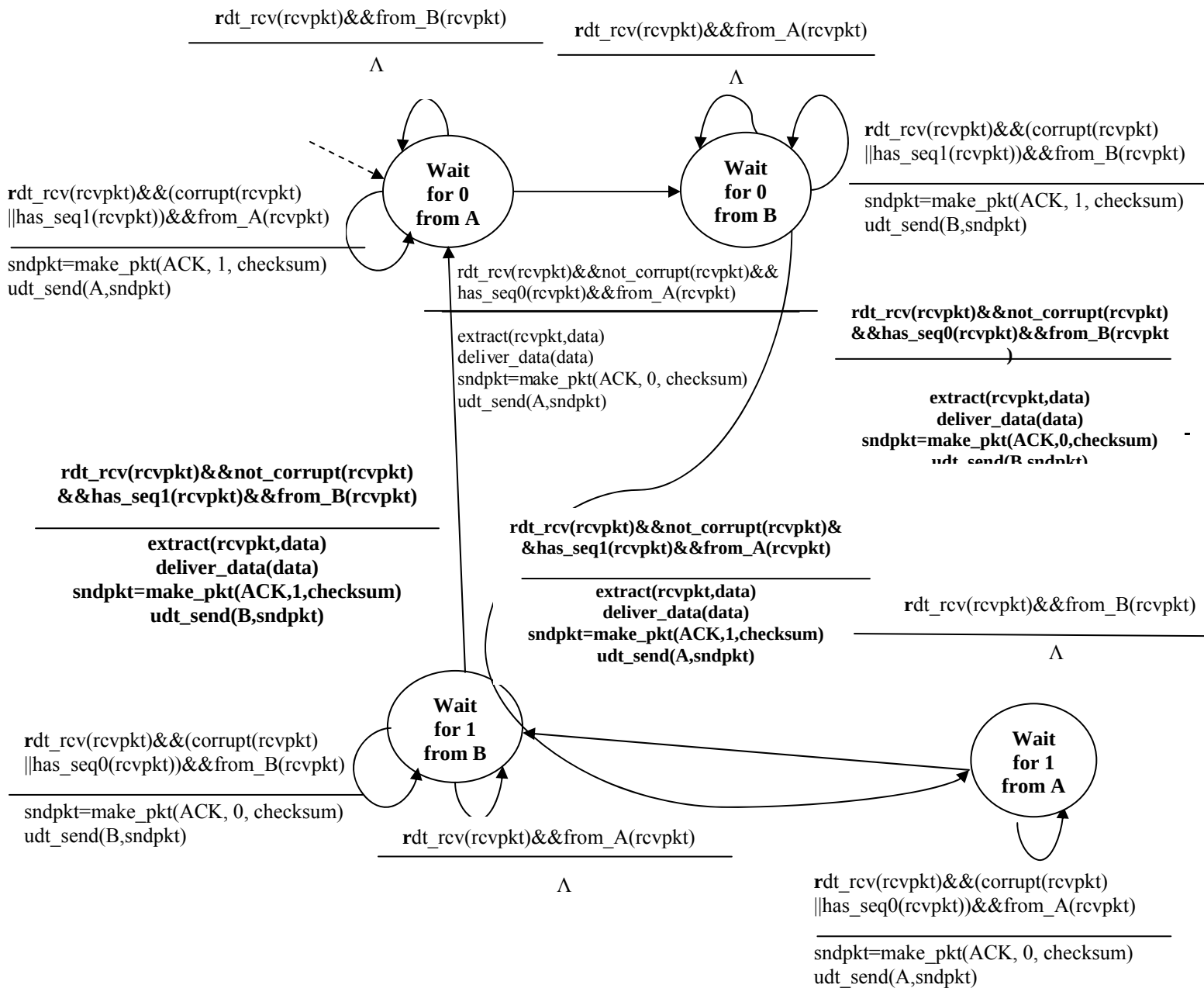


Figure 4: Receiver side FSM for 3.17

Sender

The sender side FSM is exactly same as given in Figure 3.15 in text

Problem 19

a) Here we have a window size of $N=3$. Suppose the receiver has received packet $k-1$, and has ACKed that and all other preceeding packets. If all of these ACK's have been received by sender, then sender's window is $[k, k+N-1]$. Suppose next that none of the ACKs have been received at the sender. In this second case, the sender's window contains $k-1$ and the N packets up to and including $k-1$. The sender's window is thus $[k-N, k-1]$. By these arguments, the senders window is of size 3 and begins somewhere in the range $[k-N, k]$.

b) If the receiver is waiting for packet k , then it has received (and ACKed) packet $k-1$ and the $N-1$ packets before that. If none of those N ACKs have been yet received by the sender, then ACK messages with values of $[k-N, k-1]$ may still be propagating back. Because the sender has sent packets $[k-N, k-1]$, it must be the case that the sender has already received an ACK for $k-N-1$. Once the receiver has sent an ACK for $k-N-1$ it will never send an ACK that is less than $k-N-1$. Thus the range of in-flight ACK values can range from $k-N-1$ to $k-1$.

Problem 20

Because the A-to-B channel can lose request messages, A will need to timeout and retransmit its request messages (to be able to recover from loss). Because the channel delays are variable and unknown, it is possible that A will send duplicate requests (i.e., resend a request message that has already been received by B). To be able to detect duplicate request messages, the protocol will use sequence numbers. A 1-bit sequence number will suffice for a stop-and-wait type of request/response protocol.

A (the requestor) has 4 states:

- **“Wait for Request 0 from above.”** Here the requestor is waiting for a call from above to request a unit of data. When it receives a request from above, it sends a request message, R_0 , to B, starts a timer and makes a transition to the “Wait for D_0 ” state. When in the “Wait for Request 0 from above” state, A ignores anything it receives from B.
- **“Wait for D_0 ”.** Here the requestor is waiting for a D_0 data message from B. A timer is always running in this state. If the timer expires, A sends another R_0 message, restarts the timer and remains in this state. If a D_0 message is received from B, A stops the time and transits to the “Wait for Request 1 from above” state. If A receives a D_1 data message while in this state, it is ignored.
- **“Wait for Request 1 from above.”** Here the requestor is again waiting for a call from above to request a unit of data. When it receives a request from above, it

sends a request message, R1, to B, starts a timer and makes a transition to the “Wait for D1” state. When in the “Wait for Request 1 from above” state, A ignores anything it receives from B.

- **“Wait for D1”.** Here the requestor is waiting for a D1 data message from B. A timer is always running in this state. If the timer expires, A sends another R1 message, restarts the timer and remains in this state. If a D1 message is received from B, A stops the timer and transits to the “Wait for Request 0 from above” state. If A receives a D0 data message while in this state, it is ignored.

The data supplier (B) has only two states:

- **“Send D0.”** In this state, B continues to respond to received R0 messages by sending D0, and then remaining in this state. If B receives a R1 message, then it knows its D0 message has been received correctly. It thus discards this D0 data (since it has been received at the other side) and then transits to the “Send D1” state, where it will use D1 to send the next requested piece of data.
- **“Send D1.”** In this state, B continues to respond to received R1 messages by sending D1, and then remaining in this state. If B receives a R1 message, then it knows its D1 message has been received correctly and thus transits to the “Send D1” state.

Problem 21

In order to avoid the scenario of Figure 3.27, we want to avoid having the leading edge of the receiver's window (i.e., the one with the “highest” sequence number) wrap around in the sequence number space and overlap with the trailing edge (the one with the “lowest” sequence number in the sender's window). That is, the sequence number space must be large enough to fit the entire receiver window and the entire sender window without this overlap condition. So - we need to determine how large a range of sequence numbers can be covered at any given time by the receiver and sender windows.

Suppose that the lowest-sequence number that the receiver is waiting for is packet m . In this case, its window is $[m, m+w-1]$ and it has received (and ACKed) packet $m-1$ and the $w-1$ packets before that, where w is the size of the window. If none of those w ACKs have been yet received by the sender, then ACK messages with values of $[m-w, m-1]$ may still be propagating back. If no ACKs with these ACK numbers have been received by the sender, then the sender's window would be $[m-w, m-1]$.

Thus, the lower edge of the sender's window is $m-w$, and the leading edge of the receiver's window is $m+w-1$. In order for the leading edge of the receiver's window to not overlap with the trailing edge of the sender's window, the sequence number space must thus be big enough to accommodate $2w$ sequence numbers. That is, the sequence number space must be at least twice as large as the window size, $k \geq 2w$.

Problem 22

- a) True. Suppose the sender has a window size of 3 and sends packets 1, 2, 3 at t_0 . At t_1 ($t_1 > t_0$) the receiver ACKS 1, 2, 3. At t_2 ($t_2 > t_1$) the sender times out and resends 1, 2, 3. At t_3 the receiver receives the duplicates and re-acknowledges 1, 2, 3. At t_4 the sender receives the ACKs that the receiver sent at t_1 and advances its window to 4, 5, 6. At t_5 the sender receives the ACKs 1, 2, 3 the receiver sent at t_2 . These ACKs are outside its window.
- b) True. By essentially the same scenario as in (a).
- c) True.
- d) True. Note that with a window size of 1, SR, GBN, and the alternating bit protocol are functionally equivalent. The window size of 1 precludes the possibility of out-of-order packets (within the window). A cumulative ACK is just an ordinary ACK in this situation, since it can only refer to the single packet within the window.

Problem 23

- a) Consider sending an application message over a transport protocol. With TCP, the application writes data to the connection send buffer and TCP will grab bytes without necessarily putting a single message in the TCP segment; TCP may put more or less than a single message in a segment. UDP, on the other hand, encapsulates in a segment whatever the application gives it; so that, if the application gives UDP an application message, this message will be the payload of the UDP segment. Thus, with UDP, an application has more control of what data is sent in a segment.
- b) With TCP, due to flow control and congestion control, there may be significant delay from the time when an application writes data to its send buffer until when the data is given to the network layer. UDP does not have delays due to flow control and congestion control.

Problem 24

There are $2^{32} = 4,294,967,296$ possible sequence numbers.

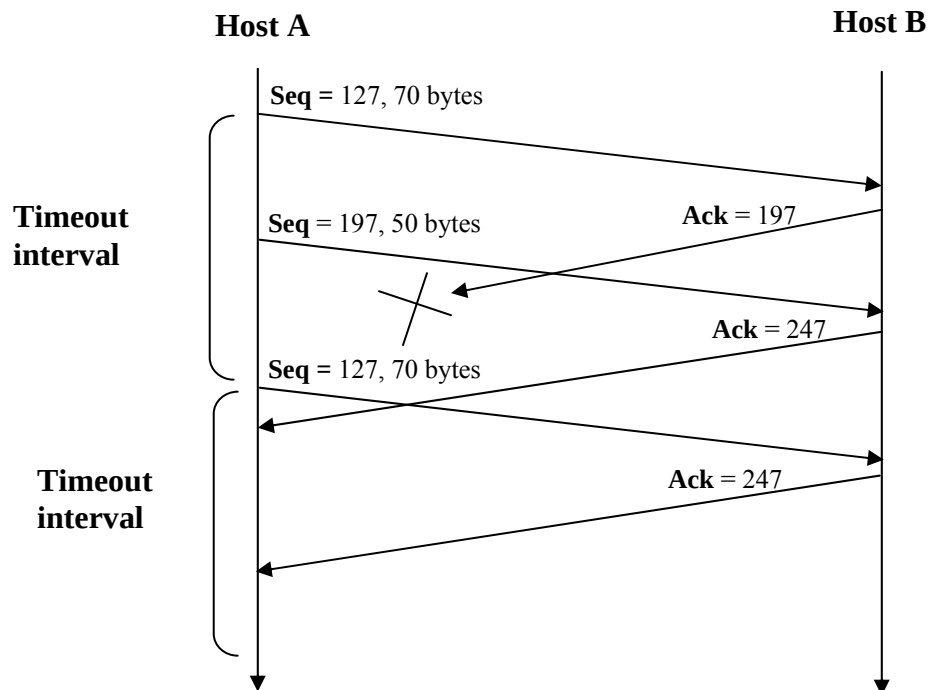
- a) The sequence number does not increment by one with each segment. Rather, it increments by the number of bytes of data sent. So the size of the MSS is irrelevant -- the maximum size file that can be sent from A to B is simply the number of bytes representable by $2^{32} \approx 4.19$ Gbytes.

- b) The number of segments is $\left\lceil \frac{2^{32}}{536} \right\rceil = 8,012,999$. 66 bytes of header get added to each segment giving a total of 528,857,934 bytes of header. The total number of bytes transmitted is $2^{32} + 528,857,934 = 4.824 \times 10^9$ bytes.

Thus it would take 249 seconds to transmit the file over a 155-Mbps link.

Problem 25

- In the second segment from Host A to B, the sequence number is 197, source port number is 302 and destination port number is 80.
- If the first segment arrives before the second, in the acknowledgement of the first arriving segment, the acknowledgement number is 197, the source port number is 80 and the destination port number is 302.
- If the second segment arrives before the first segment, in the acknowledgement of the first arriving segment, the acknowledgement number is 127, indicating that it is still waiting for bytes 127 and onwards.
-



Problem 26

Since the link capacity is only 100 Mbps, so host A's sending rate can be at most 100Mbps. Still, host A sends data into the receive buffer faster than Host B can remove data from the buffer. The receive buffer fills up at a rate of roughly 40Mbps. When the buffer is full, Host B signals to Host A to stop sending data by setting RcvWindow = 0. Host A then stops sending until it receives a TCP segment with RcvWindow > 0. Host A will thus repeatedly stop and start sending as a function of the RcvWindow values it receives from Host B. On average, the long-term rate at which Host A sends data to Host B as part of this connection is no more than 60Mbps.

Problem 27

- The server uses special initial sequence number (that is obtained from the hash of source and destination IPs and ports) in order to defend itself against SYN FLOOD attack.

b) No, the attacker cannot create half-open or fully open connections by simply sending and ACK packet to the target. Half-open connections are not possible since a server using SYN cookies does not maintain connection variables and buffers for any connection before full connections are established. For establishing fully open connections, an attacker should know the special initial sequence number corresponding to the (spoofed) source IP address from the attacker. This sequence number requires the "secret" number that each server uses. Since the attacker does not know this secret number, she cannot guess the initial sequence number.

c) No, the sever can simply add in a time stamp in computing those initial sequence numbers and choose a time to live value for those sequence numbers, and discard expired initial sequence numbers even if the attacker replay them.

Problem 28

If timeout values are fixed, then the senders may timeout prematurely. Thus, some packets are re-transmitted even they are not lost.

If timeout values are estimated (like what TCP does), then increasing the buffer size certainly helps to increase the throughput of that router. But there might be one potential problem. Queuing delay might be very large, similar to what is shown in Scenario 1.

Problem 29

Denote $EstimatedRTT^{(n)}$ for the estimate after the n th sample.

$$EstimatedRTT^{(1)} = SampleRTT_1$$

$$EstimatedRTT^{(2)} = xSampleRTT_1 + (1 - x)SampleRTT_2$$

$$\begin{aligned} EstimatedRTT^{(3)} &= xSampleRTT_1 \\ &\quad + (1 - x)[xSampleRTT_2 + (1 - x)SampleRTT_3] \\ &= xSampleRTT_1 + (1 - x)xSampleRTT_2 \\ &\quad + (1 - x)^2 SampleRTT_3 \end{aligned}$$

$$\begin{aligned} EstimatedRTT^{(4)} &= xSampleRTT_1 + (1 - x)EstimatedRTT^{(3)} \\ &= xSampleRTT_1 + (1 - x)xSampleRTT_2 \\ &\quad + (1 - x)^2 xSampleRTT_3 + (1 - x)^3 SampleRTT_4 \end{aligned}$$

b)

$$EstimatedRTT^{(n)} = x \sum_{j=1}^{n-1} (1-x)^j SampleRTT_j \\ + (1-x)^n SampleRTT_n$$

c)

$$EstimatedRTT^{(\infty)} = \frac{x}{1-x} \sum_{j=1}^{\infty} (1-x)^j SampleRTT_j \\ = \frac{1}{9} \sum_{j=1}^{\infty} .9^j SampleRTT_j$$

The weight given to past samples decays exponentially.

Problem 30

Let's look at what could wrong if TCP measures `SampleRTT` for a retransmitted segment. Suppose the source sends packet P1, the timer for P1 expires, and the source then sends P2, a new copy of the same packet. Further suppose the source measures `SampleRTT` for P2 (the retransmitted packet). Finally suppose that shortly after transmitting P2 an acknowledgment for P1 arrives. The source will mistakenly take this acknowledgment as an acknowledgment for P2 and calculate an incorrect value of `SampleRTT`.

Problem 31

At any given time t , `SendBase - 1` is the sequence number of the last byte that the sender knows has been received correctly, and in order, at the receiver. The actually last byte received (correctly and in order) at the receiver at time t may be greater if there are acknowledgements in the pipe. Thus

$$SendBase-1 \leq LastByteRcvd$$

Problem 32

When, at time t , the sender receives an acknowledgement with value y , the sender knows for sure that the receiver has received everything up through $y-1$. The actual last byte received (correctly and in order) at the receiver at time t may be greater if $y \leq SendBase$ or if there are other acknowledgements in the pipe. Thus

$$y-1 \leq LastByteRcvd$$

Problem 33

Suppose packets n , $n+1$, and $n+2$ are sent, and that packet n is received and ACKed. If packets $n+1$ and $n+2$ are reordered along the end-to-end-path (i.e., are received in the order $n+2$, $n+1$) then the receipt of packet $n+2$ will generate a duplicate ack for n and would trigger a retransmission under a policy of waiting only for second duplicate ACK for retransmission. By waiting for a triple duplicate ACK, it must be the case that two packets after packet n are correctly received, while $n+1$ was not received. The designers of the triple duplicate ACK scheme probably felt that waiting for two packets (rather than 1) was the right tradeoff between triggering a quick retransmission when needed, but not retransmitting prematurely in the face of packet reordering.

Problem 34

Note that there is a typo in the textbook.

part b, “5” is missing in “ $\cdot RTT$ ”.

“If the timeout values for all three protocol are much longer than $\cdot RTT$ ”

→

“If the timeout values for all three protocol are much longer than $5 \cdot RTT$ ”

a).

GoBackN:

A sends 9 segments in total. They are initially sent segments 1, 2, 3, 4, 5 and later re-sent segments 2, 3, 4, and 5.

B sends 8 ACKs. They are 4 ACKS with sequence number 1, and 4 ACKS with sequence numbers 2, 3, 4, and 5.

Selective Repeat:

A sends 6 segments in total. They are initially sent segments 1, 2, 3, 4, 5 and later re-sent segments 2.

B sends 5 ACKs. They are 4 ACKS with sequence number 1, 3, 4, 5. And there is one ACK with sequence number 2.

TCP:

A sends 6 segments in total. They are initially sent segments 1, 2, 3, 4, 5 and later re-sent segments 2.

B sends 5 ACKs. They are 4 ACKS with sequence number 2. There is one ACK with sequence numbers 6. Note that TCP always send an ACK with expected sequence number.

b). TCP. This is because TCP uses fast retransmit without waiting until time out.

Problem 35

Yes, the sending rate is always roughly $cwnd/RTT$.

Problem 36

If the arrival rate increases beyond $R/2$ in Figure 3.46(b), then the total arrival rate to the queue exceeds the queue's capacity, resulting in increasing loss as the arrival rate increases. When the arrival rate equals $R/2$, 1 out of every three packets that leaves the queue is a retransmission. With increased loss, even a larger fraction of the packets leaving the queue will be retransmissions. Given that the maximum departure rate from the queue for one of the sessions is $R/2$, and given that a third or more will be transmissions as the arrival rate increases, the throughput of successfully deliver data can not increase beyond λ_{out} . Following similar reasoning, if half of the packets leaving the queue are retransmissions, and the maximum rate of output packets per session is $R/2$, then the maximum value of λ_{out} is $(R/2)/2$ or $R/4$.

Problem 37

- a) TCP slowstart is operating in the intervals $[1,6]$ and $[23,26]$
- b) TCP congestion avoidance is operating in the intervals $[6,16]$ and $[17,22]$
- c) After the 16th transmission round, packet loss is recognized by a triple duplicate ACK. If there was a timeout, the congestion window size would have dropped to 1.
- d) After the 22nd transmission round, segment loss is detected due to timeout, and hence the congestion window size is set to 1.
- e) The threshold is initially 32, since it is at this window size that slowstart stops and congestion avoidance begins.
- f) The threshold is set to half the value of the congestion window when packet loss is detected. When loss is detected during transmission round 16, the congestion window size is 42. Hence the threshold is 21 during the 18th transmission round.
- g) The threshold is set to half the value of the congestion window when packet loss is detected. When loss is detected during transmission round 22, the congestion window size is 26. Hence the threshold is 13 during the 24th transmission round.
- h) During the 1st transmission round, packet 1 is sent; packet 2-3 are sent in the 2nd transmission round; packets 4-7 are sent in the 3rd transmission round; packets 8-15 are sent in the 4th transmission round; packets 16-31 are sent in the 5th transmission round; packets 32-63 are sent in the 6th transmission round; packets 64 – 96 are sent in the 7th transmission round. Thus packet 70 is sent in the 7th transmission round.
- i) The congestion window and threshold will be set to half the current value of the congestion window (8) when the loss occurred. Thus the new values of the threshold and window will be 4.
- j) Threshold is 21, and congestion window size is 1.
- k) round 17, 1 packet; round 18, 2 packets; round 19, 4 packets; round 20, 8 packets; round 21, 16 packets; round 22, 21 packets. So, the total number is 52.

Problem 38

Refer to Figure 5. In Figure 5(a), the ratio of the linear decrease on loss between connection 1 and connection 2 is the same - as ratio of the linear increases: unity. In this case, the throughputs never move off of the AB line segment. In Figure 5(b), the ratio of

the linear decrease on loss between connection 1 and connection 2 is 2:1. That is, whenever there is a loss, connection 1 decreases its window by twice the amount of connection 2. We see that eventually, after enough losses, and subsequent increases, that connection 1's throughput will go to 0, and the full link bandwidth will be allocated to connection 2.

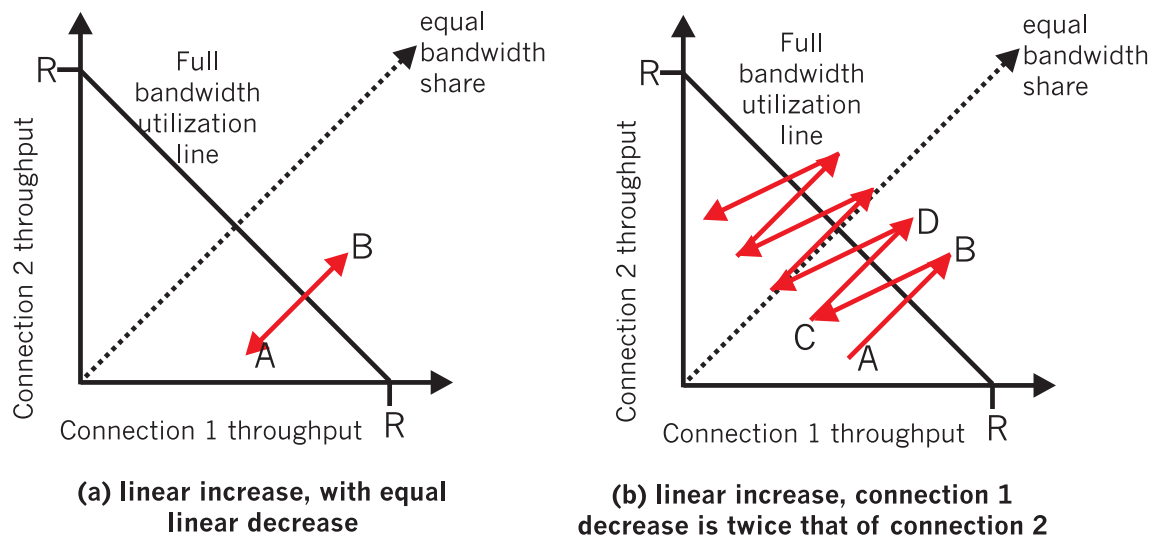


Figure 5: Lack of TCP convergence with linear increase, linear decrease

Problem 39

If TCP were a stop-and-wait protocol, then the doubling of the time out interval would suffice as a congestion control mechanism. However, TCP uses pipelining (and is therefore not a stop-and-wait protocol), which allows the sender to have multiple outstanding unacknowledged segments. The doubling of the timeout interval does not prevent a TCP sender from sending a large number of first-time-transmitted packets into the network, even when the end-to-end path is highly congested. Therefore a congestion-control mechanism is needed to stem the flow of “data received from the application above” when there are signs of network congestion.

Problem 40

In this problem, there is no danger in overflowing the receiver since the receiver's receive buffer can hold the entire file. Also, because there is no loss and acknowledgements are returned before timers expire, TCP congestion control does not throttle the sender. However, the process in host A will not continuously pass data to the socket because the send buffer will quickly fill up. Once the send buffer becomes full, the process will pass data at an average rate or $R \ll S$.

Problem 41

- a) It takes 1 RTT to increase CongWin to 6 MSS; 2 RTTs to increase to 7 MSS; 3 RTTs to increase to 8 MSS; 4 RTTs to increase to 9 MSS; 5 RTTs to increase to 10 MSS; and 6 RTTs to increase to 11 MSS.
- b) In the first RTT 5 MSS was sent; in the second RTT 6 MSS was sent; in the third RTT 7 MSS was sent; in the fourth RTT 8 MSS was sent; in the fifth RTT, 9 MSS was sent; and in the sixth RTT, 10 MSS was sent. Thus, up to time 6 RTT, $5+6+7+8+9+10 = 45$ MSS were sent (and acknowledged). Thus, we can say that the average throughput up to time 6 RTT was $(45 \text{ MSS})/(6 \text{ RTT}) = 7.5 \text{ MSS/RTT}$.

Problem 42

The loss rate, L , is the ratio of the number of packets lost over the number of packets sent. In a cycle, 1 packet is lost. The number of packets sent in a cycle is

$$\begin{aligned} \frac{W}{2} + \left(\frac{W}{2} + 1\right) + \cdots + W &= \sum_{n=0}^{W/2} \left(\frac{W}{2} + n\right) \\ &= \left(\frac{W}{2} + 1\right) \frac{W}{2} + \sum_{n=0}^{W/2} n \\ &= \left(\frac{W}{2} + 1\right) \frac{W}{2} + \frac{W/2(W/2 + 1)}{2} \\ &= \frac{W^2}{4} + \frac{W}{2} + \frac{W^2}{8} + \frac{W}{4} \\ &= \frac{3}{8}W^2 + \frac{3}{4}W \end{aligned}$$

Thus the loss rate is

$$L = \frac{1}{\frac{3}{8}W^2 + \frac{3}{4}W}$$

- b) For W large, $\frac{3}{8}W^2 \gg \frac{3}{4}W$. Thus $L \approx 8/3W^2$ or $W \approx \sqrt{\frac{8}{3L}}$. From the text, we therefore have

$$\text{average throughput} = \frac{3}{4} \sqrt{\frac{8}{3L}} \cdot \frac{\text{MSS}}{\text{RTT}}$$

$$= \frac{1.22 \cdot MSS}{RTT \cdot \sqrt{L}}$$

Problem 43

- Let W denote the max window size measured in segments. Then, $W \cdot MSS / RTT = 10\text{Mbps}$, as packets will be dropped if the maximum sending rate exceeds link capacity. Thus, we have $W \cdot 1500 \cdot 8 / 0.1 = 10 \cdot 10^6$, then W is about 84 (ceiling of 83.3) segments.
- As congestion window size varies from $W/2$ to W , then the average window size is $0.75W = 63$ segments. Average throughput is $63 \cdot 1500 \cdot 8 / 0.1 = 7.56\text{Mbps}$.
- $84/2 \cdot 0.1 = 4.2$ seconds, as the number of RTTs (that this TCP connections needs in order to increase its window size from $W/2$ to W) is given by $W/2$. Recall the window size increases by one in each RTT.

Problem 44

Let W denote max window size. Let S denote the buffer size. For simplicity, suppose TCP sender sends data packets in a round by round fashion, with each round corresponding to a RTT. If the window size reaches W , then a loss occurs. Then the sender will cut its congestion window size by half, and waits for the ACKs for $W/2$ outstanding packets before it starts sending data segments again. In order to make sure the link always busying sending data, we need to let the link busy sending data in the period $W/(2 \cdot C)$ (this is the time interval where the sender is waiting for the ACKs for the $W/2$ outstanding packets). Thus, S/C must be no less than $W/(2 \cdot C)$, that is, $S \geq W/2$.

Let T_p denote the one-way propagation delay between the sender and the receiver. When the window size reaches the minimum $W/2$ and the buffer is empty, we need to make sure the link is also busy sending data. Thus, we must have $W/2 / (2T_p) \geq C$, thus, $W/2 \geq C \cdot 2T_p$.

Thus, $S \geq C \cdot 2T_p$.

Problem 45

- Let W denote the max window size. Then, $W \cdot MSS / RTT = 10\text{Gbps}$, as packets will be dropped if maximum sending rate reaches link capacity. Thus, we have $W \cdot 1500 \cdot 8 / 0.1 = 10 \cdot 10^9$, then $W = 83334$ segments.
- As congestion window size varies from $W/2$ to W , then the average window size is $0.75W = 62501$ segments. Average throughput is $62501 \cdot 1500 \cdot 8 / 0.1 = 7.5\text{Gbps}$.
- $83334/2 \cdot 0.1 / 60 = 69$ minutes. In order to speed up the window increase process, we can increase the window size by a much larger value, instead of increasing window size only by one in each RTT. Some protocols are proposed to solve this problem, such as ScalableTCP or HighSpeed TCP.

Problem 46

As TCP's average throughput B is given by $B = \frac{1.22 \cdot MSS}{RTT \cdot \sqrt{L}}$, so we know that,

$$L = (1.22 \cdot MSS / (B \cdot RTT))^2$$

Since between two consecutive packet losses, there are 1/L packets sent by the TCP sender, thus, $T = (1/L) \cdot MSS/B$. Thus, we find that $T = B \cdot RTT^2 / (1.22^2 \cdot MSS)$, that is, T is a function of B.

Problem 47

a. The key difference between C1 and C2 is that C1's RTT is only half of that of C2. Thus C1 adjusts its window size after 100 msec, but C2 adjusts its window size after 200 msec.

Assume that whenever a loss event happens, C1 receives it after 100msec and C2 receives it after 200msec.

We further have the following simplified model of TCP.

After each RTT, a connection determines if it should increase window size or not. For C1, we compute the average total sending rate in the link in the previous 100 msec. If that rate exceeds the link capacity, then we assume that C1 detects loss and reduces its window size. But for C2, we compute the average total sending rate in the link in the previous 200msec. If that rate exceeds the link capacity, then we assume that C2 detects loss and reduces its window size.

Note that it is possible that the average sending rate in last 100msec is higher than the link capacity, but the average sending rate in last 200msec is smaller than or equal to the link capacity, then in this case, we assume that C1 will experience loss event but C2 will not.

The following table describes the evolution of window sizes and sending rates based on the above assumptions.

Time (msec)	C1		C2	
	Window Size (num. of segments sent in next 100msec)	Average data sending rate (segments per second, =Window/0.1)	Window Size(num. of segments sent in next 200msec)	Average data sending rate (segments per second, =Window/0.2)
0	10	100 (in [0-100]msec)	10	50 (in [0-100]msec)
100	5 (decreases window size as the avg. total sending rate to the link in last 100msec is 150=100+50)	50 (in [100-200]msec)		50 (in [100-200]msec)

200	2 (decreases window size as the avg. total sending rate to the link in last 100msec is $100 = 50 + 50$)	20	5 (decreases window size as the avg. total sending rate to the link in last 200msec is $125 = (100 + 50)/2 + (50 + 50)/2$)	25
300	1 (decreases window size as the avg. total sending rate to the link in last 100msec is $45 = (20 + 25)$)	10		25
400	1 (no further decrease, as window size is already 1)	10	2 (decreases window size as the avg. total sending rate to the link in last 200msec is $40 = (20 + 10)/2 + (25 + 25)/2$)	10
500	2	20		10
600	3	30	3	15
700	1	10		15
800	2	20	1	5
900	3	30		5
1000	1 (decreases window size as the avg. total sending rate to the link in last 100msec is $35 = (30 + 5)$)	10	2 (increases window size as the avg. total sending rate to the link in last 200msec is $30 = (20 + 30)/2 + (5 + 5)/2$)	10
1100	2	20		10
1200	3	30	3	15
1300	1	10		15
1400	2	20	1	5
1500	3	30		5
1600	1	10	2	10
1700	2	20		10
1800	3	30	3	15
1900	1	10		15
2000	2	20	1	5
2100	3	30		5
2200	1	10	2	10

Based on the above table, we find that after 2200 msec, C1's window size is 1 segment and C2's window size is 2 segments.

b. No. In the long run, C1's bandwidth share is roughly twice as that of C2's, because C1 has shorter RTT, only half of that of C2, so C1 can adjust its window size twice as fast as C2. If we look at the above table, we can see a cycle every 600msec, e.g. from 1400msec to 1900msec, inclusive. Within a cycle, the sending rate of C1 is $(20+30+10+20+30+10)/6 = 120$, which is twice as large as the sending of C2 given by $(5+5+10+10+15+15)/6=60$.

Problem 48

- a. Similarly as in last problem, we can compute their window sizes over time in the following table. Both C1 and C2 have the same window size 2 after 2200msec.

Time (msec)	C1		C2	
	Window Size (num. of segments sent in next 100msec)	Data sending speed (segments per second, =Window/0.1)	Window Size(num. of segments sent in next 100msec)	Data sending speed (segments per second, =Window/0.1)
0	15	150 (in [0-100]msec)	10	100 (in [0-100]msec)
100	7	70	5	50
200	3	30	2	20
300	1	10	1	10
400	2	20	2	20
500	1	10	1	10
600	2	20	2	20
700	1	10	1	10
800	2	20	2	20
900	1	10	1	10
1000	2	20	2	20
1100	1	10	1	10
1200	2	20	2	20
1300	1	10	1	10
1400	2	20	2	20
1500	1	10	1	10
1600	2	20	2	20
1700	1	10	1	10
1800	2	20	2	20
1900	1	10	1	10
2000	2	20	2	20
2100	1	10	1	10
2200	2	20	2	20

- b. Yes, this is due to the AIMD algorithm of TCP and that both connections have the same RTT.

- c. Yes, this can be seen clearly from the above table. Their max window size is 2.
- d. No, this synchronization won't help to improve link utilization, as these two connections act as a single connection oscillating between min and max window size. Thus, the link is not fully utilized (recall we assume this link has no buffer). One possible way to break the synchronization is to add a finite buffer to the link and randomly drop packets in the buffer before buffer overflow. This will cause different connections cut their window sizes at different times. There are many AQM (Active Queue Management) techniques to do that, such as RED (Random Early Detect), PI (Proportional and Integral AQM), AVQ (Adaptive Virtual Queue), and REM (Random Exponential Marking), etc.

Problem 49

Note that W represents the maximum window size.

First we can find the total number of segments sent out during the interval when TCP changes its window size from $W/2$ up to and include W . This is given by:

$$S = W/2 + (W/2)*(1+\alpha) + (W/2)*(1+\alpha)^2 + (W/2)*(1+\alpha)^3 + \dots + (W/2)*(1+\alpha)^k$$

We find $k = \log_{(1+\alpha)} 2$, then $S = W*(2\alpha+1)/(2\alpha)$.

Loss rate L is given by:

$$L = 1/S = (2\alpha) / (W*(2\alpha+1)).$$

The time that TCP takes to increase its window size from $W/2$ to W is given by:

$$k*RTT = (\log_{(1+\alpha)} 2) * RTT,$$

which is clearly independent of TCP's average throughput.

Note, TCP's average throughput is given by:

$$B = MSS * S / ((k+1)*RTT) = MSS / (L*(k+1)*RTT).$$

Note that this is different from TCP which has average throughput: $B = \frac{1.22 \cdot MSS}{RTT \cdot \sqrt{L}}$,

where the square root of L appears in the denominator.

Problem 50

Let's assume 1500-byte packets and a 100 ms round-trip time. From the TCP throughput

equation $B = \frac{1.22 \cdot MSS}{RTT \cdot \sqrt{L}}$, we have

$$10 \text{ Gbps} = 1.22 * (1500*8 \text{ bits}) / (.1 \text{ sec} * \sqrt{L}), \text{ or}$$

$$\sqrt{L} = 14640 \text{ bits} / (10^9 \text{ bits}) = 0.00001464, \text{ or}$$

$$L = 2.14 * 10^{-10}$$

Problem 51

An advantage of using the earlier values of `cwnd` and `ssthresh` at t_2 is that TCP would not have to go through slow start and congestion avoidance to ramp up to the throughput value obtained at t_1 . A disadvantage of using these values is that they may be no longer accurate. In particular, if the path has become more congested between t_1 and t_2 , the sender will send a large window's worth of segments into an already (more) congested path.

Problem 52

- a) The server will send its response to Y.
- b) The server can be certain that the client is indeed at Y. If it were at some other address spoofing Y, the SYNACK would have been sent to the address Y, and the TCP in that host would not send the TCP ACK segment back. Even if the attacker were to send an appropriately timed TCP ACK segment, it would not know the correct server sequence number (since the server uses random initial sequence numbers.)

Problem 53

a) Referring to the figure below, we see that the total delay is

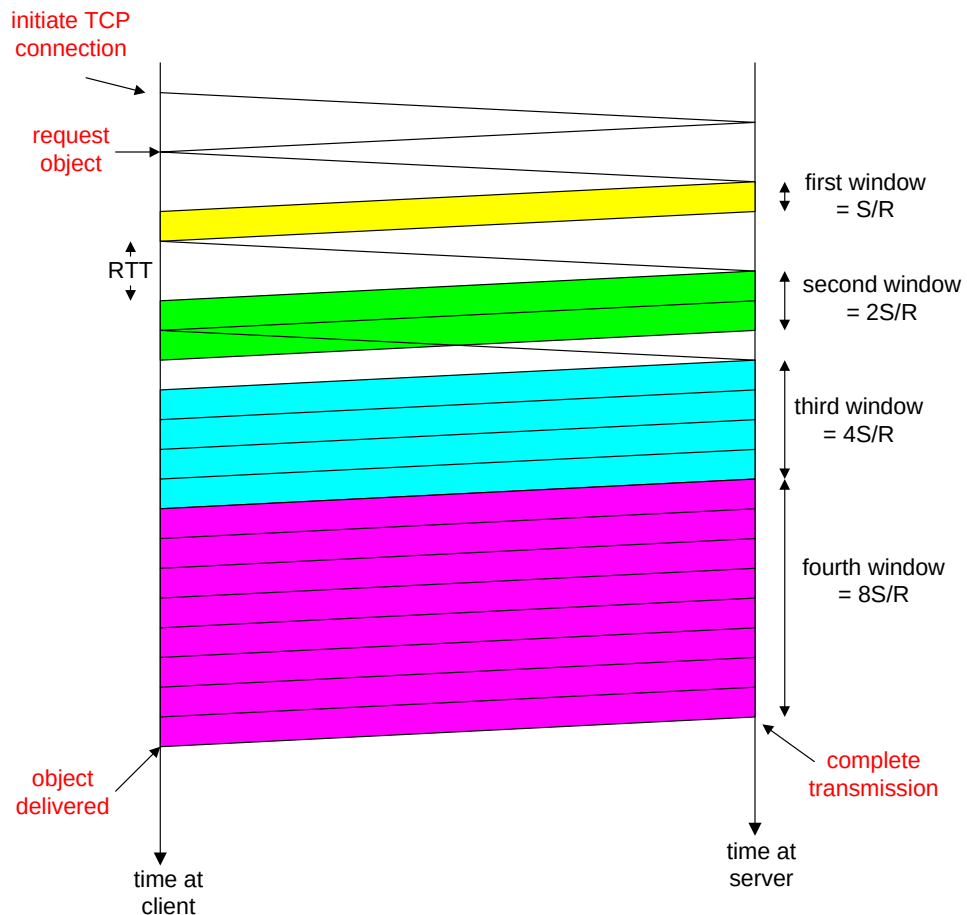
$$RTT + RTT + S/R + RTT + S/R + RTT + 12S/R = 4RTT + 14 S/R$$

b) Similarly, the delay in this case is:

$$RTT + RTT + S/R + RTT + S/R + RTT + S/R + RTT + 8S/R = 5RTT + 11 S/R$$

c) Similarly, the delay in this case is:

$$RTT + RTT + S/R + RTT + 14 S/R = 3 RTT + 15 S/R$$



Chapter 4 Review Questions

1. A network-layer packet is a datagram. A router forwards a packet based on the packet's IP (layer 3) address. A link-layer switch forwards a packet based on the packet's MAC (layer 2) address.
2. Datagram-based network layer: forwarding; routing. Additional function of VC-based network layer: call setup.
3. Forwarding is about moving a packet from a router's input link to the appropriate output link. Routing is about determining the end-to-end routes between sources and destinations.
4. Yes, both use forwarding tables. For descriptions of the tables, see Section 4.2.
5. Single packet: guaranteed delivery; guaranteed delivery with guaranteed maximum delay. Flow of packets: in-order packet delivery; guaranteed minimal bandwidth; guaranteed maximum jitter. None of these services is provided by the Internet's network layer. ATM's CBR service provides both guaranteed delivery and timing. ABR does not provide any of these services.
6. Interactive live multimedia applications, such as IP telephony and video conference, could benefit from ATM CBR's service, which maintains timing.
7. With the shadow copy, the forwarding decision is made locally, at each input port, without invoking the centralized routing processor. Such decentralized forwarding avoids creating a forwarding processing bottleneck at a single point within the router.
8. Switching via memory; switching via a bus; switching via an interconnection network
9. Packet loss occurs if queue size at the input port grows large because of slow switching fabric speed and thus exhausting router's buffer space. It can be eliminated if the switching fabric speed is at least n times as fast as the input line speed, where n is the number of input ports.
10. Packet loss can occur if the queue size at the output port grows large because of slow outgoing line-speed.
11. HOL blocking – a queued packet in an input queue must wait for transfer through the fabric because it is blocked by another packet at the head of the line. It occurs at the input port.
12. Yes. They have one address for each interface.

13. 11011111 00000001 00000011 00011100
14. Students will get different correct answers for this question.
15. 8 interfaces; 3 forwarding tables
16. 50% overhead
17. The 8-bit protocol field in the IP datagram contains information about which transport layer protocol the destination host should pass the segment to.
18. Typically the wireless router includes a DHCP server. DHCP is used to assign IP addresses to the 5 PCs and to the router interface. Yes, the wireless router also uses NAT as it obtains only one IP address from the ISP.
19. See Section 4.4.4
20. Yes, because the entire IPv6 datagram (including header fields) is encapsulated in an IPv4 datagram
21. Link state algorithms: Computes the least-cost path between source and destination using complete, global knowledge about the network. Distance-vector routing: The calculation of the least-cost path is carried out in an iterative, distributed manner. A node only knows the neighbor to which it should forward a packet in order to reach given destination along the least-cost path, and the cost of that path from itself to the destination.
22. Routers are aggregated into autonomous systems (ASs). Within an AS, all routers run the same intra-AS routing protocol. Special gateway routers in the various ASs run the inter-autonomous system routing protocol that determines the routing paths among the ASs. The problem of scale is solved since an intra-AS router need only know about routers within its AS and the gateway router(s) in its AS.
23. No. Each AS has administrative autonomy for routing within an AS.
24. No. The advertisement tells D that it can get to z in 11 hops by way of A. However, D can already get to z by way of B in 7 hops. Therefore, there is no need to modify the entry for z in the table. If, on the other hand, the advertisement said that A were only 4 hops away from z by way of C, then D would indeed modify its forwarding table.
25. With OSPF, a router periodically broadcasts routing information to all other routers in the AS, not just to its neighboring routers. This routing information sent by a router has one entry for each of the router's neighbors; the entry gives the distance from the router to the neighbor. A RIP advertisement sent by a router

contains information about all the networks in the AS, although this information is only sent to its neighboring routers.

26. “sequence of ASs on the routes”
27. See “Principles in Practice” on page 401
28. ISP C can use the BGP Multi-Exit Descriptor to suggest to ISP B that the preferred route to ISP D is through the east coast peering point. For example, the east coast BGP router in ISP C can advertise a route to D with an MED value of 5. The west coast router in ISP C can advertise a route to D with an MED value of 10. Since a lower value is preferred, ISP B knows that ISP C wants to receive traffic on the east coast. In practice, a router can ignore the MED value, and so ISP B can still use hot potato routing to pass traffic to ISP C destined to ISP D via the west coast peering point.
29. A **subnet** is a portion of a larger network; a subnet does not contain a router; its boundaries are defined by the router and host interfaces. A **prefix** is the network portion of a CDIRized address; it is written in the form a.b.c.d/x ; A prefix covers one or more subnets. When a router advertises a prefix across a BGP session, it includes with the prefix a number of BGP attributes. In BGP jargon, a prefix along with its attributes is a **BGP route** (or simply a **route**).
30. Routers use the AS-PATH attribute to detect and prevent looping advertisements; they also use it in choosing among multiple paths to the same prefix. The NEXT-HOP attribute indicates the IP address of the first router along an advertised path (outside of the AS receiving the advertisement) to a given prefix. When configuring its forwarding table, a router uses the NEXT-HOP attribute.
31. A tier-1 ISP B may not to carry transit traffic between two other tier-1 ISPs, say A and C, with which B has peering agreements. To implement this policy, ISP B would not advertise to A routes that pass through C; and would not advertise to C routes that pass through A.
32. N-way unicast has a number of drawbacks, including:
 - Efficiency: multiple copies of the same packet are sent over the same link for potentially many links; source must generate multiple copies of same packet
 - Addressing: the source must discover the address of all the recipients
33. a) uncontrolled flooding: T; controlled flooding: T; spanning-tree: F
b) uncontrolled flooding: T; controlled flooding: F; spanning-tree: F
34. False

35. IGMP is a protocol run only between the host and its first-hop multicast router. IGMP allows a host to specify (to the first-hop multicast router) the multicast group it wants to join. It is then up to the multicast router to work with other multicast routers (i.e., run a multicast routing protocol) to ensure that the data for the host-joined multicast group is routed to the appropriate last-hop router and from there to the host.
36. In a group-shared tree, all senders send their multicast traffic using the same routing tree. With source-based tree, the multicast datagrams from a given source are routed over a specific routing tree constructed for that source; thus each source may have a different source-based tree and a router may have to keep track of several source-based trees for a given multicast group.

Chapter 4 Problems

Problem 1

a) With a connection-oriented network, every router failure will involve the routing of that connection. At a minimum, this will require the router that is “upstream” from the failed router to establish a new downstream part of the path to the destination node, with all of the requisite signaling involved in setting up a path. Moreover, all of the routers on the initial path that are downstream from the failed node must take down the failed connection, with all of the requisite signaling involved to do this.

With a connectionless datagram network, no signaling is required to either set up a new downstream path or take down the old downstream path. We have seen, however, that routing tables will need to be updated (e.g., either via a distance vector algorithm or a link state algorithm) to take the failed router into account. We have seen that with distance vector algorithms, this routing table change can sometimes be localized to the area near the failed router. Thus, a datagram network would be preferable. Interestingly, the design criteria that the initial ARPAnet be able to function under stressful conditions was one of the reasons that datagram architecture was chosen for this Internet ancestor.

b) In order for a router to maintain an available fixed amount of capacity on the path between the source and destination node for that source-destination pair, it would need to know the characteristics of the traffic from all sessions passing through that link. That is, the router must have per-session state in the router. This is possible in a connection-oriented network, but not with a connectionless network. Thus, a connection-oriented VC network would be preferable.

c) In this scenario, datagram architecture has more control traffic overhead. This is due to the various packet headers needed to route the datagrams through the network. But in VC

architecture, once all circuits are set up, they will never change. Thus, the signaling overhead is negligible over the long run.

Problem 2

- Maximum number of VCs over a link = $2^8 = 256$.
- The centralized node could pick any VC number which is free from the set $\{0, 1, \dots, 2^8 - 1\}$. In this manner, it is not possible that there are fewer VCs in progress than 256 without there being any common free VC number.
- Each of the links can independently allocate VC numbers from the set $\{0, 1, \dots, 2^8 - 1\}$. Thus, a VC will likely have a different VC number for each link along its path. Each router in the VC's path must replace the VC number of each arriving packet with the VC number associated with the outbound link.

Problem 3

For a VC forwarding table, the columns are : Incoming Interface, Incoming VC Number, Outgoing Interface, Outgoing VC Number. For a datagram forwarding table, the columns are: Destination Address, Outgoing Interface.

Problem 4

- Data destined to host H3 is forwarded through interface 3

Destination Address	Link Interface
H3	3

- No, because forwarding rule is only based on destination address.

- | Incoming interface | Incoming VC# | Outgoing Interface | Outgoing VC# |
|--------------------|--------------|--------------------|--------------|
| 1 | 12 | 3 | 22 |
| 2 | 63 | 4 | 18 |

Note, those two flows (from H1 and H2) must have different VC#s, true for both incoming and outgoing VC#s.

-

Router B.

Incoming interface	Incoming VC#	Outgoing Interface	Outgoing VC#
1	22	2	24

Router C.

Incoming interface	Incoming VC#	Outgoing Interface	Outgoing VC#
1	18	2	50

Router D.

Incoming interface	Incoming VC#	Outgoing Interface	Outgoing VC#
1	24	3	70
2	50	3	76

Problem 5

- c) No VC number can be assigned to the new VC; thus the new VC cannot be established in the network.
- d) Each link has two available VC numbers. There are four links. So the number of combinations is $2^4 = 16$. One example combination is (10,00,00,10).

Problem 6

In a virtual circuit network, there *is* an end-to-end connection in the sense that each router along the path must maintain state for the connection; hence the terminology *connection service*. In a connection-oriented transport service over a connectionless network layer, such as TCP over IP, the end systems maintain connection state; however the routers have no notion of any connections; hence the terminology *connection-oriented service*.

Problem 7

To explain why there would be no input queuing, let's look at a specific design. For simplicity suppose each packet is the same size. We design the switch with time division multiplexing: time is broken into frames with each frame divided into n slots, with one slot needed to switch a packet through the fabric, and with one slot per frame devoted to each input line. Since at most one packet can arrive on each input line in each frame, the switching fabric will clear all packets in each frame.

Problem 8

The minimal number of time slots needed is 3. The scheduling is as follows.

Slot 1: send X in top input queue, send Y in middle input queue.

Slot 2: send X in middle input queue, send Y in bottom input queue

Slot 3: send Z in bottom input queue.

Largest number of slots is still 3. Actually, based on the assumption that a non-empty input queue is never idle, we see that the first time slot always consists of sending X in the top input queue and Y in either middle or bottom input queue, and in the second time

slot, we can always send two more datagram, and the last datagram can be sent in third time slot.

NOTE: Actually, if the first datagram in the bottom input queue is X, then the worst case would require 4 time slots.

Problem 9

a)

Prefix Match	Link Interface
11100000 00	0
11100000 01000000	1
1110000	2
11100001 1	3
otherwise	3

- b) Prefix match for first address is 5th entry: link interface 3
 Prefix match for second address is 3rd entry: link interface 2
 Prefix match for third address is 4th entry: link interface 3

Problem 10

Destination Address Range	Link Interface
00000000 through 00111111	0
01000000 through 01011111	1
01100000 through 01111111	2
10000000 through 10111111	2
11000000 through 11111111	3

number of addresses for interface 0 = $2^6 = 64$

number of addresses for interface 1 = $2^5 = 32$

number of addresses for interface 2 = $2^6 + 2^5 = 64 + 32 = 96$
 number of addresses for interface 3 = $2^6 = 64$

Problem 11

Destination Address Range	Link Interface
11000000 through (32 addresses) 11011111	0
10000000 through(64 addresses) 10111111	1
11100000 through (32 addresses) 11111111	2
00000000 through (128 addresses) 01111111	3

Problem 12

223.1.17.0/26
 223.1.17.128/25
 223.1.17.192/28

Problem 13

Destination Address	Link Interface
200.23.16/21	0
200.23.24/24	1
200.23.24/21	2
otherwise	3

Problem 14

Destination Address	Link Interface
---------------------	----------------

11100000 00 (224.0/10)	0
11100000 01000000 (224.64/16)	1
1110000 (224/8)	2
11100001 1 (225.128/9)	3
otherwise	3

Problem 15

Any IP address in range 128.119.40.128 to 128.119.40.191

Four equal size subnets: 128.119.40.64/28, 128.119.40.80/28, 128.119.40.96/28, 128.119.40.112/28

Problem 16

From 214.97.254/23, possible assignments are

- a) Subnet A: 214.97.255/24 (256 addresses)
 Subnet B: 214.97.254.0/25 - 214.97.254.0/29 (128-8 = 120 addresses)
 Subnet C: 214.97.254.128/25 (128 addresses)

Subnet D: 214.97.254.0/31 (2 addresses)
 Subnet E: 214.97.254.2/31 (2 addresses)
 Subnet F: 214.97.254.4/30 (4 addresses)

- b) To simplify the solution, assume that no datagrams have router interfaces as ultimate destinations. Also, label D, E, F for the upper-right, bottom, and upper-left interior subnets, respectively.

Router 1

Longest Prefix Match

Outgoing Interface

11010110 01100001 11111111	Subnet A
11010110 01100001 11111110 00000000	Subnet D
11010110 01100001 11111110 0000001	Subnet F

Router 2

Longest Prefix Match

Outgoing Interface

11010110 01100001 11111111 00000000	Subnet D
11010110 01100001 11111110 0	Subnet B
11010110 01100001 11111110 0000001	Subnet E

Router 3

Longest Prefix Match

11010110 01100001 11111111 000001
11010110 01100001 11111110 0000001
11010110 01100001 11111110 1

Outgoing Interface

Subnet F
Subnet E
Subnet C

Problem 17

The maximum size of data field in each fragment = 680 (because there are 20 bytes IP header). Thus the number of required fragments = $\left\lceil \frac{2400 - 20}{680} \right\rceil = 4$

Each fragment will have Identification number 422. Each fragment except the last one will be of size 700 bytes (including IP header). The last datagram will be of size 360 bytes (including IP header). The offsets of the 4 fragments will be 0, 85, 170, 255. Each of the first 3 fragments will have flag=1; the last fragment will have flag=0.

Problem 18

MP3 file size = 5 million bytes. Assume the data is carried in TCP segments, with each TCP segment also having 20 bytes of header. Then each datagram can carry 1500-40=1460 bytes of the MP3 file

Number of datagrams required = $\left\lceil \frac{5 \times 10^6}{1460} \right\rceil = 3425$. All but the last datagram will be

1,500 bytes; the last datagram will be 960+40 = 1000 bytes. Note that here there is not fragmentation – the source host does not create datagrams larger than 1500 bytes, and these datagrams are smaller than the MTUs of the links.

Problem 19

a) Home addresses: 192.168.1.1, 192.168.1.2, 192.168.1.3 with the router interface being 192.168.1.4

b)

NAT Translation Table

WAN Side	LAN Side
24.34.112.235, 4000	192.168.1.1, 3345
24.34.112.235, 4001	192.168.1.1, 3346

24.34.112.235, 4002	192.168.1.2, 3445
24.34.112.235, 4003	192.168.1.2, 3446
24.34.112.235, 4004	192.168.1.3, 3545
24.34.112.235, 4005	192.168.1.3, 3546

Problem 20

- a. Since all IP packets are sent outside, so we can use a packet sniffer to record all IP packets generated by the hosts behind a NAT. As each host generates a sequence of IP packets with sequential numbers and a distinct (very likely, as they are randomly chosen from a large space) initial identification number (ID), we can group IP packets with consecutive IDs into a cluster. The number of clusters is the number of hosts behind the NAT.

For more practical algorithms, see the following papers.

“A Technique for Counting NATted Hosts”, by Steven M. Bellovin, appeared in IMW’02, Nov. 6-8, 2002, Marseille, France.

“Exploiting the IPID field to infer network path and end-system characteristics.” Weifeng Chen, Yong Huang, Bruno F. Ribeiro, Kyoungwon Suh, Honggang Zhang, Edmundo de Souza e Silva, Jim Kurose, and Don Towsley. PAM’05 Workshop, March 31 - April 01, 2005. Boston, MA, USA.

- b. However, if those identification numbers are not sequentially assigned but randomly assigned, the technique suggested in part (a) won’t work, as there won’t be clusters in sniffed data.

Problem 21

It is not possible to devise such a technique. In order to establish a direct TCP connection between Arnold and Bernard, either Arnold or Bob must initiate a connection to the other. But the NATs covering Arnold and Bob drop SYN packets arriving from the WAN side. Thus neither Arnold nor Bob can initiate a TCP connection to the other if they are both behind NATs.

Problem 22

y-x-u, y-x-v-u, y-x-w-u, y-x-w-v-u,
y-w-u, y-w-v-u, y-w-x-u, y-w-x-v-u, y-w-v-x-u,
y-z-w-u, y-z-w-v-u, y-z-w-x-u, y-z-w-x-v-u, y-z-w-v-x-u,

Problem 23

x to z:

x-y-z, x-y-w-z,
x-w-z, x-w-y-z,

X-V-W-Z, X-V-W-Y-Z,
X-U-W-Z, X-U-W-Y-Z,
X-U-V-W-Z, X-U-V-W-Y-Z

z to u:

Z-W-U,
Z-W-V-U, Z-W-X-U, Z-W-V-X-U, Z-W-X-V-U, Z-W-Y-X-U, Z-W-Y-X-V-U,
Z-Y-X-U, Z-Y-X-V-U, Z-Y-X-W-U, Z-Y-X-W-Y-U, Z-Y-X-V-W-U,
Z-Y-W-V-U, Z-Y-W-X-U, Z-Y-W-V-X-U, Z-Y-W-X-V-U, Z-Y-W-Y-X-U, Z-Y-W-Y-X-V-U

z to w:

Z-W, Z-Y-W, Z-Y-X-W, Z-Y-X-V-W, Z-Y-X-U-W, Z-Y-X-U-V-W, Z-Y-X-V-U-W

Problem 24

Step	N'	$D(t), p(t)$	$D(u), p(u)$	$D(v), p(v)$	$D(w), p(w)$	$D(y), p(y)$	$D(z), p(z)$
0	x	∞	∞	3,x	6,x	6,x	8,x
1	xv	7,v	6,v	3,x	6,x	6,x	8,x
2	xvu	7,v	6,v	3,x	6,x	6,x	8,x
3	xvuw	7,v	6,v	3,x	6,x	6,x	8,x
4	xvuwy	7,v	6,v	3,x	6,x	6,x	8,x
5	xvuwyt	7,v	6,v	3,x	6,x	6,x	8,x
6	xvuwytz	7,v	6,v	3,x	6,x	6,x	8,x

Problem 25

a.

Step	N'	$D(x), p(x)$	$D(u), p(u)$	$D(v), p(v)$	$D(w), p(w)$	$D(y), p(y)$	$D(z), p(z)$
0	t	∞	2,t	4,t	∞	7,t	∞
1	tu	∞	2,t	4,t	5,u	7,t	∞
2	tuv	7,v	2,t	4,t	5,u	7,t	∞
3	tuvw	7,v	2,t	4,t	5,u	7,t	∞
4	tuvwx	7,v	2,t	4,t	5,u	7,t	15,x
5	tuvwxy	7,v	2,t	4,t	5,u	7,t	15,x
6	tuvwxyz	7,v	2,t	4,t	5,u	7,t	15,x

b.

Step	N'	$D(x), p(x)$	$D(t), p(t)$	$D(v), p(v)$	$D(w), p(w)$	$D(y), p(y)$	$D(z), p(z)$
------	------	--------------	--------------	--------------	--------------	--------------	--------------

u	∞	2,u	3,u	3,u	∞	∞
ut	∞	2,u	3,u	3,u	9,t	∞
utv	6,v	2,u	3,u	3,u	9,t	∞
utvw	6,v	2,u	3,u	3,u	9,t	∞
utvw x	6,v	2,u	3,u	3,u	9,t	14,x
utvw xy	6,v	2,u	3,u	3,u	9,t	14,x
utvw xyz	6,v	2,u	3,u	3,u	9,t	14,x

c.

Step	N'	$D(x), p(x)$	$D(u), p(u)$	$D(t), p(t)$	$D(w), p(w)$	$D(y), p(y)$	$D(z), p(z)$
	v	3,v	3,v	4,v	4,v	8,v	∞
	vx	3,v	3,v	4,v	4,v	8,v	11,x
	vxu	3,v	3,v	4,v	4,v	8,v	11,x
	vxut	3,v	3,v	4,v	4,v	8,v	11,x
	vxutw	3,v	3,v	4,v	4,v	8,v	11,x
	vxutwy	3,v	3,v	4,v	4,v	8,v	11,x
	vxutwyz	3,v	3,v	4,v	4,v	8,v	11,x

d.

Step	N'	$D(x), p(x)$	$D(u), p(u)$	$D(v), p(v)$	$D(t), p(t)$	$D(y), p(y)$	$D(z), p(z)$
	w	6,w	3,w	4,w	∞	∞	∞
	wu	6,w	3,w	4,w	5,u	∞	∞
	wuv	6,w	3,w	4,w	5,u	12,v	∞
	wuvt	6,w	3,w	4,w	5,u	12,v	∞
	wuvtx	6,w	3,w	4,w	5,u	12,v	14,x
	wuvtxy	6,w	3,w	4,w	5,u	12,v	14,x
	wuvtxyz	6,w	3,w	4,w	5,u	12,v	14,x

e.

Step	N'	$D(x), p(x)$	$D(u), p(u)$	$D(v), p(v)$	$D(w), p(w)$	$D(t), p(t)$	$D(z), p(z)$
	y	6,y	∞	8,y	∞	7,y	12,y
	yx	6,y	∞	8,y	12,x	7,y	12,y
	yxt	6,y	9,t	8,y	12,x	7,y	12,y
	yxtv	6,y	9,t	8,y	12,x	7,y	12,y
	yxtvu	6,y	9,t	8,y	12,x	7,y	12,y
	yxtvu w	6,y	9,t	8,y	12,x	7,y	12,y
	yxtvu wz	6,y	9,t	8,y	12,x	7,y	12,y

f.

Step	N'	$D(x), p(x)$	$D(u), p(u)$	$D(v), p(v)$	$D(w), p(w)$	$D(y), p(y)$	$D(t), p(t)$
	z	8,z	∞	∞	∞	12,z	∞
	zx	8,z	∞	11,x	14,x	12,z	∞
	zxv	8,z	14,v	11,x	14,x	12,z	15,v
	zxvy	8,z	14,v	11,x	14,x	12,z	15,v
	zxvyu	8,z	14,v	11,x	14,x	12,z	15,v
	zxvyuw	8,z	14,v	11,x	14,x	12,z	15,v
	zxvyuwt	8,z	14,v	11,x	14,x	12,z	15,v

Problem 26

		Cost to				
		u	v	x	y	z
From	v	∞	∞	∞	∞	∞
	x	∞	∞	∞	∞	∞
	z	∞	6	2	∞	0

		Cost to				
		u	v	x	y	z
From	v	1	0	3	∞	6
	x	∞	3	0	3	2
	z	7	5	2	5	0

		Cost to				
		u	v	x	y	z
From	v	1	0	3	3	5
	x	4	3	0	3	2
	z	6	5	2	5	0

		Cost to				
		u	v	x	y	z

	v	1	0	3	3	5
From	x	4	3	0	3	2
	z	6	5	2	5	0

Problem 27

The wording of this question was a bit ambiguous. We meant this to mean, “the number of iterations from when the algorithm is run for the first time” (that is, assuming the only information the nodes initially have is the cost to their nearest neighbors). We assume that the algorithm runs synchronously (that is, in one step, all nodes compute their distance tables at the same time and then exchange tables).

At each iteration, a node exchanges distance tables with its neighbors. Thus, if you are node A, and your neighbor is B, all of B's neighbors (which will all be one or two hops from you) will know the shortest cost path of one or two hops to you after one iteration (i.e., after B tells them its cost to you).

Let d be the “diameter” of the network - the length of the longest path without loops between any two nodes in the network. Using the reasoning above, after $d - 1$ iterations, all nodes will know the shortest path cost of d or fewer hops to all other nodes. Since any path with greater than d hops will have loops (and thus have a greater cost than that path with the loops removed), the algorithm will converge in at most $d - 1$ iterations.

ASIDE: if the DV algorithm is run as a result of a change in link costs, there is no a priori bound on the number of iterations required until convergence unless one also specifies a bound on link costs.

Problem 28

a. $D_x(w) = 2$, $D_x(y) = 4$, $D_x(u) = 7$

b.

First consider what happens if $c(x,y)$ changes. If $c(x,y)$ becomes larger or smaller (as long as $c(x,y) \geq 1$), the least cost path from x to u will still have cost at least 7. Thus a change in $c(x,y)$ (if $c(x,y) \geq 1$) will not cause x to inform its neighbors of any changes. If $c(x,y) = \delta < 1$, then the least cost path now passes through y and has cost $\delta + 6$.

Now consider if $c(x,w)$ changes. If $c(x,w) = \varepsilon \leq 1$, then the least-cost path to u continues to pass through w and its cost changes to $5 + \varepsilon$; x will inform its neighbors of this new cost. If $c(x,w) = \delta > 6$, then the least cost path now passes through y and has cost 11; again x will inform its neighbors of this new cost.

c. Any change in link cost $c(x,y)$ (and as long as $c(x,y) \geq 1$) will not cause x to inform its neighbors of a new minimum-cost path to u .

Problem 29

Node x table

		Cost to		
		x	y	z
From	x	0	3	4
	y	∞	∞	∞
	z	∞	∞	∞

		Cost to		
		x	y	z
From	x	0	3	4
	y	3	0	6
	z	4	6	0

Node y table

		Cost to		
		x	y	z
From	x	∞	∞	∞
	y	3	0	6
	z	∞	∞	∞

		Cost to		
		x	y	z
From	x	0	3	4
	y	3	0	6
	z	4	6	0

Node z table

		Cost to		
		x	y	z
From	x	∞	∞	∞
	y	∞	∞	∞
	z	4	6	0

		Cost to		
		x	y	z
From	x	0	3	4
	y	3	0	6
	z	4	6	0

Problem 30

NO, this is because that decreasing link cost won't cause a loop (caused by the next-hop relation of between two nodes of that link). Connecting two nodes with a link is equivalent to decreasing the link weight from infinite to the finite weight.

Problem 31

At each step, each updating of a node's distance vectors is based on the Bellman-Ford equation, i.e., only decreasing those values in its distance vector. There is no increasing in values. If no updating, then no message will be sent out. Thus, $D(x)$ is non-increasing. Since those costs are finite, then eventually distance vectors will be stabilized in finite steps.

Problem 32

a).

Router z	Informs w, $D_z(x)=\infty$
	Informs y, $D_z(x)=6$
Router w	Informs y, $D_w(x)=\infty$
	Informs z, $D_w(x)=5$
Router y	Informs w, $D_y(x)=4$
	Informs z, $D_y(x)=4$

b). Yes, there will be a count-to-infinity problem. The following table shows the routing converging process. Assume that at time t_0 , link cost change happens. At time t_1 , y updates its distance vector and informs neighbors w and z. In the following table, " $\rightarrow$ " stands for "informs".

time	t_0	t_1	t_2	t_3	t_4
z	$\rightarrow$ w, $D_z(x)=\infty$ $\rightarrow$ y, $D_z(x)=6$		No change	$\rightarrow$ w, $D_z(x)=\infty$ $\rightarrow$ y, $D_z(x)=11$	
w	$\rightarrow$ y, $D_w(x)=\infty$ $\rightarrow$ z, $D_w(x)=5$		$\rightarrow$ y, $D_w(x)=\infty$ $\rightarrow$ z, $D_w(x)=10$		No change
y	$\rightarrow$ w, $D_y(x)=4$ $\rightarrow$ z, $D_y(x)=4$	$\rightarrow$ w, $D_y(x)=9$ $\rightarrow$ z, $D_y(x)=\infty$		No change	$\rightarrow$ w, $D_y(x)=14$ $\rightarrow$ z, $D_y(x)=\infty$

We see that w, y, z form a loop in their computation of the costs to router x. If we continue the iterations shown in the above table, then we will see that, at t_{27} , z detects that its least cost to x is 50, via its direct link with x. At t_{29} , w learns its least cost to x is 51 via z. At t_{30} , y updates its least cost to x to be 52 (via w). Finally, at time t_{31} , no updating, and the routing is stabilized.

time	t_{27}	t_{28}	t_{29}	t_{30}	t_{31}
z	$\rightarrow$ w, $D_z(x)=50$ $\rightarrow$ y, $D_z(x)=50$				via w, ∞ via y, 55

					via z, 50
w		$\rightarrow y, D_w(x)=\infty$ $\rightarrow z, D_w(x)=50$	$\rightarrow y, D_w(x)=51$ $\rightarrow z, D_w(x)=\infty$		via w, ∞ via y, ∞ via z, 51
y		$\rightarrow w, D_y(x)=53$ $\rightarrow z, D_y(x)=\infty$		$\rightarrow w, D_y(x)=\infty$ $\rightarrow z, D_y(x)=52$	via w, 52 via y, 60 via z, 53

c). cut the link between y and z.

Problem 33

Since full AS path information is available from an AS to a destination in BGP, loop detection is simple – if a BGP peer receives a route that contains its own AS number in the AS path, then using that route would result in a loop.

Problem 34

The chosen path is not necessarily the shortest AS-path. Recall that there are many issues to be considered in the route selection process. It is very likely that a longer loop-free path is preferred over a shorter loop-free path due to economic reason. For example, an AS might prefer to send traffic to one neighbor instead of another neighbor with shorter AS distance.

Problem 35

- a. eBGP
- b. iBGP
- c. eBGP
- d. iBGP

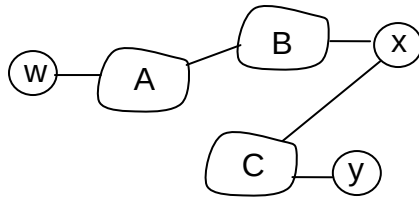
Problem 36

- a) I_1 because this interface begins the least cost path from $1d$ towards the gateway router $1c$.
- b) I_2 . Both routes have equal AS-PATH length but I_2 begins the path that has the closest NEXT-HOP router.
- c) I_1 . I_1 begins the path that has the shortest AS-PATH.

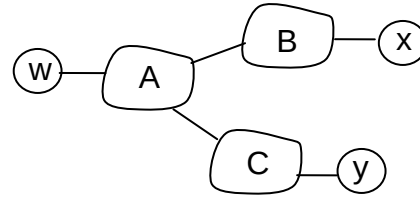
Problem 37

One way for C to force B to hand over all of B's traffic to D on the east coast is for C to only advertise its route to D via its east coast peering point with C.

Problem 38



X's view of the topology



W's view of the topology

In the above solution, X does not know about the AC link since X does not receive an advertised route to w or to y that contain the AC link (i.e., X receives no advertisement containing both AS A and AS C on the path to a destination).

Problem 39

BitTorrent file sharing and Skype P2P applications.

Consider a BitTorrent file sharing network in which peer 1, 2, and 3 are in stub networks W, X, and Y respectively. Due to the mechanism of BitTorrent's file sharing, it is possible that peer 2 gets data chunks from peer 1 and then forwards those data chunks to 3. This is equivalent to B forwarding data that is finally destined to stub network Y.

Problem 40

A should advise to B two routes, AS-paths A-W and A-V.

A should advise to C only one route, A-V.

C receives AS paths: B-A-W, B-A-V, A-V.

Problem 41

The minimal spanning tree has z connected to y via x at a cost of 14(=8+6).

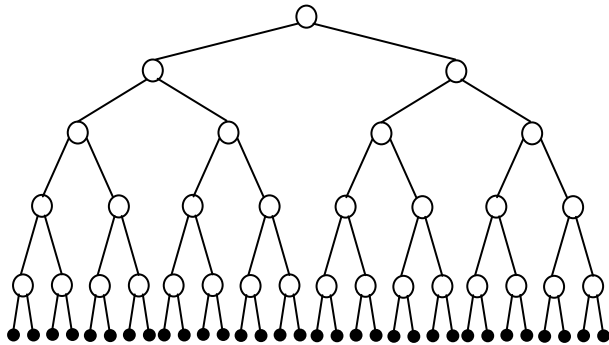
z connected to v via x at a cost of 11(=8+3);

z connected to u via x and v, at a cost of 14(=8+3+3);

z connected to w via x, v, and u, at a cost of 17(=8+3+3+3).

This can be obtained by Prim's algorithm to grow a minimum spanning tree.

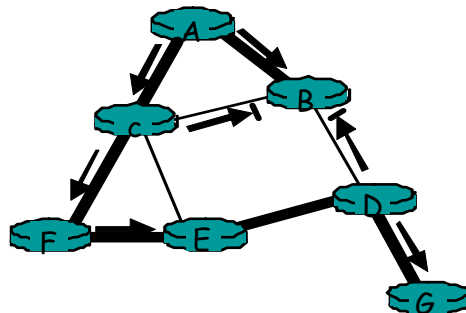
Problem 42



The 32 receivers are shown connected to the sender in the binary tree configuration shown above. With network-layer broadcast, a copy of the message is forwarded over each link exactly once. There are thus 62 link crossings ($2+4+8+16+32$). With unicast emulation, the sender unicasts a copy to each receiver over a path with 5 hops. There are thus 160 link crossings (5×32).

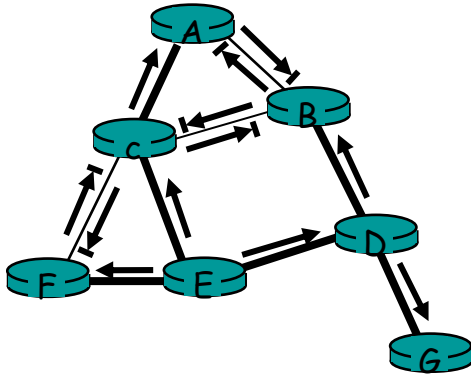
A topology in which all receivers are in a line, with the sender at one end of the line, will have the largest disparity between the cost of network-layer broadcast and unicast emulation.

Problem 43

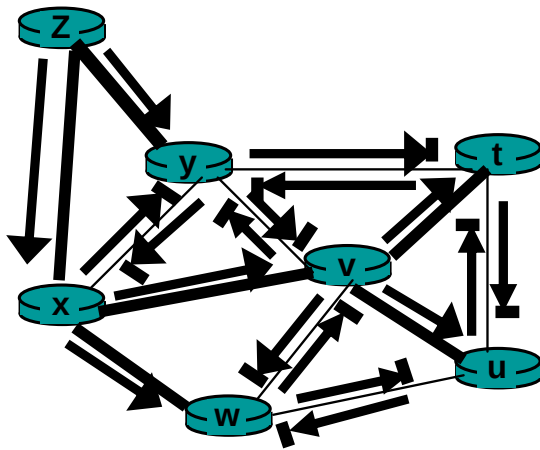


The thicker shaded lines represent the shortest path tree from A to all destination. Other solutions are possible, but in these solutions, B can not route to either C or D from A.

Problem 44



Problem 45



Problem 46

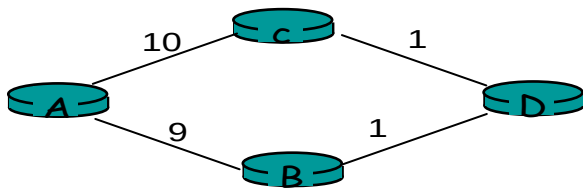
The center-based tree for the topology shown in the original figure connects A to C; B to C; E to C; and F to C (all directly). D connects to C via E, and G connects to C via D, E. This center-based tree is different from the minimal spanning tree shown in the figure.

Problem 47

The center-based tree for the topology shown in the original figure connects t to v; u to v; w to v; x to v; and y to v (all directly). And z connected to v via x. This center-based tree is different from the minimal spanning tree.

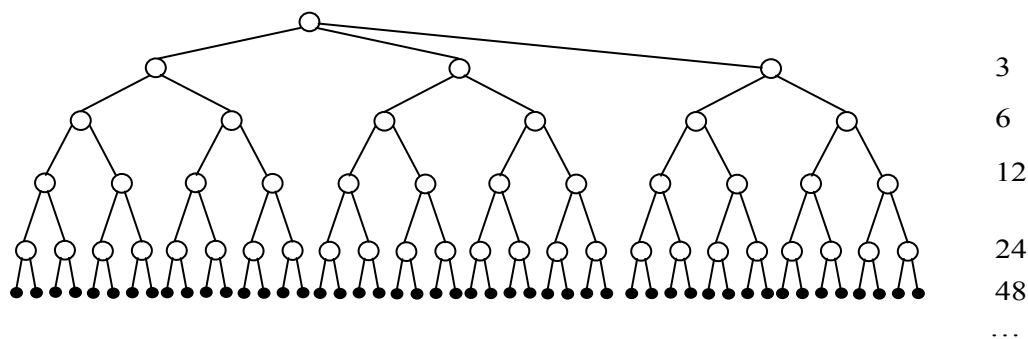
Problem 48

Dijkstra's algorithm for the network below, with node A as the source, results in a least-unicast-cost path tree of links AC, AB, and BD, with an overall free cost of 20. The minimum spanning tree contains links AB, BD, and DC, at a cost of 11.



Problem 49

After 1 step 3 copies are transmitted, after 2 steps 6 copies are transmitted. After 3 steps, 12 copies are transmitted, and so on. After k steps, $3 \cdot 2^{k-1}$ copies will be transmitted in that step.



Problem 50

The protocol must be built at the application layer. For example, an application may periodically multicast its identity to all other group members in an application-layer message.

Problem 51

A simple application-layer protocol that will allow all members to know the identity of all other members in the group is for each instance of the application to send a multicast message containing its identity to all other members. This protocol sends message in-band, since the multicast channel is used to distribute the identification messages as well as multicast data from the application itself. The use of the in-band signaling makes use of the existing multicast distribution mechanism, leading to a very simple design.

Problem 52

$32 - 4 = 28$ bits are available for multicast addresses. Thus, the size of the multicast address space is $N = 2^{28}$.

The probability that two groups choose the same address is

$$\frac{1}{N} = 2^{-28} = 3.73 \cdot 10^{-9}$$

The probability that 1000 groups all have different addresses is

$$\frac{N \cdot (N-1) \cdot (N-2) \cdots (N-999)}{N^{1000}} = \left(1 - \frac{1}{N}\right) \left(1 - \frac{2}{N}\right) \cdots \left(1 - \frac{999}{N}\right)$$

Ignoring cross-product terms, this is approximately equal to

$$1 - \left(\frac{1 + 2 + \cdots + 999}{N}\right) = 1 - \frac{999 \cdot 1000}{2N} = 0.998$$

Chapter 5 Review Questions

1. The transportation mode, e.g., car, bus, train, car.
2. Although each link guarantees that an IP datagram sent over the link will be received at the other end of the link without errors, it is not guaranteed that IP datagrams will arrive at the ultimate destination in the proper order. With IP, datagrams in the same TCP connection can take different routes in the network, and therefore arrive out of order. TCP is still needed to provide the receiving end of the application the byte stream in the correct order. Also, IP can lose packets due to routing loops or equipment failures.
3. Framing: there is also framing in IP and TCP; link access; reliable delivery: there is also reliable delivery in TCP; flow control: there is also flow control in TCP; error detection: there is also error detection in IP and TCP; error correction; full duplex: TCP is also full duplex.
4. There will be a collision in the sense that while a node is transmitting it will start to receive a packet from the other node.
5. Slotted Aloha: 1, 2 and 4 (slotted ALOHA is only partially decentralized, since it requires the clocks in all nodes to be synchronized). Token ring: 1, 2, 3, 4.
6. In polling, a discussion leader allows only one participant to talk at a time, with each participant getting a chance to talk in a round-robin fashion. For token ring, there isn't a discussion leader, but there is wine glass that the participants take turns holding. A participant is only allowed to talk if the participant is holding the wine glass.
7. When a node transmits a frame, the node has to wait for the frame to propagate around the entire ring before the node can release the token. Thus, if L/R is small as compared to t_{prop} , then the protocol will be inefficient.
8. 2^{48} MAC addresses; 2^{32} IPv4 addresses; 2^{128} IPv6 addresses.
9. C's adapter will process the frames, but the adapter will not pass the datagrams up the protocol stack. If the LAN broadcast address is used, then C's adapter will both process the frames and pass the datagrams up the protocol stack.
10. An ARP query is sent in a broadcast frame because the querying host does not which adapter address corresponds to the IP address in question. For the response, the sending node knows the adapter address to which the response should be sent, so there is no need to send a broadcast frame (which would have to be processed by all the other nodes on the LAN).

11. No it is not possible. Each LAN has its own distinct set of adapters attached to it, with each adapter having a unique LAN address.
12. The three Ethernet technologies have identical frame structures.
13. 20 million transitions per second.
14. After the 5th collision, the adapter chooses from $\{0, 1, 2, \dots, 31\}$. The probability that it chooses 4 is $1/32$. It waits 204.8 microseconds.
15. 2 (the internal subnet and the external internet)
16. In 802.1Q there is a 12- bit VLAN identifier. Thus $2^{12} = 4,096$ VLANs can be supported.
17. We can string the N switches together. The first and last switch would use one port for trunking; the middle $N-2$ switches would use two ports. So the total number of ports is $2 + 2(N-2) = 2N-2$ ports.

Chapter 5 Problems

Problem 1

```
1 1 1 0 1
1 0 1 1 1
1 0 0 1 0
1 1 0 1 1
0 0 0 1 1
```

Problem 2

Suppose we begin with the initial two-dimensional parity matrix:

```
0 0 0 0
1 1 1 1
0 1 0 1
1 0 1 0
```

With a bit error in row 2, column 3, the parity of row 2 and column 3 is now wrong in the matrix below:

```
0 0 0 0
1 1 0 1
0 1 0 1
1 0 1 0
```

Now suppose there is a bit error in row 2, column 2 and column 3. The parity of row 2 is now correct! The parity of columns 2 and 3 is wrong, but we can't detect in which rows the error occurred!

```
0 0 0 0
1 0 0 1
0 1 0 1
1 0 1 0
```

The above example shows that a double bit error can be detected (if not corrected).

Problem 3

```
  01001100 01101001
+ 01101110 01101011
-----
  10111010 11010100
+ 00100000 01001100
-----
```

```

  11011011 00100000
+ 01100001 01111001
-----

```

```

  00111100 10011010 (overflow, then wrap around)
+ 01100101 01110010
-----

```

10100010 00001100

The one's complement of the sum is 01011101 11110011

Problem 4

a) To compute the Internet checksum, we add up the values at 16-bit quantities:

```

00000001 00000010
00000011 00000100
00000101 00000110
00000111 00001000
00001001 00001010
-----
00011001 00011110

```

The one's complement of the sum is 11100110 11100001.

b) To compute the Internet checksum, we add up the values at 16-bit quantities:

```

01000001 01000010
01000011 01000100
01000101 01000110
01000111 01001000
01001001 01001010
-----
01011000 01011111

```

The one's complement of the sum is 10100111 10100000

c) To compute the Internet checksum, we add up the values at 16-bit quantities:

```

01100001 01100010
01100011 01100100
01100101 01100110
01100111 01100111
01101000 01101001
-----
11111001 11111101

```

The one's complement of the sum is 00000110 00000010.

Problem 5

If we divide 10011 into 1010101010 0000, we get 1011011100, with a remainder of R=0100. Note that, G=10011 is CRC-4-ITU standard.

Problem 6

- a) we get 1000100011, with a remainder of R=0101.
- b) we get 1011111111, with a remainder of R=0001.
- c) we get 0101101110, with a remainder of R=0010.

Problem 7

a. Without loss of generality, suppose i^{th} bit is flipped, where $0 \leq i \leq d+r-1$ and assume that the least significant bit is 0^{th} bit.

A single bit error means that the received data is $K = D * 2^r \text{ XOR } R + 2^i$. It is clear that if we divide K by G, then the remainder is not zero. In general, if G contains at least two 1's, then a single bit error can always be detected.

b. The key insight here is that G can be divided by 11 (binary number), but any number of odd-number of 1's cannot be divided by 11. Thus, a sequence (not necessarily contiguous) of odd-number bit errors cannot be divided by 11, thus it cannot be divided by G.

Problem 8

a)

$$\begin{aligned} E(p) &= Np(1-p)^{N-1} \\ E'(p) &= N(1-p)^{N-1} - Np(N-1)(1-p)^{N-2} \\ &= N(1-p)^{N-2}((1-p) - p(N-1)) \end{aligned}$$

$$E'(p) = 0 \Rightarrow p^* = \frac{1}{N}$$

b)

$$E(p^*) = N \frac{1}{N} \left(1 - \frac{1}{N}\right)^{N-1} = \left(1 - \frac{1}{N}\right)^{N-1} = \frac{\left(1 - \frac{1}{N}\right)^N}{1 - \frac{1}{N}}$$

$$\lim_{N \rightarrow \infty} \left(1 - \frac{1}{N}\right) = 1 \quad \lim_{N \rightarrow \infty} \left(1 - \frac{1}{N}\right)^N = \frac{1}{e}$$

Thus

$$\lim_{N \rightarrow \infty} E(p^*) = \frac{1}{e}$$

Problem 9

$$\begin{aligned} E(p) &= Np(1-p)^{2(N-1)} \\ E'(p) &= N(1-p)^{2(N-2)} - Np2(N-1)(1-p)^{2(N-3)} \\ &= N(1-p)^{2(N-3)}((1-p) - p2(N-1)) \end{aligned}$$

$$E'(p) = 0 \Rightarrow p^* = \frac{1}{2N-1}$$

$$E(p^*) = \frac{N}{2N-1} \left(1 - \frac{1}{2N-1}\right)^{2(N-1)}$$

$$\lim_{N \rightarrow \infty} E(p^*) = \frac{1}{2} \cdot \frac{1}{e} = \frac{1}{2e}$$

Problem 10

a). A's average throughput is given by $p_A(1-p_B)$.

Total efficiency is $p_A(1-p_B) + p_B(1-p_A)$.

b). A's throughput is $p_A(1-p_B) = 2p_B(1-p_B) = 2p_B - 2(p_B)^2$.

B's throughput is $p_B(1-p_A) = p_B(1-2p_B) = p_B - 2(p_B)^2$.

Clearly, A's throughput is not twice as large as B's.

In order to make $p_A(1-p_B) = 2p_B(1-p_A)$, we need that $p_A = 2 - (p_A/p_B)$.

c). A's throughput is $2p(1-p)^{N-1}$, and any other node has throughput $p(1-p)^{N-2}(1-2p)$.

Problem 11

a) $(1 - p(A))^4 p(A)$

where,

$p(A)$ = probability that A succeeds in a slot

$$\begin{aligned}
 p(A) &= p(\text{A transmits and B does not and C does not and D does not}) \\
 &= p(\text{A transmits}) p(\text{B does not transmit}) p(\text{C does not transmit}) p(\text{D does not transmit}) \\
 &= p(1-p)(1-p)(1-p) = p(1-p)^3
 \end{aligned}$$

$$\begin{aligned}
 &\text{Hence, } p(\text{A succeeds for first time in slot 5}) \\
 &= (1-p(A))^4 p(A) = (1-p(1-p)^3)^4 p(1-p)^3
 \end{aligned}$$

$$\begin{aligned}
 \text{b) } p(\text{A succeeds in slot 4}) &= p(1-p)^3 \\
 p(\text{B succeeds in slot 4}) &= p(1-p)^3 \\
 p(\text{C succeeds in slot 4}) &= p(1-p)^3 \\
 p(\text{D succeeds in slot 4}) &= p(1-p)^3
 \end{aligned}$$

$$\begin{aligned}
 p(\text{either A or B or C or D succeeds in slot 4}) &= 4 p(1-p)^3 \\
 &(\text{because these events are mutually exclusive})
 \end{aligned}$$

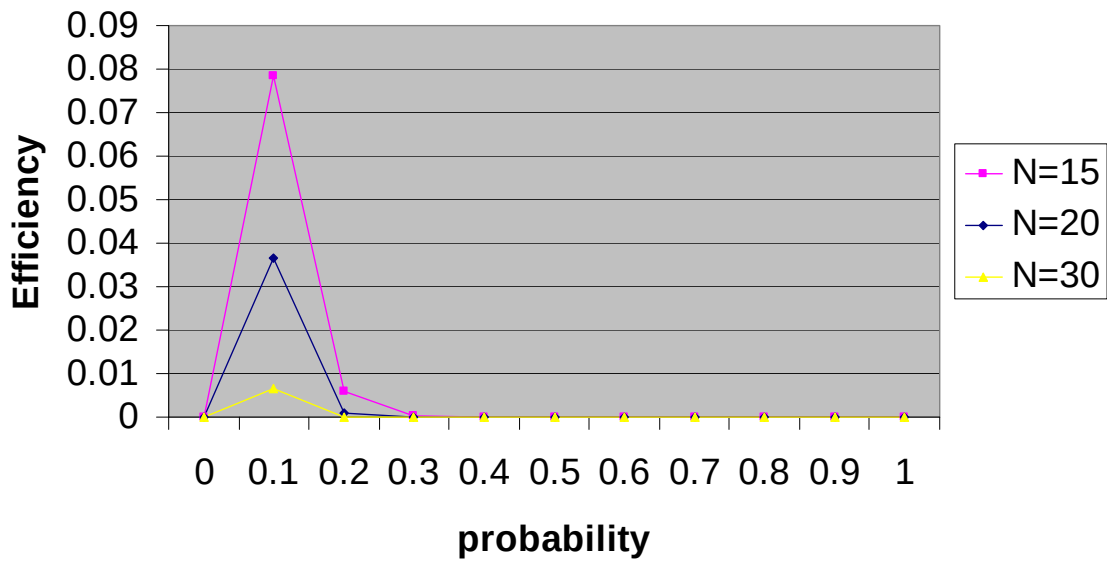
$$\begin{aligned}
 \text{c) } p(\text{some node succeeds in a slot}) &= 4 p(1-p)^3 \\
 p(\text{no node succeeds in a slot}) &= 1 - 4 p(1-p)^3
 \end{aligned}$$

$$\begin{aligned}
 &\text{Hence, } p(\text{first success occurs in slot 3}) = p(\text{no node succeeds in first 2 slots}) p(\text{some node succeeds in 3}^{\text{rd}} \text{ slot}) \\
 &= (1 - 4 p(1-p)^3)^2 4 p(1-p)^3
 \end{aligned}$$

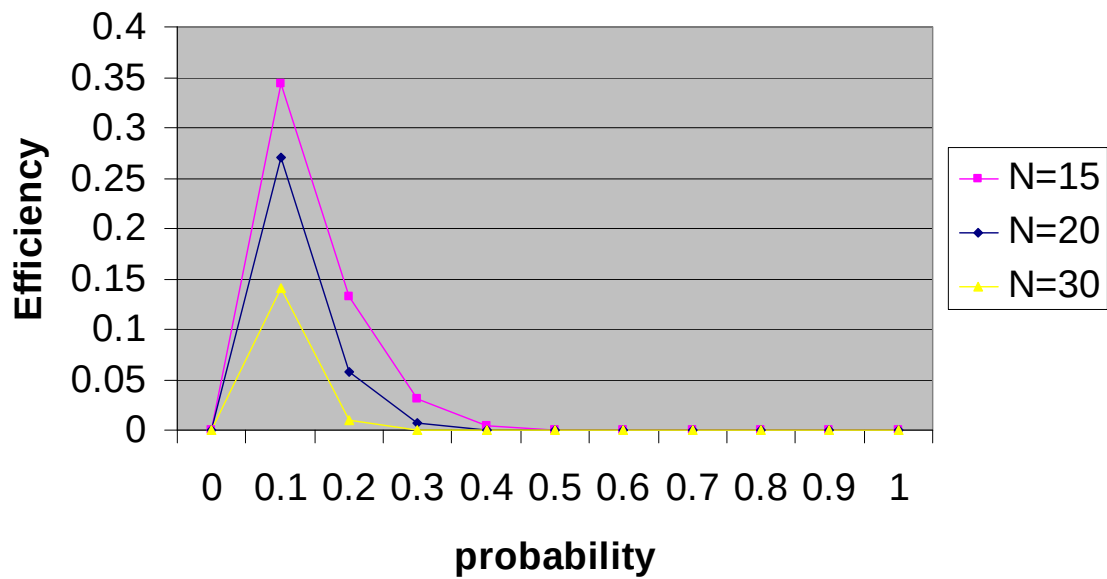
$$\text{d) efficiency} = p(\text{success in a slot}) = 4 p(1-p)^3$$

Problem 12

Pure ALOHA



Slotted ALOHA



Problem 13

The length of a polling round is

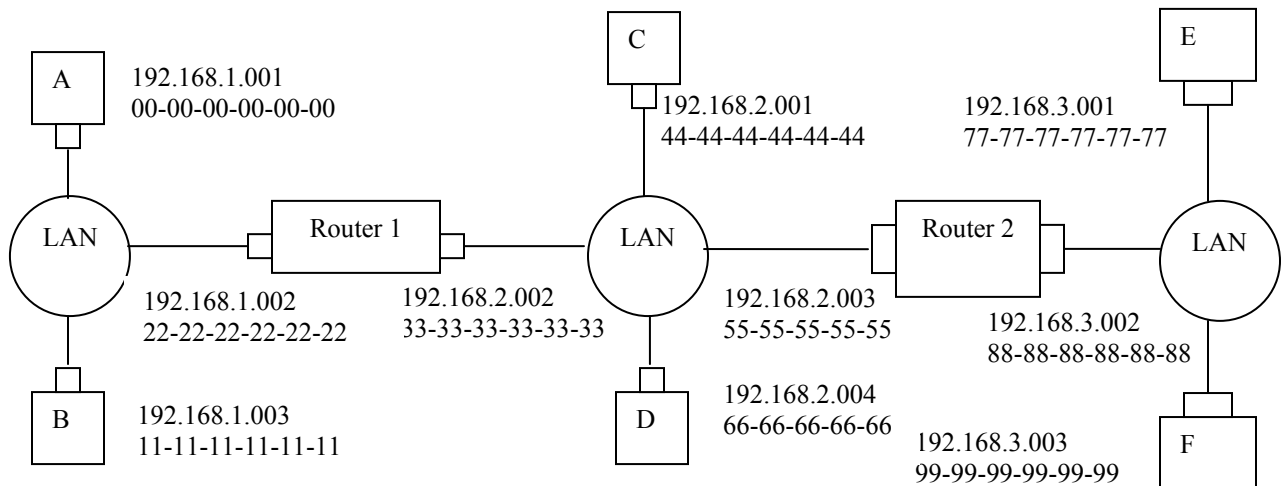
$$N(Q/R + d_{poll}).$$

The number of bits transmitted in a polling round is NQ . The maximum throughput therefore is

$$\frac{NQ}{N(Q/R + d_{poll})} = \frac{R}{1 + \frac{d_{poll} R}{Q}}$$

Problem 14

a), b) See figure below.



c)

1. Forwarding table in E determines that the datagram should be routed to interface 192.168.3.002.
2. The adapter in E creates an Ethernet packet with Ethernet destination address 88-88-88-88-88-88.
3. Router 2 receives the packet and extracts the datagram. The forwarding table in this router indicates that the datagram is to be routed to 198.162.2.002.
4. Router 2 then sends the Ethernet packet with the destination address of 33-33-33-33-33-33 and source address of 55-55-55-55-55-55 via its interface with IP address of 198.162.2.003.
5. The process continues until the packet has reached Host B.

d)

ARP in E must now determine the MAC address of 198.162.3.002. Host E sends out an ARP query packet within a broadcast Ethernet frame. Router 2 receives the query packet and sends to Host E an ARP response packet. This ARP response packet is carried by an Ethernet frame with Ethernet destination address 77-77-77-77-77-77.

Problem 15

a). No. E can check the subnet prefix of Host F's IP address, and then learn that F is on the same LAN. Thus, E will not send the packet to the default router R1.

Ethernet frame from E to F:

Source IP = E's IP address

Destination IP = F's IP address

Source MAC = E's MAC address

Destination MAC = F's MAC address

b). No, because they are not on the same LAN. E can find this out by checking B's IP address.

Ethernet frame from E to R1:

Source IP = E's IP address

Destination IP = B's IP address

Source MAC = E's MAC address

Destination MAC = The MAC address of R1's interface connecting to Subnet 3.

c).

Switch S1 will broadcast the Ethernet frame via both its interfaces as the received ARP frame's destination address is a broadcast address. And it learns that A resides on Subnet 1 which is connected to S1 at the interface connecting to Subnet 1. And, S1 will update its forwarding table to include an entry for Host A.

Yes, router R1 also receives this ARP request message, but R1 won't forward the message to Subnet 3.

B won't send ARP query message asking for A's MAC address, as this address can be obtained from A's query message.

Once switch S1 receives B's response message, it will add an entry for host B in its forwarding table, and then drop the received frame as destination host A is on the same interface as host B (i.e., A and B are on the same LAN segment).

Problem 16

Lets call the switch between subnets 2 and 3 S2. That is, *router R1 between subnets 2 and 3 is now replaced with switch S2.*

a). No. E can check the subnet prefix of Host F's IP address, and then learn that F is on the same LAN segment. Thus, E will not send the packet to S2.

Ethernet frame from E to F:

Source IP = E's IP address

Destination IP = F's IP address

Source MAC = E's MAC address

Destination MAC = F's MAC address

b). Yes, because E would like to find B's MAC address. In this case, E will send an ARP query packet with destination MAC address being the broadcast address.

This query packet will be re-broadcast by switch 1, and eventually received by Host B.

Ethernet frame from E to S2:

Source IP = E's IP address

Destination IP = B's IP address

Source MAC = E's MAC address

Destination MAC = broadcast MAC address: FF-FF-FF-FF-FF-FF.

c).

Switch S1 will broadcast the Ethernet frame via both its interfaces as the received ARP frame's destination address is a broadcast address. And it learns that A resides on Subnet 1 which is connected to S1 at the interface connecting to Subnet 1. And, S1 will update its forwarding table to include an entry for Host A.

Yes, router S2 also receives this ARP request message, and S2 will broadcast this query packet to all its interfaces.

B won't send ARP query message asking for A's MAC address, as this address can be obtained from A's query message.

Once switch S1 receives B's response message, it will add an entry for host B in its forwarding table, and then drop the received frame as destination host A is on the same interface as host B (i.e., A and B are on the same LAN segment).

Problem 17

Wait for 51,200 bit times. For 10 Mbps, this wait is

$$\frac{51.2 \times 10^3 \text{ bits}}{10 \times 10^6 \text{ bps}} = 5.12 \text{ msec}$$

For 100 Mbps, the wait is 512 μ sec.

Problem 18

At $t = 0$ A transmits. At $t = 576$, A would finish transmitting. In the worst case, B begins transmitting at time $t=324$, which is the time right before the first bit of A's frame arrives at B. At time $t=324+325=649$ B's first bit arrives at A. Because $649 > 576$, A finishes transmitting before it detects that B has transmitted. So A incorrectly thinks that its frame was successfully transmitted without a collision.

Problem 19

Following the same reasoning as in last problem, we need to make sure that one end of the Ethernet is able to detect the collision before it completes its transmission of a frame. Thus, a minimum frame size is required.

Let BW denote the bandwidth of the Ethernet. Consider the worst case for Ethernet's collision detection:

1. At $t=0$ (bit times): A sends a frame
2. At $t=d_{\text{prop}}-1$ (bit times): B sends a frame right before it can sense A's first bit.
3. At $t=2d_{\text{prop}}-2$ (bit times): If A finishes its transmission of its last bit just before B's frame arrives at A, then, A won't be able to detect a collision before finishing its transmission of a frame. Thus, in order for A to detect collision before finishing transmission, the minimum required frame size should be $\geq 2d_{\text{prop}}-1$ (bit times).

Assume that the signal propagation speed in 10BASE-T Ethernet is $1.8 \cdot 10^8$ m/sec. As $d_{\text{prop}} = d / (1.8 \cdot 10^8) \cdot \text{BW}$ (here, we need to convert the propagation delay in seconds to bit times for the specific Ethernet link), we find the minimum required frame size is $2 \cdot d / (1.8 \cdot 10^8) \cdot \text{BW} - 1$ (bits). Or approximately we choose $2 \cdot d / (1.8 \cdot 10^8) \cdot \text{BW}$ (bits). If $d=2\text{km}$, then minimum required frame size is: 222 bits.

Problem 20

Based on the solution of last problem, you know that a higher link speed requires a larger minimum required packet size. If you cannot change packet size, then you can add switches or routers to segment your LAN, in order to make sure each LAN segment's size is sufficiently small for small frame size.

Problem 21

Time, t	Event
0	A and B begin transmission
245	A and B detect collision
293	A and B finish transmitting jam signal
$293+245 = 538$	B's last bit arrives at A; A detects an idle channel
$538+96=634$	A starts transmitting
$293+512 = 805$	B returns to Step2 B must sense idle channel for 96 bit times before it transmits
$634+245=879$	A's transmission reaches B

Because A 's retransmission reaches B before B 's scheduled retransmission time $(805+96)$, B refrains from transmitting while A retransmits. Thus A and B do not collide. Thus the factor 512 appearing in the exponential backoff algorithm is sufficiently large.

Problem 22

A frame size is $1000 \cdot 8 + 64 = 8064$ bits, as 64-bit preamble is added into the frame.
 We want $1/(1+5a) = .5$ or, equivalently, $a = .2 = t_{prop} / t_{trans} \cdot t_{prop} = d / (1.8 \times 10^8) \text{ m/sec}$
 and $t_{trans} = (8064 \text{ bits}) / (10^8 \text{ bits/sec}) = 80.64 \mu \text{ sec}$. Solving for d we obtain $d = 2903$ meters.

For transmitting station A to detect whether any other station transmitted during A 's interval, t_{trans} must be greater than $2t_{prop} = 2 \cdot 2903 \text{ m} / 1.8 \times 10^8 \text{ m/sec} = 32.26 \mu \text{ sec}$.
 Because $32.26 < 80.64$, A will detect B 's signal before the end of its transmission.

Problem 23

a). minimum required frame length is given by
 $2 \cdot d_{prop} \cdot BW = 2 \cdot (500+700) / (2 \cdot 10^8) \cdot 10 \cdot 10^6 = 120 \text{ bits}$.
 There is no maximum required packet length.

b). Efficiency is given by
 $1 / (1 + 5 \cdot d_{prop} / d_{trans}) = 1 / (1 + 5 \cdot 120 / 2 / 1500) = 0.83$

Problem 24

a)

Let Y be a random variable denoting the number of slots until a success:

$$P(Y = m) = \beta(1 - \beta)^{m-1},$$

where β is the probability of a success.

This is a geometric distribution, which has mean $1/\beta$. The number of consecutive wasted slots is $X = Y - 1$ that

$$x = E[X] = E[Y] - 1 = \frac{1 - \beta}{\beta}$$

$$\beta = Np(1 - p)^{N-1}$$

$$x = \frac{1 - Np(1-p)^{N-1}}{Np(1-p)^{N-1}}$$

$$\text{efficiency} = \frac{k}{k+x} = \frac{k}{k + \frac{1 - Np(1-p)^{N-1}}{Np(1-p)^{N-1}}}$$

b)

Maximizing efficiency is equivalent to minimizing x , which is equivalent to maximizing β . We know from the text that β is maximized at $p = \frac{1}{N}$.

c)

$$\text{efficiency} = \frac{k}{k + \frac{1 - (1 - \frac{1}{N})^{N-1}}{(1 - \frac{1}{N})^{N-1}}}$$

$$\lim_{N \rightarrow \infty} \text{efficiency} = \frac{k}{k + \frac{1 - 1/e}{1/e}} = \frac{k}{k + e - 1}$$

d) Clearly, $\frac{k}{k + e - 1}$ approaches 1 as $k \rightarrow \infty$.

Problem 25

a)

$$\frac{800m}{2 \cdot 10^8 m/sec} + 4 \cdot \frac{20bits}{100 \times 10^6 bps}$$

$$= (4 \times 10^{-6} + 0.8 \times 10^{-6}) \text{sec}$$

$$= 4.8 \mu \text{sec}$$

b)

First note, the transmission time of a single frame is give by $1500/(100\text{Mbps})=15$ micro sec, longer than the propagation delay of a bit.

- At time $t = 0$, both A and B transmit.

- At time $t = 4.8\mu\text{sec}$, both A and B detect a collision, and then abort.
- At time $t = 9.6\mu\text{sec}$ last bit of B's aborted transmission arrives at A.
- At time $t = 14.4\mu\text{sec}$ first bit of A's retransmission frame arrives at B.
- At time $t = 14.4\mu\text{sec} + \frac{1500\text{bits}}{100 \times 10^6 \text{bps}} = 29.4\mu\text{sec}$ A's packet is completely delivered at B.

c) The line is divided into 5 segments by the switches, so the propagation delay between switches or between a switch and a host is given by $\frac{800\text{m}/5}{2 \cdot 10^8 \text{m/sec}} = 0.8\text{microsec}$.

The delay from Host A to the first switch is given by 15microsec (transmission delay), longer than propagation delay. Thus, the first switch will wait $16 = 15 + 0.8 + 0.2$ (note, 0.2 is processing delay) till it is ready to send the frame to the second switch. Note that the store-and-forward delay at a switch is 15 microsec. Similarly each of the other 3 switches will wait for 16 microsec before ready for transmitting the frame.

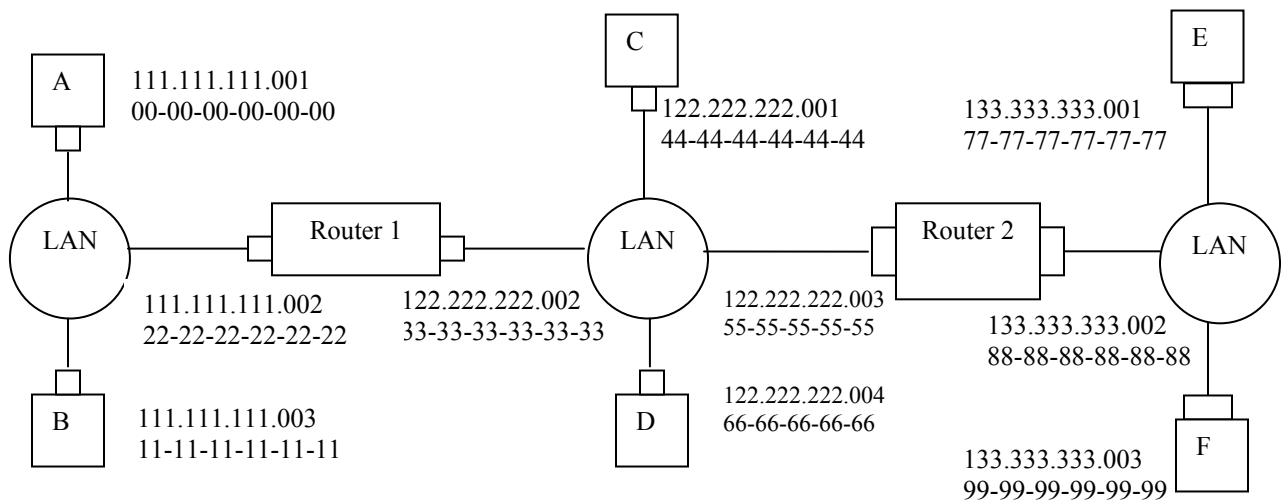
The total delay is:

$16 \cdot 4 + 15 + 0.8 = 79.8$ micro sec.

Problem 26

Higher speed of processor and memory implies that the inter-frame gap can be shortened, since it takes less time to complete the processing of a received frame. However, higher speed of Ethernet cable implies that the inter-frame gap should be increased.

Problem 27



- i) from A to left router: Source MAC address: 00-00-00-00-00-00
 Destination MAC address: 22-22-22-22-22-22
 Source IP: 111.111.111.001

Destination IP: 133.333.333.003

- ii) from the left router to the right router: Source MAC address: 33-33-33-33-33-33
Destination MAC address: 55-55-55-55-55-55
Source IP: 111.111.111.001
Destination IP: 133.333.333.003
- iii) from the right router to F: Source MAC address: 88-88-88-88-88-88
Destination MAC address: 99-99-99-99-99-99
Source IP: 111.111.111.001
Destination IP: 133.333.333.003

Problem 28

- i) from A to switch: Source MAC address: 00-00-00-00-00-00
Destination MAC address: 55-55-55-55-55-55
Source IP: 111.111.111.001
Destination IP: 133.333.333.003
- ii) from switch to right router: Source MAC address: 00-00-00-00-00-00
Destination MAC address: 55-55-55-55-55-55
Source IP: 111.111.111.001
Destination IP: 133.333.333.003
- iii) from right router to F: Source MAC address: 88-88-88-88-88-88
Destination MAC address: 99-99-99-99-99-99
Source IP: 111.111.111.001
Destination IP: 133.333.333.003

Problem 29

If all the $11=9+2$ nodes send out data at the maximum possible rate of 100 Mbps, a total aggregate throughput of $11 \times 100 = 1100$ Mbps is possible.

Problem 30

Each departmental hub is a single collision domain that can have a maximum throughput of 100 Mbps. The links connecting the web server and the mail server has a maximum throughput of 100 Mbps. Hence, if the three collision domains and the web server and mail server send out data at their maximum possible rates of 100 Mbps each, a maximum total aggregate throughput of 500 Mbps can be achieved among the 11 end systems.

Problem 31

All of the 11 end systems will lie in the same collision domain. In this case, the maximum total aggregate throughput of 100 Mbps is possible among the 11 end systems.

Problem 32

Action	Switch Table State	Link(s) packet is forwarded to	Explanation
B sends a frame to E	Switch learns interface corresponding to MAC address of B	A, C, D, E, and F	Since switch table is empty, so switch does not know the interface corresponding to MAC address of E
E replies with a frame to B	Switch learns interface corresponding to MAC address of E	B	Since switch already knows interface corresponding to MAC address of B
A sends a frame to B	Switch learns the interface corresponding to MAC address of A	B	Since switch already knows the interface corresponding to MAC address of B
B replies with a frame to A	Switch table state remains the same as before	A	Since switch already knows the interface corresponding to MAC address of A

Problem 33

The time required to fill $L \cdot 8$ bits is

$$\frac{L \cdot 8}{128 \times 10^3} \text{sec} = \frac{L}{16} \text{msec.}$$

b) For $L = 1,500$, the packetization delay is

$$\frac{1500}{16} \text{msec} = 93.75 \text{msec.}$$

For $L = 50$, the packetization delay is

$$\frac{50}{16} \text{msec} = 3.125 \text{msec.}$$

c)

$$\text{Store-and-forward delay} = \frac{L \cdot 8 + 40}{R}$$

For $L = 1,500$, the delay is

$$\frac{1500 \cdot 8 + 40}{622 \times 10^6} \text{ sec} \approx 19.4 \mu \text{ sec}$$

For $L = 50$, store-and-forward delay $< 1 \mu \text{ sec}$.

d) Store-and-forward delay is small for both cases for typical link speeds. However, packetization delay for $L = 1500$ is too large for real-time voice applications.

Problem 34

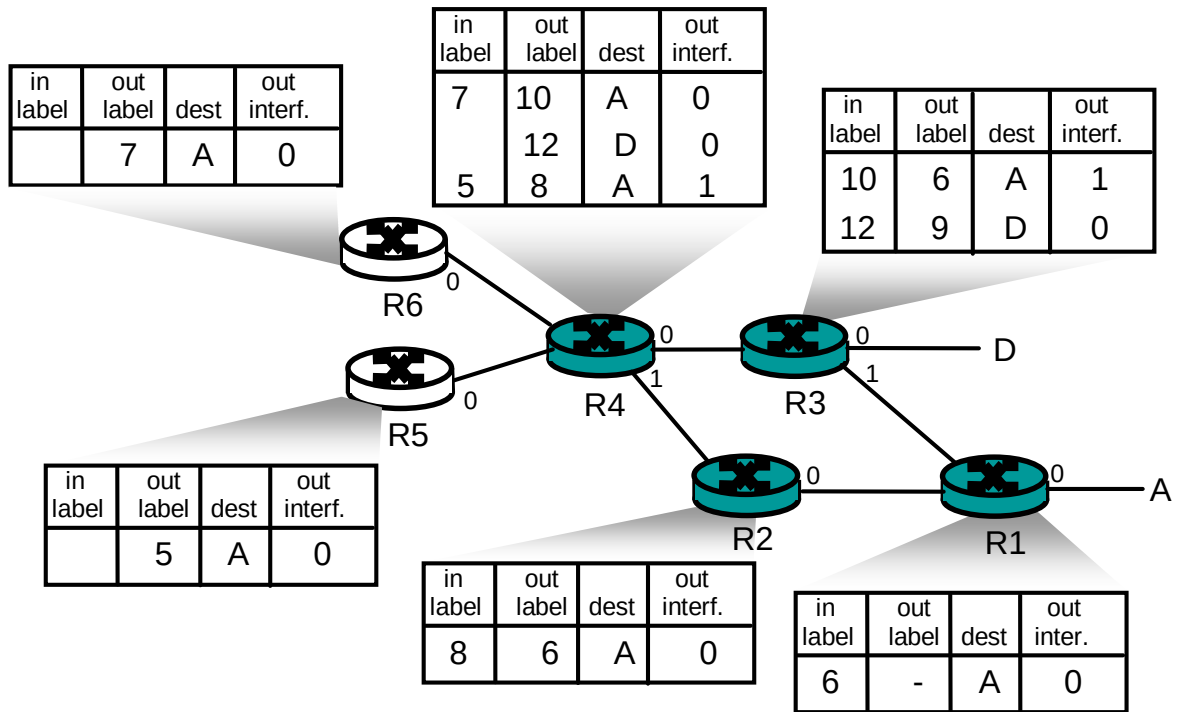
The IP addresses for those three computers (from left to right) in EE department are: 111.111.1.1, 111.111.1.2, 111.111.1.3. The subnet mask is 111.111.1/24.

The IP addresses for those three computers (from left to right) in CS department are: 111.111.2.1, 111.111.2.2, 111.111.2.3. The subnet mask is 111.111.2/24.

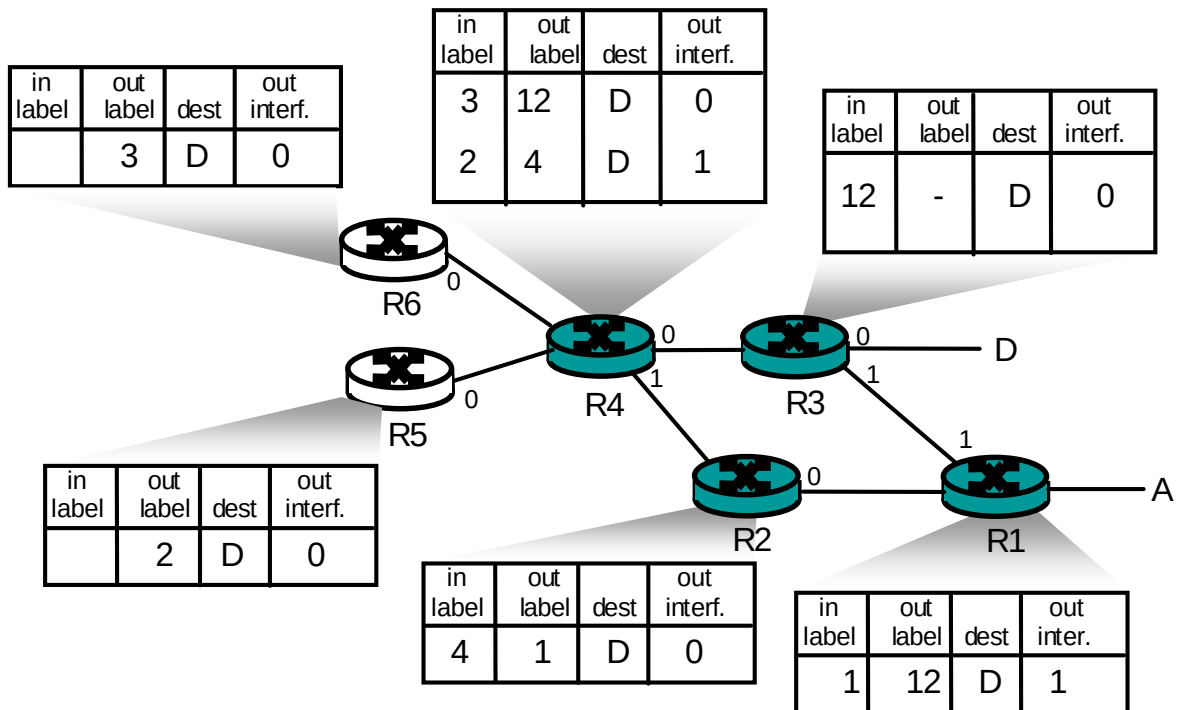
The router's interface card that connects to port 1 can be configured to contain two sub-interface IP addresses: 111.111.1.0 and 111.111.2.0. The first one is for the subnet of EE department, and the second one is for the subnet of CS department. Each IP address is associated with a VLAN ID. Suppose 111.111.1.0 is associated with VLAN 11, and 111.111.2.0 is associated with VLAN 12. This means that each frame that comes from subnet 111.111.1/24 will be added an 802.1q tag with VLAN ID 11, and each frame that comes from 111.111.2/24 will be added an 802.1q tag with VLAN ID 12.

Suppose that host A in EE department with IP address 111.111.1.1 would like to send an IP datagram to host B (111.111.2.1) in CS department. Host A first encapsulates the IP datagram (destined to 111.111.2.1) into a frame with a destination MAC address equal to the MAC address of the router's interface card that connects to port 1 of the switch. Once the router receives the frame, then it passes it up to IP layer, which decides that the IP datagram should be forwarded to subnet 111.111.2/24 via sub-interface 111.111.2.0. Then the router encapsulates the IP datagram into a frame and sends it to port 1. Note that this frame has an 802.1q tag VLAN ID 12. Once the switch receives the frame port 1, it knows that this frame is destined to VLAN with ID 12, so the switch will send the frame to Host B which is in CS department. Once Host B receives this frame, it will remove the 802.1q tag.

Problem 35



Problem 36



Problem 37

(The following description is short, but contains all major key steps and key protocols involved.)

Your computer first uses DHCP to obtain an IP address. Your computer first creates a special IP datagram destined to 255.255.255.255 in the DHCP server discovery step, and puts it in a Ethernet frame and broadcast it in the Ethernet. Then following the steps in the DHCP protocol, your computer is able to get an IP address with a given lease time.

A DHCP server on the Ethernet also gives your computer a list of IP addresses of first-hop routers, the subnet mask of the subnet where your computer resides, and the addresses of local DNS servers (if they exist).

Since your computer's ARP cache is initially empty, your computer will use ARP protocol to get the MAC addresses of the first-hop router and the local DNS server.

Your computer first will get the IP address of the Web page you would like to download. If the local DNS server does not have the IP address, then your computer will use DNS protocol to find the IP address of the Web page.

Once your computer has the IP address of the Web page, then it will send out the HTTP request via the first-hop router if the Web page does not reside in a local Web server. The HTTP request message will be segmented and encapsulated into TCP packets, and then further encapsulated into IP packets, and finally encapsulated into Ethernet frames. Your computer sends the Ethernet frames destined to the first-hop router. Once the router receives the frames, it passes them up into IP layer, checks its routing table, and then sends the packets to the right interface out of all of its interfaces.

Then your IP packets will be routed through the Internet until they reach the Web server.

The server hosting the Web page will send back the Web page to your computer via HTTP response messages. Those messages will be encapsulated into TCP packets and then further into IP packets. Those IP packets follow IP routes and finally reach your first-hop router, and then the router will forward those IP packets to your computer by encapsulating them into Ethernet frames.

Chapter 6 Review Questions

1. In infrastructure mode of operation, each wireless host is connected to the larger network via a base station (access point). If not operating in infrastructure mode, a network operates in ad-hoc mode. In ad-hoc mode, wireless hosts have no infrastructure with which to connect. In the absence of such infrastructure, the hosts themselves must provide for services such as routing, address assignment, DNS-like name translation, and more.
2.
 - a) Single hop, infrastructure-based
 - b) Single hop, infrastructure-less
 - c) Multi-hop, infrastructure-based
 - d) Multi-hop, infrastructure-less
3. Path loss is due to the attenuation of the electromagnetic signal when it travels through matter. Multipath propagation results in blurring of the received signal at the receiver and occurs when portions of the electromagnetic wave reflect off objects and ground, taking paths of different lengths between a sender and receiver. Interference from other sources occurs when the other source is also transmitting in the same frequency range as the wireless network.
4.
 - a) Increasing the transmission power
 - b) Reducing the transmission rate
5. APs transmit beacon frames. An AP's beacon frames will be transmitted over one of the 11 channels. The beacon frames permit nearby wireless stations to discover and identify the AP.
6. False
7. APs transmit beacon frames. An AP's beacon frames will be transmitted over one of the 11 channels. The beacon frames permit nearby wireless stations to discover and identify the AP.
8. False
9. Each wireless station can set an RTS threshold such that the RTS/CTS sequence is used only when the data frame to be transmitted is longer than the threshold. This ensures that RTS/CTS mechanism is used only for large frames.
10. No, there wouldn't be any advantage. Suppose there are two stations that want to transmit at the same time, and they both use RTS/CTS. If the RTS frame is as long as a DATA frames, the channel would be wasted for as long as it would have been wasted for two colliding DATA frames. Thus, the RTS/CTS exchange is only useful when the RTS/CTS frames are significantly smaller than the DATA frames.

11. Initially the switch has an entry in its forwarding table which associates the wireless station with the earlier AP. When the wireless station associates with the new AP, the new AP creates a frame with the wireless station's MAC address and broadcasts the frame. The frame is received by the switch. This forces the switch to update its forwarding table, so that frames destined to the wireless station are sent via the new AP.
12. Any ordinary Bluetooth node can be a master node whereas access points in 802.11 networks are special devices (normal wireless devices like laptops cannot be used as access points).
13. False
14. "Opportunistic Scheduling" refers to matching the physical layer protocol to channel conditions between the sender and the receiver, and choosing the receivers to which packets will be sent based on channel condition. This allows the base station to make best use of the wireless medium.
15. UMTS to GSM and CDMA-2000 to IS-95.
16. No. A node can remain connected to the same access point throughout its connection to the Internet (hence, not be mobile). A mobile node is the one that changes its point of attachment into the network over time. Since the user is always accessing the Internet through the same access point, she is not mobile.
17. A permanent address for a mobile node is its IP address when it is at its home network. A care-of-address is the one it gets when it is visiting a foreign network. The COA is assigned by the foreign agent (which can be the edge router in the foreign network or the mobile node itself).
18. False
19. The home network in GSM maintains a database called the home location register (HLR), which contains the permanent cell phone number and subscriber profile information about each of its subscribers. The HLR also contains information about the current locations of these subscribers. The visited network maintains a database known as the visitor location register (VLR) that contains an entry for each mobile user that is currently in the portion of the network served by the VLR. VLR entries thus come and go as mobile users enter and leave the network.

The edge router in home network in mobile IP is similar to the HLR in GSM and the edge router in foreign network is similar to the VLR in GSM.

20. Anchor MSC is the MSC visited by the mobile when a call first begins; anchor MSC thus remains unchanged during the call. Throughout the call's duration and regardless of the number of inter-MSC transfers performed by the mobile, the call is routed from the home MSC to the anchor MSC, and then from the anchor MSC to the visited MSC where the mobile is currently located.
21. a) Local recovery
 b) TCP sender awareness of wireless links
 c) Split-connection approaches

Chapter 6 Problems

Problem 1.

Output corresponding to bit $d_1 = [-1, 1, -1, 1, -1, 1, -1, 1]$

Output corresponding to bit $d_0 = [1, -1, 1, -1, 1, -1, 1, -1]$

Problem 2.

Sender 2 output = $[1, -1, 1, 1, 1, -1, 1, 1]; [1, -1, 1, 1, 1, -1, 1, 1]$

Problem 3.

$$d_2^1 = \frac{1 \times 1 + (-1) \times (-1) + 1 \times 1 + 1 \times 1 + 1 \times 1 + (-1) \times (-1) + 1 \times 1 + 1 \times 1}{8} = 1$$

$$d_2^2 = \frac{1 \times 1 + (-1) \times (-1) + 1 \times 1 + 1 \times 1 + 1 \times 1 + (-1) \times (-1) + 1 \times 1 + 1 \times 1}{8} = 1$$

Problem 4

Sender 1: (1, 1, 1, -1, 1, -1, -1, -1)

Sender 2: (1, -1, 1, 1, 1, 1, 1, 1)

Problem 5

- a) The two APs will typically have different SSIDs and MAC addresses. A wireless station arriving to the café will associate with one of the SSIDs (that is, one of the APs). After association, there is a virtual link between the new station and the AP. Label the APs AP1 and AP2. Suppose the new station associates with AP1. When the new station sends a frame, it will be addressed to AP1. Although AP2 will also receive the frame, it will not process the frame because the frame is not addressed to it. Thus, the two ISPs can work in parallel over the same channel. However, the two ISPs will be sharing the same wireless bandwidth. If wireless stations in different ISPs transmit at the same time, there will be a collision. For 802.11b, the maximum aggregate transmission rate for the two ISPs is 11 Mbps.

- b) Now if two wireless stations in different ISPs (and hence different channels) transmit at the same time, there will not be a collision. Thus, the maximum aggregate transmission rate for the two ISPs is 22 Mbps for 802.11b.

Problem 6

Suppose that wireless station H1 has 1000 long frames to transmit. (H1 may be an AP that is forwarding an MP3 to some other wireless station.) Suppose initially H1 is the only station that wants to transmit, but that while half-way through transmitting its first frame, H2 wants to transmit a frame. For simplicity, also suppose every station can hear every other station's signal (that is, no hidden terminals). Before transmitting, H2 will sense that the channel is busy, and therefore choose a random backoff value.

Now suppose that after sending its first frame, H1 returns to step 1; that is, it waits a short period of times (DIFS) and then starts to transmit the second frame. H1's second frame will then be transmitted while H2 is stuck in backoff, waiting for an idle channel. Thus, H1 should get to transmit all of its 1000 frames before H2 has a chance to access the channel. On the other hand, if H1 goes to step 2 after transmitting a frame, then it too chooses a random backoff value, thereby giving a fair chance to H2. Thus, fairness was the rationale behind this design choice.

Problem 7

A frame without data is 32 bytes long. Assuming a transmission rate of 11 Mbps, the time to transmit a control frame (such as an RTS frame, a CTS frame, or an ACK frame) is $(256 \text{ bits}) / (11 \text{ Mbps}) = 23 \text{ usec}$. The time required to transmit the data frame is $(8256 \text{ bits}) / (11 \text{ Mbps}) = 751$

$$\begin{aligned} & \text{DIFS} + \text{RTS} + \text{SIFS} + \text{CTS} + \text{SIFS} + \text{FRAME} + \text{SIFS} + \text{ACK} \\ &= \text{DIFS} + 3\text{SIFS} + (3 \cdot 23 + 751) \text{ usec} = \text{DIFS} + 3\text{SIFS} + 820 \text{ usec} \end{aligned}$$

Problem 8

- a) 1 message/ 2 slots
- b) 2 messages/slot
- c) 1 message/slot
- d) i) 1 message/slot
ii) 2 messages/slot
iii) 2 messages/slot
- e) i) 1 message/4 slots

ii) slot 1: Message $A \rightarrow B$, message $D \rightarrow C$

slot 2: Ack $B \rightarrow A$

slot 3: Ack $C \rightarrow D$

= 2 messages/ 3 slots

iii)

slot 1: Message $C \rightarrow D$

slot 2: Ack $D \rightarrow C$, message $A \rightarrow B$

slot 3: Ack $B \rightarrow A$

Repeat

= 2 messages/3 slots

Problem 10

a) 10 Mbps if it only transmits to node A. This solution is not fair since only A is getting served. By “fair” it means that each of the four nodes should be allotted equal number of slots.

b) For the fairness requirement such that each node receives an equal amount of data during each downstream sub-frame, let n_1 , n_2 , n_3 , and n_4 respectively represent the number of slots that A, B, C and D get.

Now,

data transmitted to A in 1 slot = $10t$ Mbits

(assuming the duration of each slot to be t)

Hence,

Total amount of data transmitted to A (in n_1 slots) = $10t n_1$

Similarly total amounts of data transmitted to B, C, and D equal to $5t n_2$, $2.5t n_3$, and $t n_4$ respectively.

Now, to fulfill the given fairness requirement, we have the following condition:

$$10t n_1 = 5t n_2 = 2.5t n_3 = t n_4$$

Hence,

$$n_2 = 2 n_1$$

$$n_3 = 4 n_1$$

$$n_4 = 10 n_1$$

Now, the total number of slots is N. Hence,

$$n_1 + n_2 + n_3 + n_4 = N$$

$$\text{i.e. } n_1 + 2n_1 + 4n_1 + 10n_1 = N$$

$$\text{i.e. } n_1 = N/17$$

Hence,

$$n_2 = 2N/17$$

$$n_3 = 4N/17$$

$$n_4 = 10N/17$$

The average transmission rate is given by:

$$\begin{aligned} & (10t n_1 + 5t n_2 + 2.5t n_3 + t n_4) / tN \\ &= (10N/17 + 5 * 2N/17 + 2.5 * 4N/17 + 1 * 10N/17) / N \\ &= 40/17 = 2.35 \text{ Mbps} \end{aligned}$$

- c) Let node A receives twice as much data as nodes B, C, and D during the sub-frame.

Hence,

$$10t n_1 = 2 * 5t n_2 = 2 * 2.5t n_3 = 2 * t n_4$$

$$\text{i.e. } n_2 = n_1$$

$$n_3 = 2n_1$$

$$n_4 = 5n_1$$

Again,

$$n_1 + n_2 + n_3 + n_4 = N$$

$$\text{i.e. } n_1 + n_1 + 2n_1 + 5n_1 = N$$

$$\text{i.e. } n_1 = N/9$$

Now, average transmission rate is given by:

$$\begin{aligned} & (10t n_1 + 5t n_2 + 2.5t n_3 + t n_4) / tN \\ &= 25/9 = 2.78 \text{ Mbps} \end{aligned}$$

Similarly, considering nodes B, C, or D receive twice as much data as any other nodes, different values for the average transmission rate can be calculated.

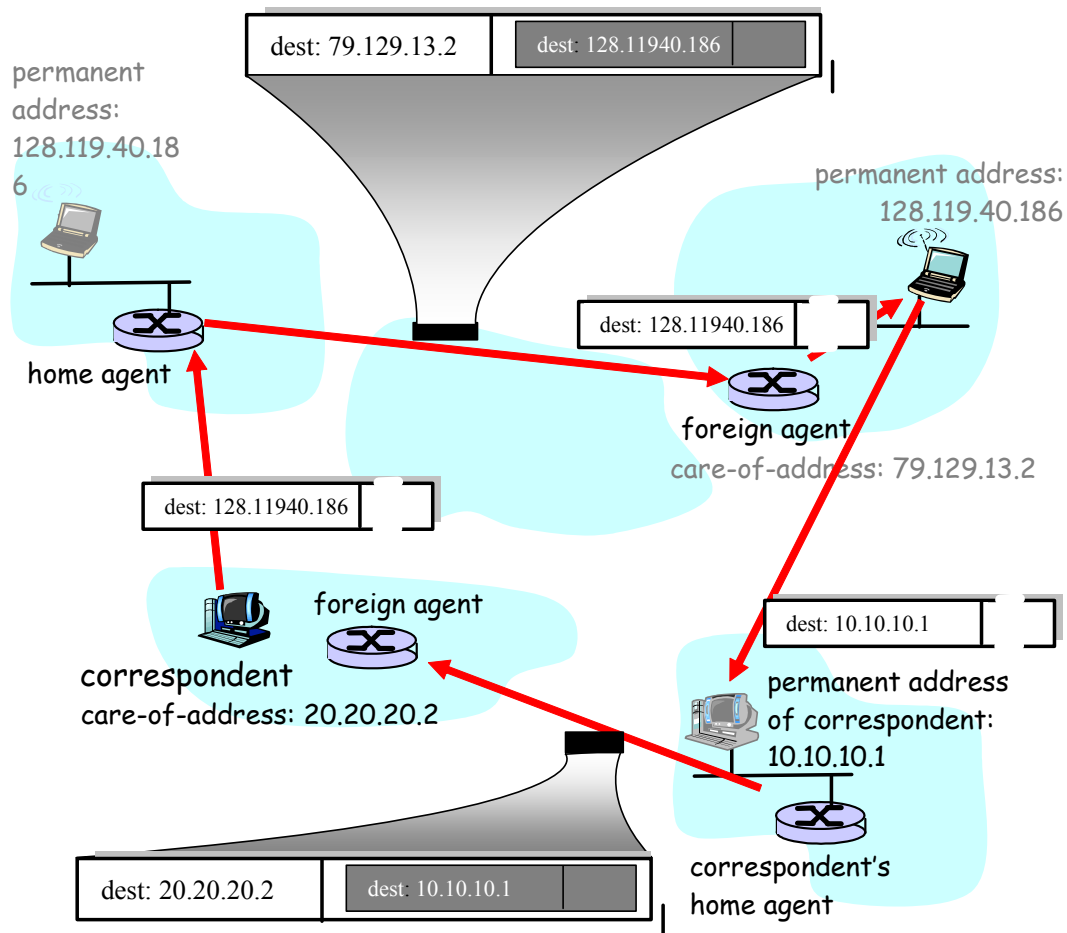
Problem 11

- a) No. All the routers might not be able to route the datagram immediately. This is because the Distance Vector algorithm (as well as the inter-AS routing protocols like BGP) is decentralized and takes some time to terminate. So, during the time when the algorithm is still running as a result of advertisements from the new foreign network, some of the routers may not be able to route datagrams destined to the mobile node.
- b) Yes. This might happen when one of the nodes has just left a foreign network and joined a new foreign network. In this situation, the routing entries from the old

foreign network might not have been completely withdrawn when the entries from the new network are being propagated.

- c) The time it takes for a router to learn a path to the mobile node depends on the number of hops between the router and the edge router of the foreign network for the node.

Problem 12



If the correspondent is mobile, then any datagrams destined to the correspondent would have to pass through the **correspondent's home agent**. The **foreign agent** in the network being visited also needs to be involved, since it is this foreign agent that notifies the correspondent's home agent of the location of the correspondent. Datagrams received by the correspondent's home agent would need to be encapsulated/tunneled between the correspondent's home agent and foreign agent, (as in the case of the encapsulated diagram at the top of Figure 6.23).

Problem 13

Because datagrams must be first forwarded to the home agent, and from there to the mobile, the delays will generally be longer than via direct routing. Note that it is possible, however, that the direct delay from the correspondent to the mobile (i.e., if the datagram is not routed through the home agent) could actually be smaller than the sum of the delay

from the correspondent to the home agent and from there to the mobile. It would depend on the delays on these various path segments. Note that indirect routing also adds a home agent processing (e.g., encapsulation) delay.

Problem 14

First, we note that chaining was discussed at the end of section 6.5. In the case of chaining using indirect routing through a home agent, the following events would happen:

- The mobile node arrives at A, A notifies the home agent that the mobile is now visiting A and that datagrams to the mobile should now be forwarded to the specified care-of-address (COA) in A.
- The mobile node moves to B. The foreign agent at B must notify the foreign agent at A that the mobile is no longer resident in A but in fact is resident in B and has the specified COA in B. From then on, the foreign agent in A will forward datagrams it receives that are addressed to the mobile's COA in A to the mobile's COA in B.
- The mobile node moves to C. The foreign agent at C must notify the foreign agent at B that the mobile is no longer resident in B but in fact is resident in C and has the specified COA in C. From then on, the foreign agent in B will forward datagrams it receives (from the foreign agent in A) that are addressed to the mobile's COA in B to the mobile's COA in C.

Note that when the mobile goes offline (i.e., has no address) or returns to its home network, the datagram-forwarding state maintained by the foreign agents in A, B and C must be removed. This teardown must also be done through signaling messages. Note that the home agent is not aware of the mobile's mobility beyond A, and that the correspondent is not at all aware of the mobile's mobility.

In the case that chaining is not used, the following events would happen:

- The mobile node arrives at A, A notifies the home agent that the mobile is now visiting A and that datagrams to the mobile should now be forwarded to the specified care-of-address (COA) in A.
- The mobile node moves to B. The foreign agent at B must notify the foreign agent at A and the home agent that the mobile is no longer resident in A but in fact is resident in B and has the specified COA in B. The foreign agent in A can remove its state about the mobile, since it is no longer in A. From then on, the home agent will forward datagrams it receives that are addressed to the mobile's COA in B.
- The mobile node moves to C. The foreign agent at C must notify the foreign agent at B and the home agent that the mobile is no longer resident in B but in fact is resident in C and has the specified COA in C. The foreign agent in B can remove its state about the mobile, since it is no longer in B. From then on, the home agent will forward datagrams it receives that are addressed to the mobile's COA in C.

When the mobile goes offline or returns to its home network, the datagram-forwarding state maintained by the foreign agent in C must be removed. This teardown must also be done through signaling messages. Note that the home agent is always aware of the mobile's current foreign network. However, the correspondent is still blissfully unaware of the mobile's mobility.

Problem 15

Two mobiles could certainly have the same care-of-address in the same visited network. Indeed, if the care-of-address is the address of the foreign agent, then this address would be the same. Once the foreign agent decapsulates the tunneled datagram and determines the address of the mobile, then separate addresses would need to be used to send the datagrams separately to their different destinations (mobiles) within the visited network.

Problem 16

If the MSRN is provided to the HLR, then the value of the MSRN must be updated in the HLR whenever the MSRN changes (e.g., when there is a handoff that requires the MSRN to change). The advantage of having the MSRN in the HLR is that the value can be provided quickly, without querying the VLR. By providing the address of the VLR (rather than the MSRN), there is no need to be refreshing the MSRN in the HLR.

Chapter 7 Review Questions

1. Streaming stored audio/video: pause/resume, re-positioning, fast-forward; real-time interactive audio and video: people communicating and responding in real time.
2. Camp 1: No fundamental changes in TCP/IP protocols; add bandwidth where needed; also use caching, content distribution networks, and multicast overlay networks. Camp 2: Provide a network service that allows applications to reserve bandwidth in the network. Camp 3, differentiated service: introduce simple classifying and policing schemes at the edge of the network, and give different datagrams different levels of service according to their class in the router queues.
3. Uncompressed audio stored on CD has a bit rate of 1411.2 Kbps. mp3 files are typically encoded in 128 Kbps or less, thereby giving them a compression ratio of almost 11. Image compression ratios are in the range of 10 to 100.
4. Figure 6.1: simple, doesn't require meta file or streaming server; Figure 6.2: allows media player to interact directly with the web server, doesn't require a streaming server; Figure 6.3: media player interacts directly with a streaming server, which has been designed for the specific streaming application.
5. End-to-end delay is the time it takes a packet to travel across the network from source to destination. Delay jitter is the fluctuation of end-to-end delay from packet to the next packet.
6. A packet that arrives after its scheduled play out time can not be played out. Therefore, from the perspective of the application, the packet has been lost.
7. First scheme: send a redundant encoded chunk after every n chunks; the redundant chunk is obtained by exclusive OR-ing the n original chunks. Second scheme: send a lower-resolution low-bit rate scheme along with the original stream. Interleaving does not increase the bandwidth requirements of a stream.
8. The role of the DNS is to forward HTTP requests to DNS server managed by the CDN, which in turn redirects the request to an appropriate CDN server. The DNS does not have to be modified to support a CDN. A CDN should provide DNS with the host name and IP address of its authoritative name server (See Section 2.5.3).
9.
 - a) Models of traffic demand between network end points
 - b) Well-defined performance requirements
 - c) Models to predict end-end performance for a given workload model, and techniques to find a minimal high cost bandwidth allocation that will result in all user requirements being met.

10. RTP streams in different sessions: different multicast addresses; RTP streams in the same session: SSRC field; RTP packets are distinguished from RTCP packets by using distinct port numbers.
11. Reception report packets: includes info about fraction of packets lost, last sequence number, inter-arrival jitter; sender report packets: timestamp and wall clock time of most recently generated RTP packet, number of packets sent, number of bytes sent ; source description packets: e-mail address of the sender, the sender's name, the application that generates the RTP stream.
12. The role of a SIP registrar is to keep track of the users and their corresponding IP addresses which they are currently using. Each SIP registrar keeps track of the users that belong to its domain. It also forwards INVITE messages (for users in its domain) to the IP address which the user is currently using. In this regard, its role is similar to that of an authoritative name server in DNS.
13. In non-preemptive priority queuing, the transmission of a packet is not interrupted once it has begun. In preemptive priority queuing, the transmission of a packet will be interrupted if a higher priority packet arrives before transmission completes. This would mean that portions of the packet would be sent into the network as separate chunks; these chunks would no longer all have the appropriate header fields. For this reason, preemptive priority queuing is not used.
14. A scheduling discipline that is not work conserving is time division multiplexing, whereby a rotating frame is partitioned into slots, with each slot exclusively available to a particular class.
15. FIFO: line at Starbucks. RR: merging traffic (taking 1 vehicle from first lane then 1 from the second, then 1 from the first, and so on). WFQ: ticket counter at airport providing service to 2 people from first class and 1 from economy, and again 2 from first class, and so on.
16. Scalability: Per-flow resource reservation implies the need for a router to process resource reservations and maintain per-flow state for each flow passing through the router. Flexibly service: The Intserv framework provides for a small number of pre-specified service classes.

Chapter 7 Problems

Problem 2

$x(t)$ will ramp up exponentially fast to the TCP available bandwidth due to TCP slow start. Then $x(t)$ will remain roughly constant at the TCP available bandwidth until the client buffer fills. At that time, $x(t)$ will drop to approximately d until the media is sent and the client buffer remains roughly full.

Problem 3

No, they are not the same thing. The client application reads data from the TCP receive buffer and puts it in the client buffer. If the client buffer becomes full, then application will stop reading from the TCP receive buffer until some room opens up in the client buffer.

Problem 4

a) $160 + h$ bytes are sent every 20 msec. Thus the transmission rate is

$$\frac{(160 + h) \cdot 8}{20} Kbps = (64 + .4h) Kbps$$

b)

IP header: 20bytes
UDP header: 8bytes
RTP header: 12bytes

$h=40$ bytes (a 25% increase in the transmission rate!)

Problem 5

a) Denote $d^{(n)}$ for the estimate after the n th sample.

$$d^{(1)} = r_4 - t_4$$

$$d^{(2)} = u(r_3 - t_3) + (1 - u)(r_4 - t_4)$$

$$d^{(3)} = u(r_2 - t_2) + (1 - u)[u(r_3 - t_3) + (1 - u)(r_4 - t_4)]$$

$$= u(r_2 - t_2) + (1 - u)u(r_3 - t_3) + (1 - u)^2(r_4 - t_4)$$

$$d^{(4)} = u(r_1 - t_1) + (1 - u)d^{(3)}$$

$$= u(r_1 - t_1) + (1 - u)u(r_2 - t_2) + (1 - u)^2u(r_3 - t_3) + (1 - u)^3(r_4 - t_4)$$

b)

$$d^{(n)} = u \sum_{j=1}^{n-1} (1 - u)^j (r_j - t_j) + (1 - u)^n (r_n - t_n)$$

c)

$$d^{(\infty)} = \frac{u}{1-u} \sum_{j=1}^{\infty} (1-u)^j (r_j - t_j)$$

$$= \frac{1}{9} \sum_{j=1}^{\infty} 9^j (r_j - t_j)$$

The weight given to past samples decays exponentially.

Problem 6

a) Denote $v^{(n)}$ for the estimate after the n th sample. Let $\Delta_j = r_j - t_j$.

$$v^{(1)} = \left| \Delta_4 - d^{(1)} \right| \quad (=0)$$

$$v^{(2)} = u \left| \Delta_3 - d^{(2)} \right| + (1-u) \left| \Delta_4 - d^{(1)} \right|$$

$$v^{(3)} = u \left| \Delta_2 - d^{(3)} \right| + (1-u)v^{(2)}$$

$$= u \left| \Delta_2 - d^{(3)} \right| + u(1-u) \left| \Delta_3 - d^{(2)} \right| + (1-u)^2 \left| \Delta_4 - d^{(1)} \right|$$

$$v^{(4)} = u \left| \Delta_1 - d^{(4)} \right| + (1-u)v^{(3)}$$

$$= u \left| \Delta_1 - d^{(4)} \right| + (1-u)u \left| \Delta_2 - d^{(3)} \right| + u(1-u)^2 \left| \Delta_3 - d^{(2)} \right|$$

$$+ (1-u)^3 \left| \Delta_4 - d^{(1)} \right|$$

$$= u \left[\left| \Delta_1 - d^{(4)} \right| + (1-u) \left| \Delta_2 - d^{(3)} \right| + (1-u)^2 \left| \Delta_3 - d^{(2)} \right| \right]$$

$$+ (1-u)^3 \left| \Delta_4 - d^{(1)} \right|$$

b)

$$v^{(n)} = u \sum_{j=1}^{n-1} (1-u)^{j-1} \left| \Delta_j - d^{(n-j+1)} \right| + (1-u)^n \left| \Delta_n - d^{(1)} \right|$$

Problem 7

a) $r_1 - t_1 + r_2 - t_2 + \dots + r_{n-1} - t_{n-1} = (n-1)d_{n-1}$

Substituting this into the expression for d_n gives

$$d_n = \frac{n-1}{n} d_{n-1} + \frac{r_n - t_n}{n}$$

b) The delay estimate in part (a) is an average of the delays. It gives equal weight to recent delays and to “old” delays. The delay estimate in Section 6.3 gives more weight to recent delays; delays in the distant past have relatively little impact on the estimate.

Problem 8

The two procedures are very similar. They both use the same formula, thereby resulting in exponentially decreasing weights for past samples.

One difference is that for estimating average RTT, the time when the data is sent and when the acknowledgement is received is recorded on the same machine. For the delay estimate, the two values are recorded on different machines. Thus the sample delay can actually be negative.

Problem 9

a) If there is packet lost, then other packets may have been transmitted in between the two received packets.

b) Let S_i denote the sequence number of the i th received packet. If

$$t_i - t_{i-1} > 20msec$$

and

$$S_i = S_{i-1} + 1$$

then packet I begins a new talkspurt.

Problem 10

a) The delay of packet 2 is 7 slots. The delay of packet 3 is 9 slots. The delay of packet 4 is 8 slots. The delay of packet 5 is 7 slots. The delay of packet 6 is 9 slots. The delay of packet 7 is 8 slots. The delay of packet 8 is > 8 slots.

b) Packets 3, 4, 6, 7, and 8 will not be received in time for their playout if playout begins at $t=8$.

c) Packets 3 and 6 will not be received in time for their playout if playout begins at $t=9$.

d) No packets will arrive after their playout time if playout time begins at $t=10$.

Problem 11

The answers to parts a and b are in the table below:

Packet Number	$r_i - t_i$	d_i	v_i
1	7	7	0
2	8	7.10	0.09
3	8	7.19	0.162
4	7	7.17	0.163
5	9	7.35	0.311
6	9	7.52	0.428
7	8	7.57	0.429
8	8	7.61	0.425

Problem 12

a) Both schemes require 25% more bandwidth. The first scheme has a playback delay of 5 packets. The second scheme has a delay of 2 packets.

b) The first scheme will be able to reconstruct the original high-quality audio encoding. The second scheme will use the low quality audio encoding for the lost packets and will therefore have lower overall quality.

c) For the first scheme, many of the original packets will be lost and audio quality will be very poor. For the second scheme, every audio chunk will be available at the receiver, although only the low quality version will be available for every other chunk. Audio quality will be acceptable.

Problem 13

The CDN company provides a mechanism so that when a client requests content, the content is provided by the CDN server that can best deliver the content to the specific client. The server may be the closest CDN server to the client (perhaps in the same ISP as the client) or may be a CDN server with a congestion-free path to the client. In this way, even if a CDN does not increase the amount of link capacity in the network, the performance seen by the hosts is improved.

Problem 14

Yes. If the path between the requesting host and the CDN server, chosen by the CDN, is already too congested or if the chosen CDN server is already too busy, then it might be possible that the host experiences worse performance than what it would have if it contacted the remote server directly.

Problem 15

The inter-arrival jitter J is defined to be the mean deviation (smoothed absolute value) of the difference D in packet spacing at the receiver compared to the sender for a pair of packets. As shown in the equation below, this is equivalent to the “relative transit time”

for the two packets; the relative transit time is the difference between a packet's RTP timestamp and the receiver's clock at the time of arrival. If T_i is the RTP timestamp for packet i and r_i is the time of arrival in RTP timestamp units for packet i , then for two packets i and j , D is defined as

$$D(i, j) = (r_i - r_j) - (s_i - s_j) = (r_j - s_j) - (r_i - s_i)$$

The inter-arrival jitter is calculated continuously as each data packet i is received, using this difference D for that packet and the previous packet $i-1$ in order of arrival (not necessarily in sequence), according to the formula

$$J = J + \frac{|D(i, j) - J|}{16}$$

Whenever a reception report is issued, the current value of J is sampled.

Problem16

- a) As discussed in Chapter 2, UDP sockets are identified by the two-tuple consisting of destination IP address and destination port number. So the two packets will indeed pass through the same socket.
- b) Yes, Alice only needs one socket. Bob and Claire will choose different SSRC's, so Alice will be able distinguish between the two streams. Another question we could have asked is: How does Alice's software know which stream (i.e. SSRC) belongs to Bob and which stream belongs to Alice? Indeed, Alice's software may want to display the sender's name when the sender is talking. Alice's software gets the SSRC to name mapping from the RTCP source description reports.

Problem 17

- a) The session bandwidth is $4 * 100 \text{ kbps} = 400 \text{ kbps}$. Five percent of the session bandwidth is 20 kbps .
- b) Sends each user is both a sender and receiver, each user gets 5 kbps for RTCP packets (receiver reports, sender reports, and source description packets).

Problem 18

- a) Like HTTP, all request and response methods are in ASCII text. RTSP also has methods (SETUP, PLAY, PAUSE), and the server responds with standardized reply codes. Yes, using the GET method, HTTP can be used to request a stream.
- b) RTSP messages use different port numbers than the media streams. Thus RTSP is out-of-band. HTTP objects are sent within the HTTP response message. Thus HTTP is in-band. HTTP does not keep session state: each request is handled independently. RTSP uses the Session ID to maintain session state. For example, in the lab (programming assignment) for this chapter, the RTSP server is in one several states for each session ID. When in the pause state, the server stores the number of the frame at which the pause occurred.

Problem 19

- a) True
- b) True
- c) No, RTP streams can be sent to/from any port number. See the SIP example in Section 6.4.3
- d) No, typically they are assigned different SSRC values.
- e) True
- f) False, she is indicating that she wishes to *receive* GSM audio
- g) False, she is indicating that she wishes to *receive* audio on port 48753
- h) True, 5060 for both source and destination port numbers
- i) True
- j) False, this is a requirement of H.323 and not SIP.

Problem 20

- a) One possible sequence is 1 2 1 3 1 2 1 3 1 2 1 3 ...
Another possible sequence is 1 1 2 1 1 3 1 1 2 1 1 3 1 1 2 1 1 3 ...
- b) 1 1 3 1 1 3 1 1 3 1 1 3 ...

Problem 21

a)

Packet	Time leaving the queue	Delay
1	0	0
2	1	1
3	2	1
4	3	2
5	5	2
6	4	2
7	6	3
8	7	2
9	8	3
10	9	2
11	10	2
12	11	3
Average Delay		1.91

b)

Packet	Time leaving the queue	Delay
1	0	0
2	2	2
3	1	0

4	6	5
5	4	1
6	7	5
7	3	0
8	9	4
9	5	0
10	10	3
11	8	0
12	11	3
Average Delay		1.91

c)

Packet	Time leaving the queue	Delay
1	0	0
2	2	2
3	4	3
4	1	0
5	3	0
6	6	4
7	5	2
8	7	2
9	9	4
10	11	4
11	8	0
12	10	2
Average Delay		1.91

d)

Packet	Time leaving the queue	Delay	Note
1	0	0	WFQ
2	2	2	WFQ
3	1	0	WFQ
4	5	4	WFQ
5	3	0	WFQ
6	7	5	Idealized WFQ scheduling
7	4	1	WFQ
8	9	4	WFQ
9	6	1	Idealized WFQ scheduling
10	10	3	WFQ
11	8	0	WFQ
12	11	3	WFQ
Average Delay		1.91	

e) It can be noticed that the average delay for all four cases is the same (1.91 seconds).

Problem 22

a)

Packet	Time leaving the queue	Delay
1	0	0
2	4	4
3	5	4
4	1	0
5	3	0
6	2	0
7	6	3
8	9	4
9	7	2
10	10	3
11	8	0
12	11	3
Average Delay		1.91

b)

Packet	Time leaving the queue	Delay
1	0	0
2	1	1
3	3	2
4	2	1
5	6	3
6	4	2
7	5	2
8	9	4
9	7	2
10	10	3
11	8	0
12	11	3
Average Delay		1.91

c)

Packet	Time leaving the queue	Delay
1	0	0
2	1	1
3	2	1
4	3	2

5	9	6
6	6	4
7	4	1
8	7	2
9	5	0
10	8	1
11	11	3
12	10	2
Average Delay		1.91

Problem 23

Time Slot	Packets in the queue	Number of tokens in bucket
0	1, 2, 3	2
1	3, 4	1
2	4, 5	1
3	5, 6	1
4	6	1
5	-	1
6	7, 8	2
7	9, 10	1
8	10	1

Time Slot	Packets in output buffer
0	1, 2
1	3
2	4
3	5
4	6
5	-
6	7, 8
7	9
8	10

Problem 24

Time Slot	Packets in the queue	Number of tokens in bucket
0	1, 2, 3	2
1	3, 4	2
2	5	2
3	6	2
4	-	2
5	-	2
6	7, 8	2
7	9, 10	2

8	-	2
---	---	---

Time Slot	Packets in output buffer
0	1, 2
1	3, 4
2	5
3	6
4	-
5	-
6	7, 8
7	9, 10
8	-

Problem 25

No. The answer still remains the same as in Problem 24.

Problem 26

See figure below. For the second leaky bucket, $r = p, b = 1$.

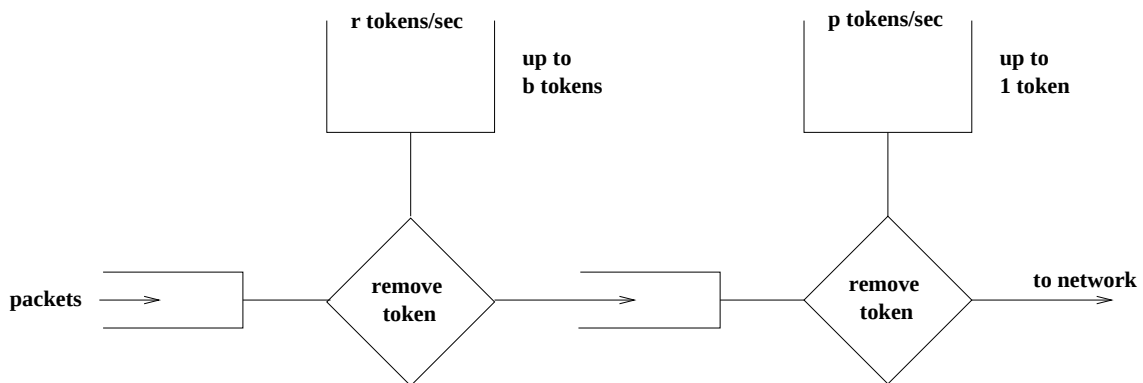


Figure: Solution to problem 26

Problem 27

No.

Problem 28

Let τ be a time at which flow 1 traffic starts to accumulate in the queue. We refer to τ as the beginning of a flow-1 busy period. Let $t > \tau$ be another time in the same flow-1 busy period. Let $T_1(\tau, t)$ be the amount of flow-1 traffic transmitted in the interval $[\tau, t]$. Clearly,

$$T_1(\tau, t) \geq \frac{W_1}{\sum W_j} R(t - \tau)$$

Let $Q_1(t)$ be the amount of flow-1 traffic in the queue at time t . Clearly,

$$Q_1(t) = b_1 + r_1(t - \tau) - T_1(\tau, t)$$

$$\begin{aligned} &\leq b_1 + r_1(t - \tau) + \frac{W_1}{\sum W_j} R(t - \tau) \\ &= b_1 + (t - \tau) \left[r_1 - \frac{W_1}{\sum W_j} R \right] \end{aligned}$$

Since $r_1 < \frac{W_1}{\sum W_j} R$, $Q_1(t) \leq b_1$. Thus the maximum amount of flow-1 traffic in the queue

is b_1 . The minimal rate at which this traffic is served is $\frac{W_1 R}{\sum W_j}$.

Thus, the maximum delay for a flow-1 bit is

$$\frac{b_1}{W_1 R / \sum W_j} = d_{\max}.$$

Chapter 8 Review Questions

Question 1.

Confidentiality is the property that the original plaintext message can not be determined by an attacker who intercepts the ciphertext-encryption of the original plaintext message. Message integrity is the property that the receiver can detect whether the message sent (whether encrypted or not) was altered in transit. The two are thus different concepts, and one can have one without the other. An encrypted message that is altered in transit may still be confidential (the attacker can not determine the original plaintext) but will not have message integrity if the error is undetected. Similarly, a message that is altered in transit (and detected) could have been sent in plaintext and thus would not be confidential.

Question 2.

(i) User's laptop and a web server; (ii) two routers; (iii) two DNS name servers.

Question 3.

One important difference between symmetric and public key systems is that in symmetric key systems both the sender and receiver must know the same (secret) key. In public key systems, the encryption and decryption keys are distinct. The encryption key is known by the entire world (including the sender), but the decryption key is known only by the receiver.

Question 4.

In this case, a known plaintext attack is performed. If, somehow, the message encrypted by the sender was chosen by the attacker, then this would be a chosen-plaintext attack.

Question 5.

An 8-block cipher has 2^8 possible input blocks. Each mapping is a permutation of the 2^8 input blocks; so there are $2^8!$ possible mappings; so there are $2^8!$ possible keys.

Question 6.

If each user wants to communicate with N other users, then each pair of users must have a shared symmetric key. There are $N(N-1)/2$ such pairs and thus there are $N(N-1)/2$ keys. With a public key system, each user has a public key which is known to all, and a private key (which is secret and only known by the user). There are thus $2N$ keys in the public key system.

Question 7.

$a \bmod n = 23$, $b \bmod n = 4$. So $(a*b) \bmod n = 23*4=92$

Question 8.

Question 9.

One requirement of a message digest is that given a message M , it is very difficult to find another message M' that has the same message digest and, as a corollary, that given a message digest value it is difficult to find a message M'' that has that given message digest value. We have “message integrity” in the sense that we have reasonable confidence that given a message M and its signed message digest that the message was not altered since the message digest was computed and signed. This is not true of the Internet checksum, where we saw in Figure 7.18 that it is easy to find two messages with the same Internet checksum.

Question 10.

No. This is because a hash function is a one-way function. That is, given any hash value, the original message cannot be recovered (given h such that $h=H(m)$, one cannot recover m from h).

Question 11.

This scheme is clearly flawed. Trudy, an attacker, can first sniff the communication and obtain the shared secret s by extracting the last portion of digits from $H(m)+s$. Trudy can then masquerade as the sender by creating her own message t and send $(t, H(t)+s)$.

Question 12.

Suppose Bob sends an encrypted document to Alice. To be verifiable, Alice must be able to convince herself that Bob sent the encrypted document. To be non-forgeable, Alice must be able to convince herself that only Bob could have sent the encrypted document (e.g., no one else could have guessed a key and encrypted/sent the document). To be non-reputable, Alice must be able to convince someone else that only Bob could have sent the document. To illustrate the latter distinction, suppose Bob and Alice share a secret key, and they are the only ones in the world who know the key. If Alice receives a document that was encrypted with the key, and knows that she did not encrypt the document herself, then the document is known to be verifiable and non-forgeable (assuming a suitably strong encryption system was used). However, Alice can *not* convince someone else that Bob must have sent the document, since in fact Alice knew the key herself and could have encrypted/sent the document.

Question 13.

A public-key signed message digest is “better” in that one need only encrypt (using the private key) a short message digest, rather than the entire message. Since public key encryption with a technique like RSA is expensive, it's desirable to have to sign (encrypt) a smaller amount of data than a larger amount of data.

Question 14.

This is false. To create the certificate, certifier.com would include a digital signature, which is a hash of foo.com's information (including its public key), and signed with certifier.com's private key.

Question 15.

For a MAC-based scheme, Alice would have to establish a shared key with each potential recipient. With digital signatures, she uses the same digital signature for each recipient; the digital signature is created by signing the hash of the message with her private key. Digital signatures are clearly a better choice here.

Question 16.

The purpose of the nonce is to defend against the replay attack.

Question 17.

Once in a lifetime means that the entity sending the nonce will never again use that value to check whether another entity is "live".

Question 18.

In a man-in-the-middle attack, the attacker puts himself between Alice and Bob, altering the data sent between them. If Bob and Alice share a secret authentication key, then any alterations will be detected.

Question 19.

Alice provides a digital signature, from which Bob can verify that message came from Alice. PGP uses digital signatures, not MACs, for message integrity.

Question 20.

False. SSL uses implicit sequence numbers.

Question 21.

The purpose of the random nonces in the handshake is to defend against the connection replay attack.

Question 22.

True. The IV is always sent in the clear. In SSL, it is sent during the SSL handshake.

Question 23.

After the client will generate a pre-master secret (PMS), it will encrypt it with Alice's public key, and then send the encrypted PMS to Trudy. Trudy will not be able to decrypt the PMS, since she does not have Alice's private key. Thus Trudy will not be able to determine the shared authentication key. She may instead guess one by choosing a random key. During the last step of the handshake, she sends to Bob a MAC of all the handshake messages, using the guessed authentication key. When Bob receives the MAC, the MAC test will fail, and Bob will end the TCP connection.

Question 24.

False. Typically an IPsec SA is first established between Host A and Host B. Then all packets in the stream use the SA.

Question 25.

False. IPsec will increment the sequence number for every packet it sends.

Question 26.

False. An IKE SA is used to establish one or more IPsec SAs.

Question 27.

01011100

Question 28.

True

Question 29.

Filter table and connection table. The connection table keeps track of connections, allowing for a finer degree of packet filtering.

Question 30.

True

Question 31.

True

Question 32.

If there isn't a packet filter, than users inside the institution's network will still be able to make direct connections to hosts outside the institution's network. The filter forces the users to first connect to the application gateway.

Question 33.

True

Chapter 8 Problems

Problem 1.

The encoding of "This is an easy problem" is "uasi si my cmiw lokngch".

The decoding of “rmij'u uamu xyj” is “wasn't that fun”.

Problem 2.

If Trudy knew that the words “bob” and “alice” appeared in the text, then she would know the ciphertext for b,o,a,l,i,c,e (since “bob” is the only palindrome in the message, and “alice” is the only 5-letter word. If Trudy knows the ciphertext for 7 of the letters, then she only needs to try $19!$, rather than $26!$, plaintext-ciphertext pairs. The difference between $19!$ and $26!$ is $26*25*24...*20$, which is 3315312000, or approximately 10^9 .

Problem 3.

Every letter in the alphabet appears in the phrase “The quick fox jumps over the lazy brown dog.” Given this phrase in a chosen plaintext attack (where the attacker has both the plain text, and the ciphertext), the Caesar cipher would be broken - the intruder would know the ciphertext character for every plaintext character. However, the Vigenere cipher does not always translate a given plaintext character to the same ciphertext character each time, and hence a Vigenere cipher would not be immediately broken by this chosen plaintext attack.

Problem 4.

- (a) The output is equal to 00000101 repeated eight times.
- (b) The output is equal to 00000101 repeated seven times + 10000101.
- (c) We have $(A^R B^R C^R)^R = CBA$, where A, B, C are strings, and R means inverse operation. Thus:
 - 1. For (a), the output is 10100000 repeated eight times;
 - 2. For (b), the output is 10100001 + 10100000 repeated seven times.

Problem 5.

- (a) There are 8 tables. Each table has 2^8 entries. Each entry has 8 bits.
number of tables * size of each table * size of each entry = $8*2^8*8 = 2^{14}$ bits
- (b) There are 2^{64} entries. Each entry has 64 bits. 2^{71} bits

Problem 6.

- (a) 100100100 ==> 011011011
- (b) Trudy will know the three block plaintexts are the same.
- (c) $c(i) = K_S(m(i) \text{ XOR } c(i-1))$
 - $c(1) = K_S(100 \text{ XOR } 111) = K_S(011) = 100$
 - $c(2) = K_S(100 \text{ XOR } 100) = K_S(000) = 110$
 - $c(1) = K_S(100 \text{ XOR } 110) = K_S(010) = 101$

Problem 7.

- (a) We are given $p = 3$ and $q = 11$. We thus have $n = 33$ and $q = 11$. Choose $e = 9$ (it might be a good idea to give students a hint that 9 is a good value to choose, since the resulting calculations are less likely to run into numerical stability problems than other

choices for e .) since 3 and $(p-1)*(q-1) = 20$ have no common factors. Choose $d = 9$ also so that $e*d = 81$ and thus $e*d - 1 = 80$ is exactly divisible by 20. We can now perform the RSA encryption and decryption using $n = 33$, $e = 9$ and $d = 9$.

letter	m	$m^{**}e$	ciphertext = $m^{**}e \bmod 33$
d	4	262144	25
o	15	38443359375	3
g	7	40353607	19

ciphertext	$c^{**}d$	$m = c^{**}d \bmod n$	letter
25	38146972265625	4	d
3	19683	15	o
19	322687697779	7	g

(b) We first consider each letter as a 5-bit number: 00100, 01111, 00111. Now we concatenate each letter to get 001000111100111 and encrypt the resulting decimal number $m=4583$. The concatenated decimal number $m (= 4583)$ is larger than current $n (= 33)$. We need $m < n$. So we use $p = 43$, $q = 107$, $n = p*q = 4601$, $z = (p-1)(q-1) = 4452$. $e = 61$, $d = 73$

ciphertext = $m^{**}e \bmod 4601$

$m^{**}e = 21386577601828057804089602156530567188611499869029788733808438804$
 $302864595620613956725840720949764845640956118784875246785033236197777129$
 $730258961756918400292048632806197527785447791567255101894492820972508185$
 769802881718983

ciphertext = $m^{**}e \bmod 4601 = 402$

$c^{**}d$
 $= 1283813313619771634195712132539793287643533147482536209328405262793027$
 $158861012392053287249633570967493122280221453815012934241370540204581459$
 $8714979387232141014703227794586499817945633390592$

ciphertext = $m^{**}e \bmod 4601 = 4583$

Problem 8.

$p = 5$, $q = 11$

(a) $n = p*q = 55$, $z = (p-1)(q-1) = 40$

(b) $e = 3$ is less than n and has no common factors with z .

(c) $d = 27$

(d) $m = 8$, $m^e = 512$, Ciphertext $c = m^e \bmod n = 17$

Problem 9.

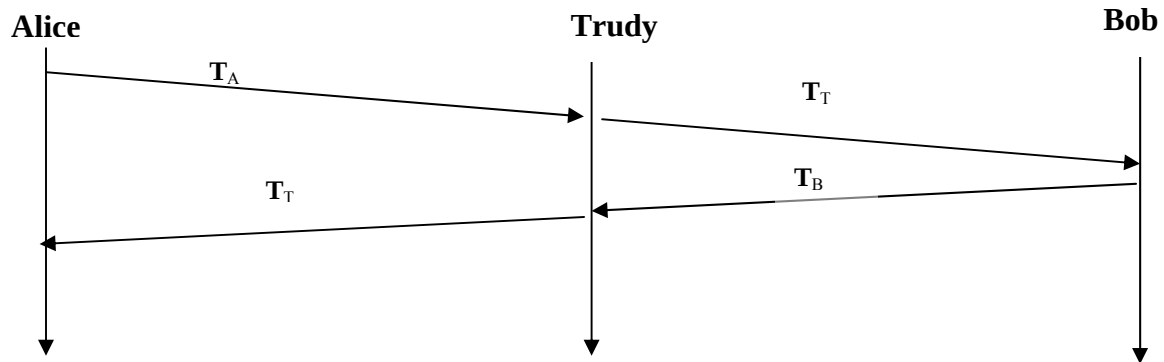
	<u>Alice</u>	<u>Bob</u>
secret key:	S_A	S_B
public key:	$T_A = (g^{S_A}) \bmod p$	$T_B = (g^{S_B}) \bmod p$
shared key:	$S = (T_B^{S_A}) \bmod p$	$S' = (T_A^{S_B}) \bmod p$

(a) $S = (T_B^{S_A}) \bmod p = ((g^{S_B} \bmod p)^{S_A}) \bmod p = (g^{(S_B S_A)}) \bmod p$
 $= ((g^{S_A} \bmod p)^{S_B}) \bmod p = (T_A^{S_B}) \bmod p = S'$

(b and c) $p = 11, g = 2$

	<u>Alice</u>	<u>Bob</u>
secret key:	$S_A = 5$	$S_B = 12$
public key:	$T_A = (g^{S_A}) \bmod p = 10$	$T_B = (g^{S_B}) \bmod p = 4$
shared key:	$S = (T_B^{S_A}) \bmod p = 1$	$S' = (T_A^{S_B}) \bmod p = 1$

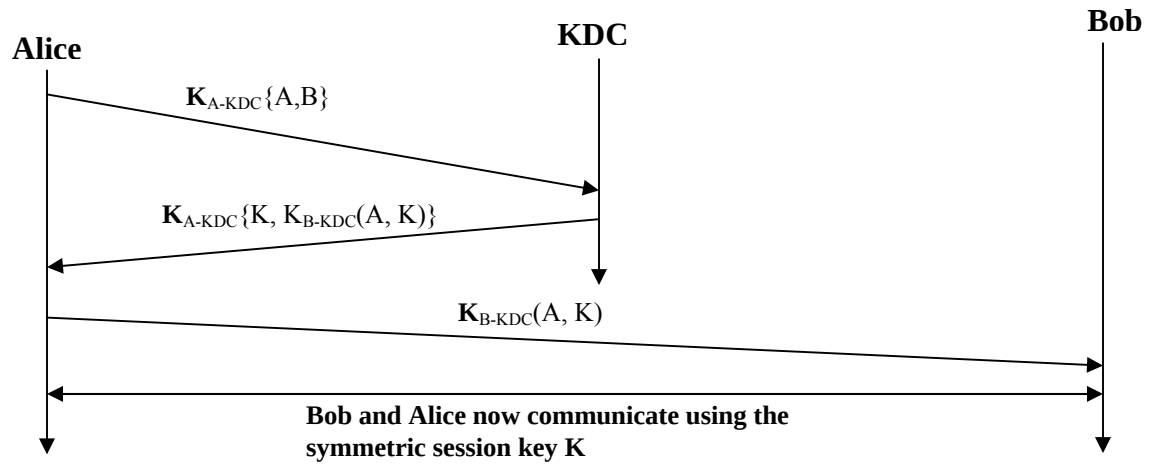
(d)



The Diffie-Hellman public key encryption algorithm is possible to be attacked by man-in-the-middle.

1. In this attack, Trudy receives Alice's public value (T_A) and sends her own public value (T_T) to Bob.
2. When Bob transmits his public value (T_B), Trudy sends her public key to Alice (T_T).
3. Trudy and Alice thus agree on one shared key (S_{AT}) and Trudy and Bob agree on another shared key (S_{BT}).
4. After this exchange, Trudy simply decrypts any messages sent out by Alice or Bob by the public keys S_{AT} and S_{BT} .

Problem 10.



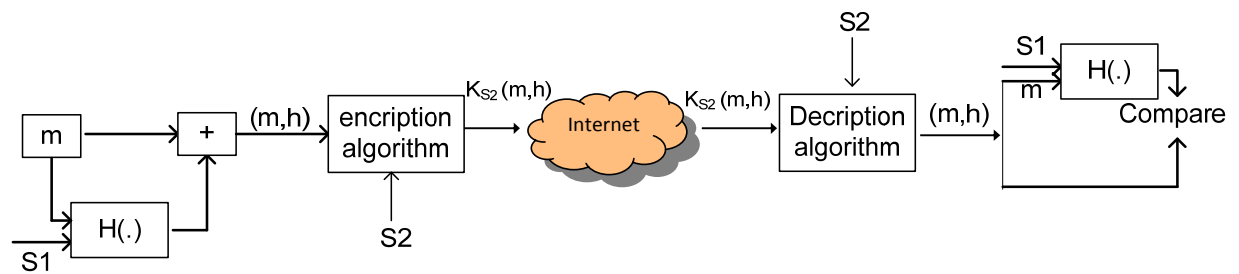
Problem 11.

The message

I	O	U	1
9	0	.	9
0	B	O	B

has the same checksum

Problem 12.



Problem 13.

The file is broken into blocks of equal size. For each block, calculate the hash (for example with MD5 or SHA-1). The hashes for all of the blocks are saved in the .torrent

file. Whenever a peer downloads a block, it calculates the hash of this block and compares it to the hash in the .torrent file. If the two hashes are equal, the block is valid. Otherwise, the block is bogus, and should be discarded.

Problem 14.

Digital signatures require an underlying Public Key Infrastructure (PKI) with certification authorities. For OSPF, all routers are in a same domain, so the administrator can easily deploy the symmetric key on each router, without the need of a PKI.

Problem 15.

Bob does not know if he is talking to Trudy or Alice initially. Bob and Alice share a secret key K_{A-B} that is unknown to Trudy. Trudy wants Bob to authenticate her (Trudy) as Alice. Trudy is going to have Bob authenticate himself, and waits for Bob to start:

1. **Bob-to-Trudy: "I am Bob"** Commentary: Bob starts to authenticate himself. Bob's authentication of himself to the other side then stops for a few steps.
2. **Trudy-to-Bob: "I am Alice"** Commentary: Trudy starts to authenticate herself as Alice
3. **Bob-to-Trudy: "R"** Commentary: Bob responds to step 2 by sending a nonce in reply. Trudy does not yet know $K_{A-B}(R)$ so she can not yet reply.
4. **Trudy-to-Bob: "R"** Commentary: Trudy responds to step 1 now continuing Bob's authentication, picking as the nonce for Bob to encrypt, *the exact same value that Bob sent her to encrypt in Step 3.*
5. **Bob-to-Trudy: " $K_{A-B}(R)$ "** Bob completes his own authentication of himself to the other side by encrypting the nonce he was sent in step 4. Trudy now has $K_{A-B}(R)$. (Note: she does not have, nor need, K_{A-B})
6. **Trudy-to-Bob: " $K_{A-B}(R)$ "** Trudy completes her authentication, responding to the R that Bob sent in step 3 above with $K_{A-B}(R)$. Since Trudy has returned the properly encrypted nonce that Bob sent in step 3, Bob thinks Trudy is Alice!

Problem 16.

This wouldn't really solve the problem. Just as Bob thinks (incorrectly) that he is authenticating Alice in the first half of Figure 7.14, so too can Trudy fool Alice into thinking (incorrectly) that she is authenticating Bob. The root of the problem that neither Bob nor Alice can tell is the public key they are getting is indeed the public key of Alice of Bob.

Problem 17.

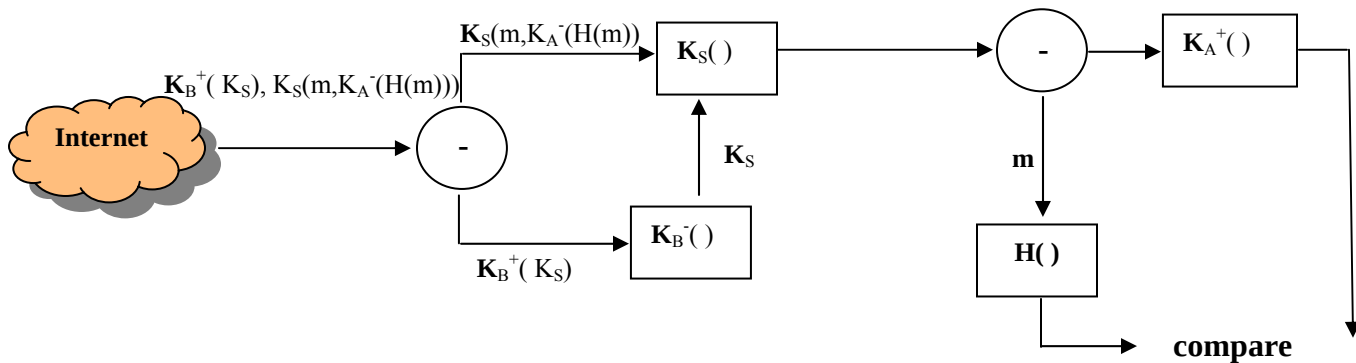


Figure: Operations performed by Bob for confidentiality, integrity, and authentication

Problem 18

- (a) No, without a public-private key pair or a pre-shared secret, Bob cannot verify that Alice created the message.
- (b) Yes, Alice simply encrypts the message with Bob's public key and sends the encrypted message to Bob.

Problem 19

- a) Client
- b) IP: 216.75.194.220, port: 443
- c) 283
- d) 3 SSL records
- e) Yes, it contains an encrypted master secret
- f) First byte: bc; Last byte: 29
- g) 6

Problem 20.

Again we suppose that SSL does not provide sequence numbers. Suppose that Trudy, a woman-in-the-middle, deletes a TCP segment. So that Bob doesn't anything, Trudy needs to also adjust the sequence numbers in the subsequent packets sent from Alice to Bob, and the acknowledgment numbers sent from Bob to Alice. The result will be that Bob will, unknowingly, be missing a packet's worth of bytes in the byte stream.

Problem 21.

No, the bogus packet will fail the integrity check (which uses a shared MAC key).

Problem 22.

- (a) F
- (b) T
- (c) T
- (d) F

Problem 23.

If Trudy does not bother to change the sequence number, R2 will detect the duplicate when checking the sequence number in the ESP header. If Trudy increments the sequence number, the packet will fail the integrity check at R2.

Problem 24.

a) Since IV = 11, the key stream is 111110100000

Given, $m = 10100000$

Hence, $ICV = 1010 \text{ XOR } 0000 = 1010$

The three fields will be:

IV: 11

Encrypted message: $10100000 \text{ XOR } 11111010 = 01011010$

Encrypted ICV: $1010 \text{ XOR } 0000 = 1010$

b) The receiver extracts the IV (11) and generates the key stream 111110100000

XORs the encrypted message with the key stream to recover the original message:

$01011010 \text{ XOR } 11111010 = 10100000$

XORs the encrypted ICV with the keystream to recover the original ICV:

$1010 \text{ XOR } 0000 = 1010$

The receiver then XORs the first 4 bits of recovered message with its last 4 bits:

$1010 \text{ XOR } 0000 = 1010$ (which equals the recovered ICV)

c) Since the ICV is calculated as the XOR of first 4 bits of message with last 4 bits of message, either the 1st bit or the 5th bit of the message has to be flipped for the received packet to pass the ICV check.

d) For part (a), the encrypted message was 01011010

Flipping the 1st bit gives, 11011010

Trudy XORs this message with the keystream:

$11011010 \text{ XOR } 11111010 = 00100000$

If Trudy flipped the first bit of the encrypted ICV, the ICV value received by the receiver is 0010

The receiver XORs this value with the keystream to get the ICV:

0010 XOR 0000 = 0010

The receiver now calculates the ICV from the recovered message:

0010 XOR 0000 = 0010 (which equals the recovered ICV and so the received packet passes the ICV check)

Problem 25.

Filter Table:

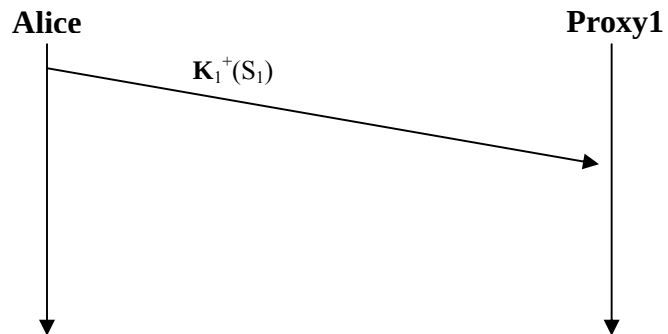
Action	Source address	Dest address	Protocol	Source port	Dest port	Flag bit	Check connection
allow	222.22/16	outside of 222.22/16	TCP	> 1023	22	any	
allow	outside of 222.22/16	222.22/16	TCP	22	> 1023	ACK	x
Allow	outside of 222.22/16	222.22.0.12	TCP	>1023	80	any	
Allow	222.22.0.12	outside of 222.22/16	TCP	80	>1023	any	
deny	All	all	all	all	all	all	

Connection Table:

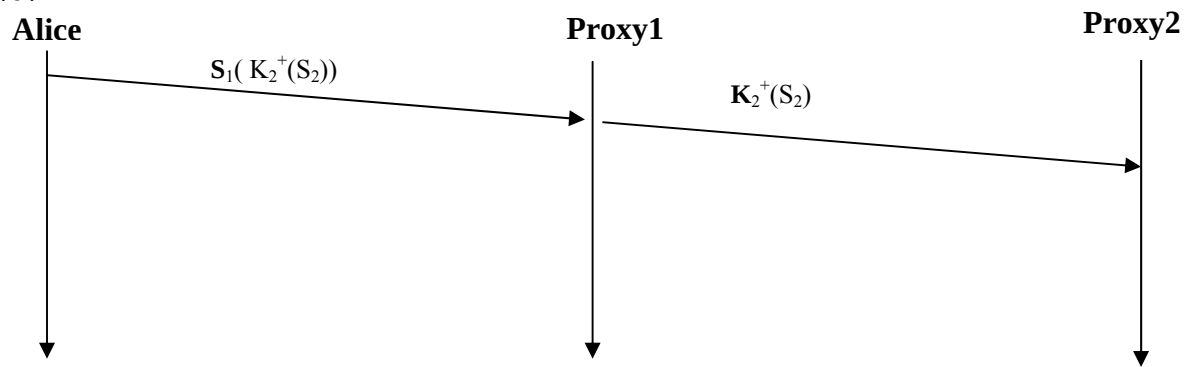
Source address	Dest address	Source port	Dest port
222.22.1.7	37.96.87.123	12699	22
222.22.93.2	199.1.205.23	37654	22
222.22.65.143	203.77.240.43	48712	22

Problem 26.

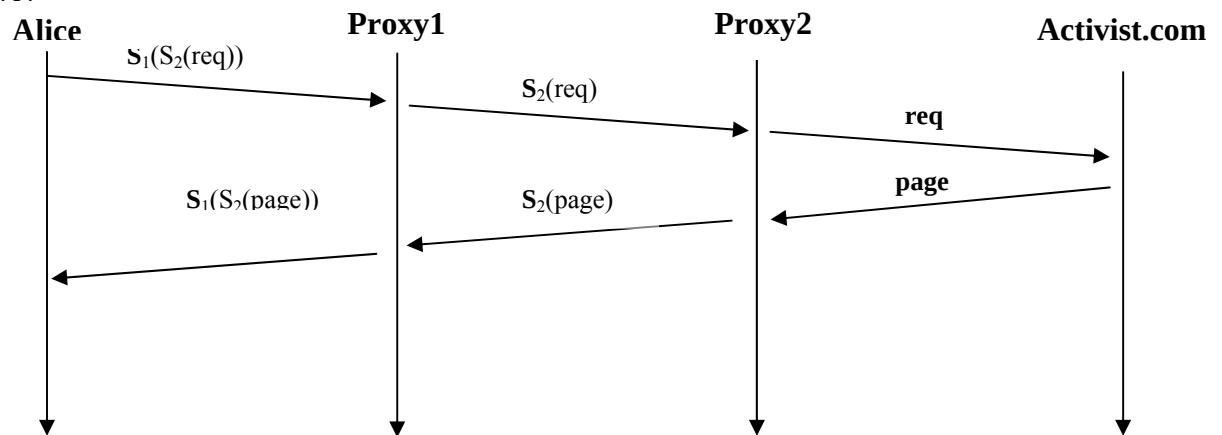
(a)



(b)



(c)



Chapter 9 Review Questions

Question 1.

A network manager would like to have network management capabilities when (a) a component of the network fails, (b) a component of the network is about to fail, and is acting “flaky” (c) a component of the network has been compromised from a security standpoint and is attacking the network, e.g., by launching a DOS attack by flooding the network with packets, (d) traffic levels exceed a certain threshold on a link, causing packets to be dropped, (e) everything is running smoothly (in order to *know* that everything is running smoothly and there are no problems). There are many additional reasons as well.

Question 2.

Performance management, fault management, configuration management, accounting management, security management.

Question 3.

Network management is more narrowly defined, as it focuses on the resources in the network – monitoring their functions and controlling their operation. These resources are combined (used) in various ways to implement services. Note that while the network resources may all be functioning as they should, they may not be sufficient to implement a service with a given level of performance; this latter concern is an aspect of service management.

Question 4.

- **Managing entity:** control the collection, processing, analysis, display of network management information, and is used by the network manager to control the devices in the network.
- **Managed device:** a piece of network equipment that is under the control of the managing entity.
- **Management agent:** a software process running on a managed device that communicated with the managing entity and takes action on the managed device under the control of the managing entity.
- **MIB:** pieces of information associated with all of the managed objects in a device.
- **Network management protocol:** runs between the managing entity of the management agents on the managed devices, allowing the agents to alert the managing entity to potential problems, and allowing the managing entity to send commands to the management agents.

Question 5.

The SMI is a data-definition language used to defined the pieces of information in an SNMP MIB.

Question 6.

The trap message is sent by the management agent to the managing entity (and requires no response from the managing entity). A request-response message is sent by the managing entity, with the response coming back from the management agent.

Question 7.

GetRequest, GetNextRequest, GetBulkRequest, SetRequest, InformRequest, Response, Trap

Question 8.

The SNMP engine is the part of an SNMP implementation that handles the dispatching, processing, authentication, access control, and timeliness of the SNMP messages. See Figure 8.5.

Question 9.

The ASN.1 object identifier tree provides a standard way to name objects.

Question 10.

The role of the presentation layer is to allow the sending and receiving of data in a machine-independent format (i.e., without regard to the particular storage and architectural conventions of the sender and receiver).

Question 11.

The Internet does not have presentation layer. It is up to the implementers of protocols to make sure that data is sent and received in the desired format.

Question 12.

In TLV encoding, each piece of data is tagged with its type, length, and value.

Chapter 9 Problems**Problem 1.**

Request response mode will generally have more overhead (measured in terms of the number of messages exchanged) for several reasons. First, each piece of information received by the manager requires two messages: the poll and the response. Trapping generates only a single message to the sender. If the manager really only wants to be notified when a condition occurs, polling has more overhead, since many of the polling messages may indicate that the waited-for condition has not yet occurred. Trapping generates a message only when the condition occurs.

Trapping will also immediately notify the manager when an event occurs. With polling, the manager needs will need to wait for half a polling cycle (on average) between when the event occurs and the manager discovers (via its poll message) that the event has occurred.

If a trap message is lost, the managed device will not send another copy. If a poll message, or its response, is lost the manager would know there has been a lost message (since the reply never arrives). Hence the manager could repoll, if needed.

Problem 2.

Often, the time when network management is most needed is in times of stress, when the network may be severely congested and packets are being lost. With SNMP running over TCP, TCP's congestion control would cause SNMP to back-off and stop sending messages at precisely the time when the network manager needs to send SNMP messages.

Problem 3.

1.3.6.1.2.1.5

Problem 4.

Microsoft file formats are under 1.2.840.113556.4 (see section 8.3.2)

Problem 6.

45Julia22 '00000001' '00010100'

Problem 7

45Child 21 '10100000'